**UNIVERSITY OF GHANA**

College of Basic and Applied Sciences
Department of Computer Science

**CSCD602 — Advanced Software Engineering**

Individual Project-Based Examination

# SusuBook

A Digital Susu Collection and Accountability System

| | |
|---|---|
| Student | [STUDENT NAME] |
| Student ID | [STUDENT ID] |
| Examiner | Prof. Solomon Mensah |
| Live application | https://susubook-fdtbppd7sq-uc.a.run.app |
| Source repository | https://github.com/tee-jay7/susubook |

## Contents

# Mapping to Examination Requirements

Examination Part A §10 lists nineteen required sections. This table shows where each is answered in this document.

| Exam § | Required section | Where in this document |
|---|---|---|
| 1 | Project title | Title page, and *Problem Definition* §1.3 |
| 2 | Problem statement | *Problem Definition* §1.2–1.3 |
| 3 | Aim and objectives | *Problem Definition* §1.5–1.6 |
| 4 | Stakeholders | *Stakeholder Analysis* §2.1–2.6 |
| 5 | Requirements analysis | *Requirements Analysis* §3.1–3.8 |
| 6 | SRS | *Software Requirements Specification* — also supplied as `SRS.pdf` |
| 7 | Software effort estimation | *Software Effort Estimation* §4.1–4.9 |
| 8 | System analysis | *System Analysis and Design*, Part A (§7.1–7.5) |
| 9 | System design | *System Analysis and Design*, Part B (§7.6–7.17) |
| 10 | Implementation | *Implementation* §10.1–10.8 |
| 11 | Testing | *Testing and Quality Assurance* §9.1–9.10 — also `Testing_Report.pdf` |
| 12 | Technical debt | *Technical Debt Identification and Management* §8.1–8.8 — also `Technical_Debt_Plan.pdf` |
| 13 | Deployment | *Deployment* §12.1–12.9 |
| 14 | User manual | *SusuBook User Manual* — also `User_Manual.pdf` |
| 15 | Maintenance strategy | *Maintenance Strategy and Future Evolution*, Part A (§11.1–11.3) |
| 16 | Future evolution | *Maintenance Strategy and Future Evolution*, Part B (§11.4–11.7), including the technical debt repayment plan at §11.6 |
| 17 | Limitations | *Limitations, Conclusion and References* §17 |
| 18 | Conclusion | *Limitations, Conclusion and References* §18 |
| 19 | References | *Limitations, Conclusion and References* §19 |

**A note on numbering.** Each part of this document keeps its own section numbers (§4.x for estimation, §9.x for testing, and so on), because every cross-reference in the text — and there are more than eighty — is written against them, and because four parts each answer more than one examination section. Those internal numbers therefore do not run consecutively through the document. The banner above each part states which examination section it satisfies, and this table is the index.

## Supplementary material

| Item | Location |
|---|---|
| Requirements change log (CR-001) | Appendix B |
| Project scope definition | Appendix A |
| UML diagrams — rendered and source | `Supporting_Files/diagrams/` |
| Application screenshots | `Supporting_Files/screenshots/` |
| Live application, credentials, repository | `Deployment_and_Source_Links.txt` |

# 1. Problem Definition

**Project title:** SusuBook — A Digital Susu Collection and Accountability System **Course:** CSCD602 Advanced Software Engineering **Assessment:** Individual Project-Based Examination (48 hours)

## 1.1 Domain background

*Susu* is Ghana's dominant informal savings mechanism. In the **collector model**, a susu collector visits a client daily at their place of business and receives a small, fixed contribution — commonly the same amount every day. The collector records the payment by marking one box on a paper **susu card** that typically carries 31 boxes, one per day of the contribution cycle. At the end of the cycle the client receives the accumulated sum, less **one day's contribution**, which is retained by the collector as commission.

The clients are overwhelmingly people who are excluded from, or poorly served by, formal banking: market women, kiosk operators, small shop owners, artisans and transport workers. They deal in cash, earn daily rather than monthly, and save in amounts too small for a conventional bank account to be worthwhile. Collectors operate either independently or as mobilisation officers attached to a rural bank or microfinance institution.

## 1.2 The problem

The mechanism works because of trust, and its record-keeping cannot support that trust.

**The record is held by the party it is meant to hold accountable.** The paper card is carried and marked by the collector. The client has no independent copy of their own contribution history. A card can be altered, lost, damaged by weather, or simply disputed — and when it is, the client has no evidence.

This produces four concrete failures:

**P1 — Client has no verifiable record of their savings.** The client's only proof of months of daily contributions is a paper card that the collector marks. If the card is lost or the entries are disputed, the client cannot demonstrate what they paid. The saver carries all the evidentiary risk.

**P2 — Collector absconding and misappropriation.** Because cash is collected in the field and recorded only on paper, a collector can under-record collections, delay remittance, or disappear with an entire route's takings. This is the most damaging failure mode in the sector: it destroys the savings of the people least able to absorb the loss, and it deters others from saving at all.

**P3 — Supervising institutions cannot reconcile in real time.** A rural bank employing several collectors learns of a shortfall only when cash is banked, or later. There is no same-day view of *what was recorded in the field* against *what was remitted at the branch*, so variances surface late, when recovery is hardest.

**P4 — Manual computation of payouts is error-prone and opaque.** Days paid, days missed, total accumulated, commission deducted and the final payout are all computed by hand from ticked boxes. Errors are easy, disputes are hard to settle, and the client cannot independently check the arithmetic.

## 1.3 Problem statement

> Susu collection in Ghana relies on a paper card that is held and marked by the collector, leaving the client with no independent record of their own savings. This asymmetry enables under-recording and misappropriation of client funds, prevents supervising institutions from reconciling field collections against cash remitted on the same day, and makes payout computation manual, error-prone and unverifiable. The result is financial loss for savers who can least afford it and an erosion of trust in the informal savings sector.

## 1.4 Proposed solution

SusuBook replaces the paper card with a shared digital record that **both parties can see and neither can silently alter**.

1. **An independent client record.** Every collection is recorded against a client and is immediately visible to that client on their own login, with the amount, the date, the time it was recorded and the identity of the recording collector. The client no longer depends on the collector's copy.
2. **Daily float reconciliation.** At the end of each collection day the collector declares the cash remitted. The system compares this against the sum of collections recorded in the field and raises a variance for supervisor attention the same day.
3. **Automated, transparent payout computation.** Days paid, days missed, total collected, the one-day commission and the net payout are derived by the system from the collection record, using the same rule for every client.
4. **An append-only audit trail.** Every collection, payout and adjustment is attributed to a user and a timestamp and cannot be edited in place; corrections are recorded as new, linked entries.

## 1.5 Aim

To design, implement, test and deploy a functional web-based susu collection system that gives clients an independent and verifiable record of their contributions, and gives supervising institutions same-day reconciliation of field collections against cash remitted.

## 1.6 Objectives

| # | Objective |
|---|---|
| O1 | Elicit, analyse, specify and prioritise the requirements of a susu collection system and document them in an SRS. |
| O2 | Estimate the software effort required using an appropriate technique, and use that estimate to define a scope achievable within the 48-hour examination window. |
| O3 | Design a maintainable layered architecture, expressed through appropriate UML artefacts. |
| O4 | Implement the prioritised requirements as a functional, deployed web application with authentication, role-based authorisation, input validation and an audit trail. |
| O5 | Verify the system through unit, integration, system and user acceptance testing, and document the results. |
| O6 | Identify, classify, prioritise and document the technical debt introduced under the time constraint, with a repayment plan. |
| O7 | Define a maintenance strategy and a future evolution plan grounded in software evolution theory. |

## 1.7 Intended users

| User | Description |
|---|---|
| **Client** | A market trader, kiosk operator, shop owner or artisan who contributes a fixed amount daily and needs a trustworthy record of it. |
| **Collector** | A field officer who visits clients daily, records collections and remits cash to the branch. |
| **Supervisor** | A branch or field manager who oversees several collectors, reconciles daily remittances and investigates variances. |
| **Administrator** | Manages user accounts, roles and institutional settings such as cycle length and commission policy. |

## 1.8 Scope boundary

SusuBook digitises the **record of collection**, not the movement of money. Contributions remain physical cash handed to the collector; the system records that event, computes its consequences and makes it visible to all parties. Mobile money and bank payment integration are deliberately excluded — see `06-scope.md` for the reasoning and `11-maintenance-evolution.md` for their treatment as future evolution.

---

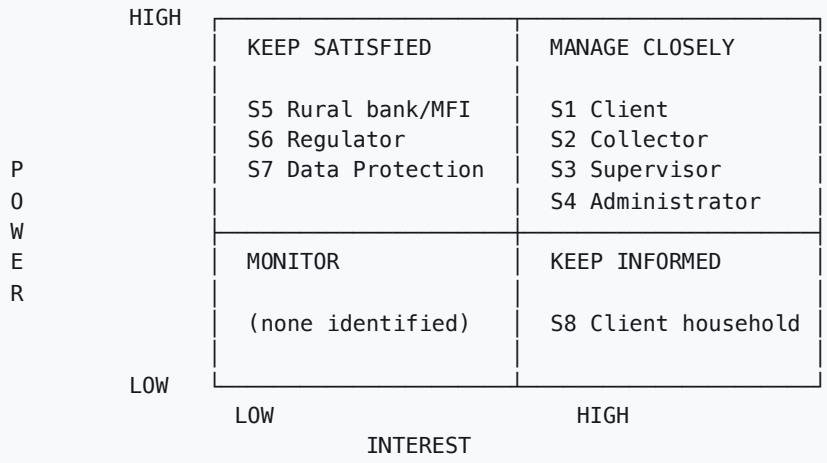
> **Note for final submission:** §1.1–1.2 describe the domain qualitatively. Where the submitted document would benefit from published figures on susu participation or losses in Ghana's informal savings sector, cite a verifiable source (e.g. Bank of Ghana or GSS publications) in `References`. No statistics have been asserted here without a source.

# 2. Stakeholder Analysis

## 2.1 Stakeholder register

| ID | Stakeholder | Type | Interest in the system | Primary concern |
|---|---|---|---|---|
| S1 | **Client (saver)** | Primary, direct user | Wants a record of their savings that they control and can verify | "Can I prove what I paid?" |
| S2 | **Susu Collector** | Primary, direct user | Records collections in the field; needs speed on a route | "Can I record a payment in seconds while standing in the market?" |
| S3 | **Field Supervisor** | Primary, direct user | Reconciles collectors' recorded collections against cash remitted | "Did the money that was recorded actually reach the branch today?" |
| S4 | **System Administrator** | Secondary, direct user | Manages accounts, roles and institutional policy | "Are the right people able to do only the right things?" |
| S5 | **Rural bank / MFI** | Secondary, indirect | Owns the collection operation and bears the loss on fraud | "Are our client funds and our reputation protected?" |
| S6 | **Regulator (Bank of Ghana)** | External | Oversight of deposit-taking and microfinance conduct | "Is there an auditable record of client funds?" |
| S7 | **Data Protection Commission** | External | Enforces the Data Protection Act 2012 (Act 843) | "Is personal data lawfully collected and minimised?" |
| S8 | **Client's household** | Indirect beneficiary | Depends on the savings maturing intact | "Will the money be there when we need it?" |

## 2.2 Power / interest grid




**Implication for the project.** S1, S2 and S3 are the *manage closely* group and are the three roles the system implements. Their conflicting needs drive the central design tension recorded in §2.4.

## 2.3 Elicitation techniques applied

Following Session 2 (Advanced Requirements Elicitation Techniques):

| Technique | How it was applied | Output |
|---|---|---|
| **Artefact analysis** | The existing paper susu card was analysed as the current system's data model: 31 boxes, one per day; client name and daily rate on the header; collector's signature. | The card's structure became the `ContributionCycle` / `Collection` model (FR-11, FR-12). |
| **Process observation (documentary)** | The established susu collection workflow — route visit → cash handed over → box marked → end-of-day remittance to branch → maturity payout less one day — was traced end to end. | Workflow requirements FR-08, FR-17, FR-18, FR-13. |
| **Analysis of failure modes** | The known ways the manual process fails (P1–P4 in `01-problem-definition.md`) were treated as the source of the system's differentiating requirements. | Transparency and audit requirements FR-19 to FR-22. |
| **Expert/analyst judgement** | Applied where stakeholder access was unavailable, and recorded as an assumption rather than a finding. | Default cycle length, commission policy, arrears tolerance. |

### Elicitation limitation (declared)

Primary elicitation with live stakeholders — interviews with collectors and clients, or a JAD workshop with branch staff — **was not conducted**, because the 48-hour examination window does not permit field access. The requirements below are therefore derived from analysis of a well-documented existing manual process, not from primary data. Every point where a real stakeholder would have been consulted is recorded as a numbered assumption in §2.5 so that it can be validated later. This limitation is carried forward into `Limitations` in the final project document.

## 2.4 Conflicting stakeholder needs

Requirements engineering is largely the work of resolving these, and two genuine conflicts shape the design:

**C1 — Collector speed vs. client verifiability.** The collector (S2) needs to record a payment in seconds while standing in a market, in sunlight, on mobile data. The client (S1) needs every collection to be attributable, timestamped and immutable. Strong controls slow the collector down; weak controls destroy the client's guarantee.

*Resolution:* the recording action is reduced to a single confirmation on a pre-populated amount (the client's agreed daily rate), while attribution, timestamping and audit-logging happen server-side without any additional collector input. Integrity is obtained without adding a single tap. → NFR-02, NFR-09.

**C2 — Correction of genuine mistakes vs. immutability of the record.** Collectors will make honest errors — wrong client, wrong date. S2 needs to fix them. S1, S5 and S6 need the record to be non-repudiable.

*Resolution:* records are never edited or deleted in place. A correction is a new, linked reversal entry attributed to the user who made it, leaving both the original and the correction visible. → NFR-04, FR-20.

**C3 — Data minimisation vs. operational usefulness.** S7 requires that personal data be minimised under Act 843. S2 and S3 want enough detail to identify a client on a route.

*Resolution:* the system stores name, phone number, business type and market location, and nothing more — no Ghana Card number, no address, no next of kin. → NFR-06.

## 2.5 Assumptions requiring stakeholder validation

These are the points at which a real stakeholder would have been consulted. Each is stated so that it can be challenged and corrected without redesigning the system.

| ID | Assumption | Would be validated by | Risk if wrong |
|---|---|---|---|
| A1 | The contribution cycle is 31 days. | Collector, branch manager | Low — cycle length is a configurable institutional setting. |
| A2 | Commission equals exactly one day's contribution. | Branch manager, client | Low — commission policy is configurable. |
| A3 | Clients may pay for several missed days at once ("catch-up"). | Collector | Medium — affects the collection allocation rule (FR-10). |
| A4 | A client belongs to exactly one collector at a time. | Branch manager | Medium — a many-to-many relationship would change the data model. |
| A5 | Clients have a phone capable of accessing a mobile web page, or can view via an agent. | Client | **High** — if false, the client-transparency feature (the system's core value) needs an SMS channel instead. |
| A6 | Early withdrawal before maturity is permitted, at the institution's discretion. | Branch manager | Low — feature is gated behind supervisor approval. |

> **A5 is the assumption on which the value proposition rests** and is treated accordingly: the SMS notification channel is specified as the mitigation and is carried in the future evolution plan rather than dropped.

## 2.6 Stakeholder-derived quality expectations

| Stakeholder | Expectation | Becomes |
|---|---|---|
| S1 Client | "I can see every payment I made, and who recorded it." | FR-21, FR-22 |
| S1 Client | "The screen is readable in sunlight and easy if I read slowly." | NFR-08 |
| S2 Collector | "Recording is fast and works on a weak network." | NFR-01, NFR-02 |
| S3 Supervisor | "I see today's variance today, not next week." | FR-19 |
| S4 Admin | "A collector cannot see another collector's route." | NFR-03 |
| S5 Bank | "Nothing can be changed without leaving a trace." | NFR-04, NFR-09 |
| S7 DPC | "Only necessary personal data is held." | NFR-06 |

# 3. Requirements Analysis

Requirements are classified using the taxonomy from Session 2: business, stakeholder, functional, non-functional, and constraints.

## 3.1 Business requirements (why the system is being built)

| ID | Requirement |
|---|---|
| BR-01 | Reduce loss of client funds through under-recording or misappropriation of field collections. |
| BR-02 | Give every client an independent, verifiable record of their own contributions. |
| BR-03 | Enable same-day reconciliation of field collections against cash remitted to the branch. |
| BR-04 | Eliminate manual arithmetic errors in payout computation. |
| BR-05 | Produce an auditable trail of client funds sufficient for institutional and regulatory oversight. |
| BR-06 | Increase saver confidence in the collector model, supporting client retention and growth. |

## 3.2 Stakeholder requirements (what each group expects)

| ID | Stakeholder | Requirement |
|---|---|---|
| SR-01 | Client | See every contribution recorded against me, with date, amount and recording officer. |
| SR-02 | Client | Know my current balance and what I will receive at maturity, without asking anyone. |
| SR-03 | Client | Be confident that a recorded entry cannot later be silently removed. |
| SR-04 | Collector | Record a client's daily contribution in a few seconds on a phone. |
| SR-05 | Collector | See my route for today and who I have not yet collected from. |
| SR-06 | Collector | Correct an honest mistake without needing a developer. |
| SR-07 | Supervisor | Compare what my collectors recorded today against what they remitted today. |
| SR-08 | Supervisor | Be alerted to a variance on the day it occurs. |
| SR-09 | Supervisor | Authorise payouts and early withdrawals. |
| SR-10 | Administrator | Create, suspend and assign roles to users. |
| SR-11 | Administrator | Ensure a collector can access only their own clients. |

## 3.3 Functional requirements (what the system must do)

### Authentication and authorisation

| ID | Requirement | Priority |
|---|---|---|
| FR-01 | The system shall authenticate users with a phone number or email and a password. | Must |
| FR-02 | The system shall support four roles — Client, Collector, Supervisor, Administrator — and enforce role-based access on every route. | Must |
| FR-03 | The system shall store passwords only as salted one-way hashes. | Must |
| FR-04 | The system shall terminate a session on logout and after a period of inactivity. | Must |
| FR-05 | The system shall restrict a Collector to viewing and modifying only clients assigned to them. | Must |

### Client and enrolment management

| ID | Requirement | Priority |
|---|---|---|
| FR-06 | A Collector shall enrol a client, capturing name, phone number, business type, market/location and agreed daily contribution amount. | Must |
| FR-07 | The system shall assign each client to exactly one Collector at a time. | Must |
| FR-08 | A Collector shall view and search their own client list. | Must |
| FR-09 | A Supervisor shall reassign a client from one Collector to another, recording the change in the audit log. | Should |

## Contribution cycles and collection

| ID | Requirement | Priority |
|---|---|---|
| FR-10 | The system shall open a contribution cycle of the configured length (default 31 days) when a client is enrolled and again after each payout. | Must |
| FR-11 | A Collector shall record a contribution against a client for a given date. | Must |
| FR-12 | The system shall reject a second contribution recorded for the same client on the same date. | Must |
| FR-13 | The system shall reject a contribution dated in the future or before the cycle start date. | Must |
| FR-14 | The system shall reject a contribution recorded against a cycle that is already closed or paid out. | Must |
| FR-15 | The system shall accept a catch-up contribution covering several unpaid days, allocating it to specific dates. | Should |
| FR-16 | The system shall display a digital susu card showing each day of the cycle as paid, missed or pending. | Must |
| FR-17 | The system shall compute, for any cycle, the days paid, days missed, total collected and projected payout. | Must |

## Payout

| ID | Requirement | Priority |
|---|---|---|
| FR-18 | At cycle maturity the system shall compute the payout as total collected less one day's contribution retained as commission. | Must |
| FR-19 | The system shall retain the whole balance as commission and pay zero where the total collected does not exceed one day's contribution. | Must |
| FR-20 | A Supervisor shall release a matured payout, closing the cycle and opening the next. | Must |
| FR-21 | The system shall reject any attempt to release a payout for a cycle already paid out. | Must |
| FR-22 | A Client shall request an early withdrawal before maturity, subject to Supervisor approval. | Should |

### Reconciliation and oversight

| ID | Requirement | Priority |
|---|---|---|
| FR-23 | The system shall produce a daily collection sheet per Collector listing clients due and their collection status. | Must |
| FR-24 | A Collector shall declare the total cash remitted to the branch for a given day. | Must |
| FR-25 | The system shall compute the variance between contributions recorded and cash declared for each Collector each day. | Must |
| FR-26 | The system shall present a Supervisor with all non-zero variances for the current day. | Must |
| FR-27 | A Supervisor shall record the resolution of a variance, with a note. | Could |

### Client transparency

| ID | Requirement | Priority |
|---|---|---|
| FR-28 | A Client shall log in and view their own digital card, showing every contribution with amount, date, time recorded and recording Collector. | Must |
| FR-29 | A Client shall view their running balance, cycle maturity date and projected net payout. | Must |
| FR-30 | The system shall issue a unique reference for every recorded contribution. | Must |
| FR-31 | The system shall notify a Client by SMS on each recorded contribution. | Won't (this release) |

### Audit and correction

| ID | Requirement | Priority |
|---|---|---|
| FR-32 | The system shall record every contribution, payout, reassignment and adjustment in an append-only audit log with actor, action, target and timestamp. | Must |
| FR-33 | The system shall never edit or delete a contribution in place; a correction shall be recorded as a new linked reversal entry. | Must |
| FR-34 | A Supervisor shall view the audit trail for any client or collector. | Should |

### Administration

| ID | Requirement | Priority |
|---|---|---|
| FR-35 | An Administrator shall create, suspend and assign roles to user accounts. | Should |
| FR-36 | An Administrator shall configure institutional defaults — cycle length and commission policy. | Could |
| FR-37 | The system shall provide a Supervisor dashboard summarising active clients, today's collections and outstanding variances. | Could |
| FR-38 | The system shall export a collector's cycle report as CSV. | Could |

### Client identification *(added by CR-001)*

| ID | Requirement | Priority |
|---|---|---|
| FR-39 | The system shall assign each client an opaque, non-sequential public reference and render it as a printable QR code encoding the client's contribution URL. | Should |
| FR-40 | The system shall resolve a scanned client reference to that client's contribution screen, subject to the same authorisation as any other route. | Should |

## 3.4 Non-functional requirements (how well it must perform)

| ID | Category | Requirement | Verification |
|---|---|---|---|
| NFR-01 | Performance | Any page shall render within 2 seconds on a 3G-class mobile connection. | Timed page-load test on throttled connection |
| NFR-02 | Usability | A Collector shall be able to record a routine contribution in no more than three interactions from the route sheet. | Task-based usability test, interaction count |
| NFR-03 | Security | Authorisation shall be enforced server-side on every route; passwords hashed; CSRF protection on all state-changing forms. | Security test cases, forced-browsing attempts |
| NFR-04 | Data integrity | All monetary values shall be stored and computed as integer pesewas; floating-point arithmetic shall not be used for money. | Unit tests, schema inspection |
| NFR-05 | Availability | The deployed system shall be available during collection hours (06:00–20:00 GMT). | Deployment verification |
| NFR-06 | Compliance | Only name, phone, business type and location shall be stored as client personal data, per the Data Protection Act 2012 (Act 843). | Schema review |
| NFR-07 | Maintainability | The domain layer shall have no dependency on Flask or SQLAlchemy and shall reach at least 70% unit-test coverage. | Coverage report, import inspection |
| NFR-08 | Accessibility | Interface shall meet WCAG 2.1 AA contrast, use touch targets of at least 44×44 px, and remain legible in outdoor light. | Contrast audit, device check |
| NFR-09 | Auditability | Every state change shall be attributable to an authenticated user and a timestamp. | Audit log inspection |
| NFR-10 | Portability | The application shall run against the same PostgreSQL engine in development and production, configured only by environment variable. | Docker parity, deploy verification |

## 3.5 Constraints

| ID | Constraint | Source |
|---|---|---|
| CO-01 | Development must be completed within a 48-hour examination window. | Examination rules |
| CO-02 | The system must be developed by a single developer. | Examination rules (individual assessment) |
| CO-03 | The application must be deployed and publicly accessible for grading. | Examination rules 8, 9 |
| CO-04 | Hosting is limited to free-tier services. | Project resources |
| CO-05 | No integration with mobile money or banking APIs is possible — no merchant credentials are available. | Technical/commercial access |
| CO-06 | Client personal data must comply with the Data Protection Act 2012 (Act 843). | Ghanaian law |
| CO-07 | Field users operate on low-end Android phones over mobile data. | Target user context |

## 3.6 Prioritisation (MoSCoW)

Prioritisation is driven by one test: **does this requirement serve BR-01/BR-02 — the protection and verifiability of client funds?** Anything that does is a *Must*; anything that merely improves convenience is deferred.

| Priority | Count | Requirements |
|---|---|---|
| **Must** | 28 | FR-01…08, FR-10…14, FR-16…21, FR-23…26, FR-28…30, FR-32, FR-33 |
| **Should** | 7 | FR-09, FR-15, FR-22, FR-34, FR-35, FR-39, FR-40 |
| **Could** | 4 | FR-27, FR-36, FR-37, FR-38 |
| **Won't** | 1 | FR-31 (SMS notification) |
| **Total** | **40** | FR-01 … FR-40 |

> FR-39 and FR-40 were added after the baseline was set, under change request **CR-001** ( `CHANGELOG-requirements.md` ). They are **Should**, so that abandoning them under schedule pressure costs no Must requirement.

**Why FR-31 is *Won't* despite its importance.** SMS notification is the mitigation for assumption A5 — the risk that clients cannot access a web page. It is arguably the single most valuable feature for the real user. It is excluded because it requires a paid SMS gateway (CO-04) and an account approval lead time that does not fit CO-01. It is therefore not dropped but *carried*: it is the first item in the future evolution plan.

## 3.7 Requirements traceability matrix

| Business req. | Stakeholder req. | Functional req. | Verified by |
|---|---|---|---|
| BR-01 | SR-07, SR-08 | FR-23, FR-24, FR-25, FR-26 | TC-REC-01…04 |
| BR-02 | SR-01, SR-02 | FR-28, FR-29, FR-30 | TC-CLI-01…03 |
| BR-03 | SR-07 | FR-24, FR-25, FR-26 | TC-REC-02 |
| BR-04 | SR-02 | FR-17, FR-18, FR-19 | TC-PAY-01…05 |
| BR-05 | SR-03, SR-11 | FR-32, FR-33, FR-34, FR-05 | TC-AUD-01…03 |
| BR-06 | SR-01, SR-03 | FR-28, FR-32, FR-33 | UAT-01…03 |
| BR-01, BR-06 | SR-04 | FR-39, FR-40 *(CR-001)* | TC-QR-01…03 |

*Test case identifiers are defined in* `09-testing.md`.

## 3.8 Change management

Following Session 2's formal change control process, and scaled to a single-developer project:

1. **Change request logging** — any scope change during the 48 hours is recorded as a dated entry in `docs/CHANGELOG-requirements.md` with the reason.
2. **Impact analysis** — the traceability matrix (§3.7) identifies which business requirements and test cases a change touches.
3. **Cost/schedule assessment** — the change is re-estimated against the remaining hours in the window.
4. **Approval** — with no Change Control Board available, the developer records the decision and its justification explicitly, so it is reviewable.
5. **Implementation** — design, code, tests and documentation are updated together.

Any requirement dropped after this point is recorded there rather than silently removed, so that the delivered scope can be compared against the specified scope.

**Changes raised to date:** CR-001 (QR-based client identification) — approved, adding FR-39 and FR-40 and business rules BR-R14 and BR-R15. See `CHANGELOG-requirements.md` for the impact analysis, cost assessment and decision record.

# Software Requirements Specification

**System:** SusuBook — Digital Susu Collection and Accountability System **Version:** 1.0 **Standard:** Structured after IEEE 830, adapted to the scope of a 48-hour capstone

---

## 1. Introduction

### 1.1 Purpose

This document specifies the requirements for SusuBook, a web-based system that records daily susu contributions, gives clients an independent view of their own savings, and enables same-day reconciliation of field collections against cash remitted to a branch.

It is written for the examiner assessing the system, and for any developer who subsequently maintains or extends it.

### 1.2 Scope

SusuBook digitises the **record** of susu collection. It does not move money. Cash continues to pass physically from client to collector; the system records that event, derives its consequences, and makes the record visible to every party with a legitimate interest in it.

**In scope:** client enrolment, contribution cycles, daily contribution recording with validation, the digital susu card, payout computation with commission, daily remittance declaration and variance detection, client self-service access, role-based authorisation, and an append-only audit trail.

**Out of scope:** mobile money and bank integration (CO-05), SMS notification (FR-31), offline operation, multi-institution tenancy, and native mobile applications. Section `06-scope.md` records the reasoning; `11-maintenance-evolution.md` carries them forward.

## 1.3 Definitions

| Term | Meaning |
|---|---|
| **Susu** | A Ghanaian informal savings practice; here, the collector variant. |
| **Client** | A saver who contributes a fixed amount daily to a collector. |
| **Collector** | The field officer who visits clients, receives cash and records contributions. |
| **Supervisor** | A branch officer who oversees collectors, reconciles remittances and authorises payouts. |
| **Daily rate** | The fixed amount a client agrees to contribute each day. |
| **Contribution cycle** | A fixed-length savings period, 31 days by default, ending in a payout. |
| **Contribution** | One recorded payment by a client, allocated to a specific date in a cycle. |
| **Susu card** | The 31-day grid view of a cycle, showing each day as paid, missed or pending. Digital equivalent of the paper card. |
| **Commission** | The collector's fee — one day's contribution, retained at payout. |
| **Payout** | The net amount released to the client at maturity: total collected less commission. |
| **Remittance declaration** | A collector's statement of cash banked at the branch for a given day. |
| **Variance** | The difference between contributions recorded in the field and cash declared, for one collector on one day. |
| **Reversal** | A linked correction entry that negates an earlier contribution without deleting it. |
| **Pesewa** | One hundredth of a Ghana Cedi (GHS). All monetary values are stored as integer pesewas. |
| **Public reference** | An opaque, non-sequential UUIDv4 identifying a client in externally visible URLs. Identifies but does not authorise. |
| **QR card** | A printed card carrying a client's public reference as a QR code encoding their contribution URL. The digital successor to the paper susu card. |

## 1.4 References

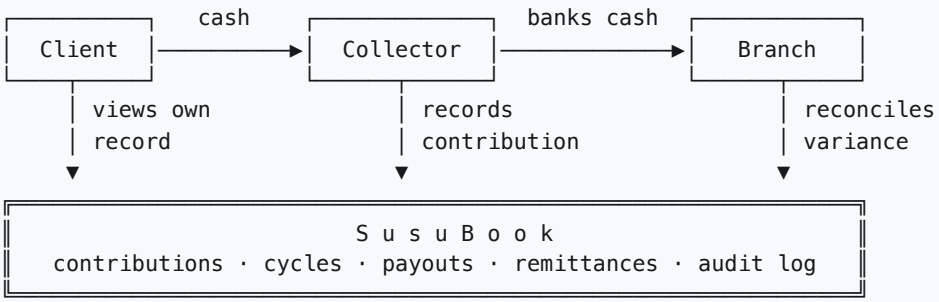
- CSCD602 Session 2 — Requirements Engineering
- CSCD602 Session 3 — Technical Debt
- CSCD602 Session 5 — Software Design and Architecture
- CSCD602 Session 6 — Software Effort Estimation
- Data Protection Act 2012 (Act 843), Republic of Ghana
- `01-problem-definition.md`, `02-stakeholder-analysis.md`, `03-requirements.md`

---

# 2. Overall description

## 2.1 Product perspective

SusuBook is a new, self-contained system. It replaces a paper artefact — the susu card — rather than an existing software system, so there is no legacy data migration and no external interface (hence zero External Interface Files in the function point count, §4.2.2 of the estimation).




The system's defining property is that the client's arrow **into** it is independent of the collector's. That is what the paper card cannot provide.

## 2.2 User classes

| Class | Frequency of use | Technical skill | Device |
|---|---|---|---|
| Client | Weekly to monthly | Low; some limited literacy | Low-end Android, mobile data |
| Collector | Many times daily | Moderate | Low-end Android, mobile data, outdoors |
| Supervisor | Daily | Moderate | Desktop or tablet at branch |
| Administrator | Occasional | High | Desktop |

## 2.3 Operating environment

Server-rendered web application over HTTPS, accessed through a mobile browser. Runs on Python 3.12 with Flask, against PostgreSQL 16. Identical database engine in development (Docker) and production (managed Postgres) per NFR-10.

## 2.4 Design and implementation constraints

Carried from §3.5: 48-hour window (CO-01), single developer (CO-02), public deployment required (CO-03), free-tier hosting (CO-04), no payment API access (CO-05), Act 843 compliance (CO-06), low-end devices on mobile data (CO-07).

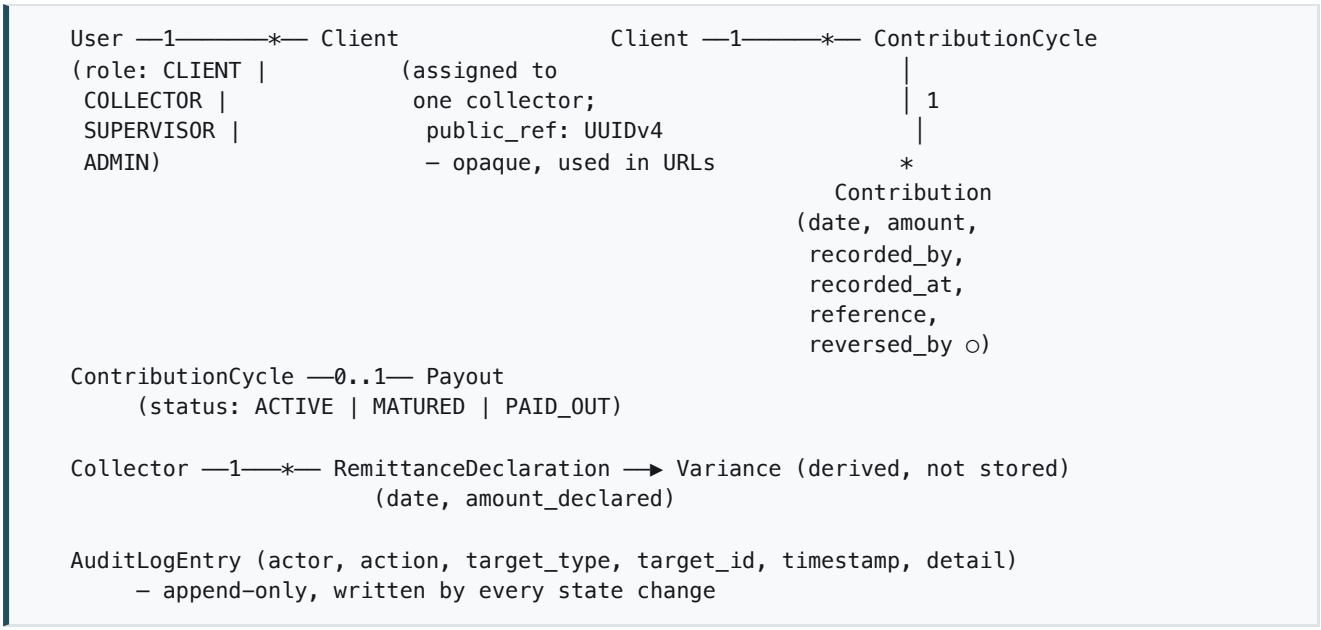
## 2.5 Assumptions and dependencies

Carried from §2.5, of which **A5 is critical**: clients are assumed able to reach a mobile web page. If false, the system's central value — independent client visibility — requires an SMS channel instead.

---

# 3. System features

## 3.1 Domain model




## 3.2 Business rules

These are the rules the domain layer enforces, independently of any user interface. They are the primary subject of unit testing.

| ID | Rule |
|---|---|
| BR-R1 | All monetary values are integer pesewas. Floating-point arithmetic is never applied to money. |
| BR-R2 | A client has at most one ACTIVE cycle at any time. |
| BR-R3 | A contribution must be allocated to a date within its cycle's start and end dates inclusive. |
| BR-R4 | A contribution date may not be in the future. |
| BR-R5 | At most one non-reversed contribution may exist per client per date. |
| BR-R6 | No contribution may be recorded against a cycle whose status is MATURED or PAID_OUT. |
| BR-R7 | A contribution amount must be a positive whole multiple of the client's daily rate. |
| BR-R8 | Payout = total collected − one day's rate (commission). |
| BR-R9 | Where total collected ≤ one day's rate, payout is zero and the whole balance is retained as commission. |
| BR-R10 | A cycle may be paid out at most once. |
| BR-R11 | A contribution is never edited or deleted; correction is a linked reversal entry. |
| BR-R12 | A cycle reaches MATURED when the current date passes its end date, regardless of days paid. |
| BR-R13 | Variance for a collector on a date = sum of contributions recorded − amount declared. |
| BR-R14 | Public references exposed in URLs or QR codes are opaque and non-sequential (UUIDv4). Sequential database identifiers never appear in a URL. *(CR-001)* |
| BR-R15 | A client reference identifies; it does not authorise. Possession confers no permission — authorisation remains the collector–client assignment enforced server-side (FR-05). *(CR-001)* |

> **BR-R14 and BR-R15 together are what make a QR card safe to carry through a public market.** The card must be assumed photographable. BR-R14 prevents an observer deriving other clients' references by incrementing a number; BR-R15 ensures that holding a reference grants nothing, because the authorisation decision never consults it. The QR encodes a URL containing an opaque reference and nothing else — no name, phone, balance or rate — so a photographed card leaks no personal data (NFR-06, Act 843).

> **BR-R9 exists because of a real edge case:** a client who contributes on only one day receives nothing, because the single day's contribution *is* the commission. Stating the rule explicitly prevents a negative payout, which is the defect this rule guards against.

### 3.3 Use cases

| ID | Use case | Primary actor | FRs |
|---|---|---|---|
| UC-01 | Log in | All | FR-01…04 |
| UC-02 | Enrol client | Collector | FR-06, FR-07, FR-10 |
| UC-03 | Record daily contribution | Collector | FR-11…14, FR-30, FR-32 |
| UC-04 | View digital susu card | Collector, Client | FR-16, FR-17 |
| UC-05 | Declare daily remittance | Collector | FR-24 |
| UC-06 | Review daily variance | Supervisor | FR-25, FR-26 |
| UC-07 | Release matured payout | Supervisor | FR-18…21 |
| UC-08 | View own contribution history | Client | FR-28, FR-29 |
| UC-09 | Reverse an erroneous contribution | Supervisor | FR-33, FR-32 |
| UC-10 | Issue a client QR card *(CR-001)* | Collector | FR-39 |

#### *UC-03 — Record daily contribution (core use case)*

| | |
|---|---|
| **Actor** | Collector |
| **Precondition** | Collector authenticated; client assigned to this collector; client has an ACTIVE cycle |
| **Postcondition** | Contribution persisted with a unique reference; audit entry written; client's view updated |
| **Frequency** | Highest in the system — dozens of times per collector per day |

**Main flow**

1. Collector opens today's route sheet.
2. System lists assigned clients with today's collection status.
3. Collector selects a client not yet collected from.
4. System presents a confirmation showing the client's name and their daily rate, pre-filled.
5. Collector confirms.
6. System validates against BR-R3 – BR-R7.
7. System persists the contribution with date, amount, recording collector, server timestamp and a unique reference.
8. System writes an audit entry.
9. System returns the updated route sheet with the client marked collected.

**Alternate flows**

- **3a. Catch-up payment** *(FR-15, Should)* — collector indicates several unpaid days; system allocates the amount across the specific unpaid dates and validates each against BR-R3 and BR-R5.
- **3b. Entry by QR scan** *(FR-39, FR-40, Should — CR-001)* — instead of steps 1–3, the collector scans the client's QR card with the phone's camera application, which opens the client's contribution URL directly. The system resolves the public reference (BR-R14), verifies the client is assigned to this collector (FR-05, BR-R15), and enters the flow at step 4. Reduces the interaction to **scan and confirm**. If the card is missing or unreadable, the collector falls back to the main flow.
- **4a. Amount differs from the daily rate** — collector overrides; system validates BR-R7 (whole multiple of the rate) and rejects any other value.

**Exception flows**

- **6a. Contribution already exists for this client and date** (BR-R5) — system rejects, states the existing reference, records nothing.
- **6b. Date is in the future** (BR-R4) — system rejects.
- **6c. Cycle is MATURED or PAID_OUT** (BR-R6) — system rejects and directs the collector to the payout process.
- **6d. Client not assigned to this collector** (FR-05) — system denies authorisation and records the attempt in the audit log.

> Step 4 pre-fills the amount and step 5 is a single confirmation. That is what satisfies NFR-02's three-interaction limit, and it is the design resolution of conflict C1 (collector speed vs. client verifiability) from §2.4 — integrity is added server-side at steps 7 and 8, costing the collector nothing.

### *UC-07 — Release matured payout*

| | |
|---|---|
| **Actor** | Supervisor |
| **Precondition** | Cycle status is MATURED; supervisor authenticated |
| **Postcondition** | Payout recorded; cycle set PAID_OUT; a new ACTIVE cycle opened; audit entry written |

**Main flow**

1. Supervisor opens the matured cycles list.
2. System shows each client, days paid, total collected, commission and net payout.
3. Supervisor selects a cycle and confirms release.
4. System re-validates BR-R8 – BR-R10 at the moment of release.
5. System records the payout, sets the cycle PAID_OUT, opens the next ACTIVE cycle.
6. System writes an audit entry and displays a payout advice.

**Exception flows**

- **4a. Cycle already PAID_OUT** (BR-R10) — rejected; no second payout is possible.
- **4b. Total collected ≤ daily rate** (BR-R9) — payout of zero is shown explicitly with the reason, and requires confirmation rather than failing silently.
- **4c. Cycle not yet MATURED** — rejected; directed to early withdrawal (FR-22) instead.

#### *Remaining use cases in brief*

- **UC-01 Log in** — credentials verified against a salted hash; role determines the landing page; failed attempts audited.
- **UC-02 Enrol client** — collector captures name, phone, business type, location and daily rate; system creates the client, assigns them to this collector and opens their first cycle atomically.
- **UC-04 View susu card** — 31-day grid; each day rendered paid, missed or pending; header shows days paid, total collected and projected net payout.
- **UC-05 Declare remittance** — collector enters cash banked for the day; system stores it and computes the variance.
- **UC-06 Review variance** — supervisor sees all non-zero variances for the day, by collector, with the underlying contributions.
- **UC-08 Client history** — client sees every contribution against them with amount, date, time recorded, recording collector and reference; the independent record that answers P1.
- **UC-09 Reverse contribution** — supervisor records a reversal; original remains visible, linked to the reversal; both appear in the audit trail (BR-R11).
- **UC-10 Issue QR card** — collector opens a client's printable card page; system renders the client's public reference as a QR code encoding their contribution URL, with the client's name for human identification; collector prints and issues it. Re-issuing a lost card by rotating the reference is deferred to the evolution plan.

### 3.4 User stories

| As a… | I want to… | So that… | FR |
|---|---|---|---|
| Client | see every payment recorded against me | I can prove what I have saved | FR-28 |
| Client | see what I will receive at maturity | I can plan without asking the collector | FR-29 |
| Collector | record a contribution in a couple of taps | I can move through my route quickly | FR-11, NFR-02 |
| Collector | see who I have not yet collected from today | I do not miss a client | FR-23 |
| Supervisor | see today's variance today | I can act while the money is still recoverable | FR-25, FR-26 |
| Supervisor | authorise payouts | funds are not released without oversight | FR-20 |
| Administrator | restrict a collector to their own clients | client data is not exposed across routes | FR-05 |

---

## 4. External interface requirements

**User interfaces.** Mobile-first, server-rendered HTML. Minimum 44×44 px touch targets, WCAG 2.1 AA contrast, legible in outdoor light (NFR-08). Four role-specific landing pages.

**Hardware interfaces.** None. Browser only; no card readers, printers or peripherals.

**Software interfaces.** PostgreSQL 16 over the SQLAlchemy ORM. No third-party APIs (CO-05).

**Communications interfaces.** HTTPS only. Session cookies marked `Secure`, `HttpOnly` and `SameSite`. CSRF tokens on every state-changing form.

---

## 5. Non-functional requirements

Specified in full at §3.4 of `03-requirements.md`: NFR-01 performance, NFR-02 usability, NFR-03 security, NFR-04 data integrity, NFR-05 availability, NFR-06 compliance, NFR-07 maintainability, NFR-08 accessibility, NFR-09 auditability, NFR-10 portability. Each carries a verification method, and each is exercised by a test case in `09-testing.md`.

---

## 6. Verification

Every requirement in this specification is traceable to at least one test case through the matrix at §3.7 of `03-requirements.md`. Requirements not covered by an executed test are listed as such in `09-testing.md` rather than presented as satisfied.

# 4. Software Effort Estimation

> This section answers the examination's required points: why the technique was selected, estimated effort, person-hours, development duration, assumptions, constraints, and **how the estimation influenced the project scope**.

## 4.1 Technique selection and justification

Session 6 sets out five families of estimation technique. Each was assessed against this project's actual situation — a novel system, no historical project data, a single developer, and a hard 48-hour ceiling.

| Technique | Suitability here | Decision |
|---|---|---|
| **Expert judgement** | No prior susu-system experience to draw on; single estimator means no Delphi panel and no protection against optimism bias. | Rejected as primary |
| **Estimation by analogy** | Requires similar completed projects with recorded actual effort. None available. | Rejected |
| **Machine-learning estimation** | Requires a sizeable historical dataset. None available; would also be a black box to the examiner. | Rejected |
| **Function Point Analysis** | Countable directly from the SRS *before any code exists*, independent of language, and defensible line by line. | **Selected — sizing** |
| **COCOMO (Basic, organic)** | Converts size into an industry-calibrated effort figure, giving an objective benchmark against the 48-hour budget. | **Selected — effort** |
| **Three-point / PERT (bottom-up over a WBS)** | Produces the operative schedule for the actual window and expresses uncertainty as a range rather than a false-precision number. | **Selected — planning** |
| **Agile story points** | Relative sizing needs a team velocity baseline to convert to time. A solo developer with no sprint history has none. | Rejected |

**Why three techniques rather than one.** Session 6's stated best practice is triangulation — "use multiple techniques and compare" — and the three chosen answer different questions:

- **FPA + COCOMO** answers "*what would this system cost to build properly?*" It is the objective benchmark. It is deliberately **not** calibrated to a 48-hour student project, and that is precisely its value: it establishes the size of the gap.
- **PERT over a WBS** answers "*what can actually be delivered in the hours available?*" Session 6 notes bottom-up estimation is "more accurate and defensible" where a detailed task breakdown exists — which the SRS provides.

Using only COCOMO would produce a number with no operational meaning. Using only PERT would let optimism bias set the scope unchallenged. Together, the first exposes the over-commitment and the second resolves it.

## 4.2 Function Point Analysis

Function point analysis follows Albrecht's method [20] as presented in Session 6 [6].

### 4.2.1 Weights applied

Weights are taken from the Session 6 table. External Interface Files are not weighted on that slide; the standard IFPUG values (5 / 7 / 10) are noted for completeness, though the count is zero here.

| Component | Simple | Average | Complex |
|---|---|---|---|
| External Input (EI) | 3 | 4 | 6 |
| External Output (EO) | 4 | 5 | 7 |
| External Inquiry (EQ) | 3 | 4 | 6 |
| Internal Logical File (ILF) | 7 | 10 | 15 |
| External Interface File (EIF) | 5 | 7 | 10 |

### 4.2.2 Count A — full specified scope (all 38 functional requirements)

**Internal Logical Files**

| Logical file | Complexity | FP |
|---|---|---|
| User (accounts, roles, credentials) | Average | 10 |
| Client (enrolment, daily rate, assignment) | Average | 10 |
| ContributionCycle | Average | 10 |
| Collection | Average | 10 |
| Payout | Simple | 7 |
| RemittanceDeclaration | Simple | 7 |
| AuditLogEntry | Simple | 7 |
| | **Subtotal** | **61** |

**External Interface Files** — none. CO-05 excludes all external system integration. **0**

**External Inputs**

| Input | FR | Complexity | FP |
|---|---|---|---|
| Login | FR-01 | Simple | 3 |
| Enrol client | FR-06 | Complex | 6 |
| Update client details | FR-06 | Average | 4 |
| Reassign client to collector | FR-09 | Simple | 3 |
| Record contribution | FR-11 | Complex | 6 |
| Record catch-up contribution | FR-15 | Complex | 6 |
| Record correction / reversal | FR-33 | Average | 4 |
| Declare cash remittance | FR-24 | Simple | 3 |
| Release payout | FR-20 | Complex | 6 |
| Request early withdrawal | FR-22 | Simple | 3 |
| Approve early withdrawal | FR-22 | Average | 4 |
| Resolve variance with note | FR-27 | Simple | 3 |
| Create / suspend user, assign role | FR-35 | Average | 4 |
| Configure institutional defaults | FR-36 | Simple | 3 |
| | | **Subtotal** | **58** |

### External Outputs

| Output | FR | Complexity | FP |
|---|---|---|---|
| Digital susu card with computed day states | FR-16 | Complex | 7 |
| Cycle summary (days paid/missed, total, projected payout) | FR-17 | Complex | 7 |
| Daily collection sheet per collector | FR-23 | Average | 5 |
| Daily variance report | FR-25, FR-26 | Complex | 7 |
| Client statement (balance, maturity, net payout) | FR-29 | Average | 5 |
| Supervisor dashboard | FR-37 | Complex | 7 |
| CSV cycle export | FR-38 | Average | 5 |
| Payout advice | FR-18 | Average | 5 |
| | | **Subtotal** | **48** |

### External Inquiries

| Inquiry | FR | Complexity | FP |
|---|---|---|---|
| Client list / search | FR-08 | Average | 4 |
| Client detail view | FR-08 | Simple | 3 |
| Contribution history list | FR-28 | Average | 4 |
| Audit trail view | FR-34 | Average | 4 |
| User list | FR-35 | Simple | 3 |
| Contribution reference lookup | FR-30 | Simple | 3 |
| | | **Subtotal** | **21** |

**Unadjusted Function Points (full scope)**

```
UFP = ILF 61 + EIF 0 + EI 58 + EO 48 + EQ 21 = 188
```

### 4.2.3 Value Adjustment Factor

Fourteen General System Characteristics, each rated 0 (no influence) to 5 (strong):

| # | Characteristic | Rating | Reasoning |
|---|---|---|---|
| 1 | Data communications | 4 | Web application, users distributed across field and branch |
| 2 | Distributed data processing | 1 | Single application server, single database |
| 3 | Performance | 4 | NFR-01 mandates 2 s render on a 3G-class connection |
| 4 | Heavily used configuration | 2 | Free-tier hosting, modest concurrent load |
| 5 | Transaction rate | 3 | Collection activity concentrated in the morning route window |
| 6 | Online data entry | 5 | All data entry is online and field-based |
| 7 | End-user efficiency | 5 | NFR-02 caps a routine collection at three interactions |
| 8 | Online update | 5 | Internal files updated in real time by field users |
| 9 | Complex processing | 3 | Commission rule, day allocation, variance computation |
| 10 | Reusability | 2 | Single institution; domain layer reusable |
| 11 | Installation ease | 2 | Containerised, environment-variable configuration |
| 12 | Operational ease | 3 | Audit logging, minimal routine administration |
| 13 | Multiple sites | 2 | Branch structure modelled, single deployment |
| 14 | Facilitate change | 4 | Layered architecture (NFR-07), configurable cycle and commission |
| | **ΣGSC** | **45** | |

```
VAF = 0.65 + (0.01 × 45) = 0.65 + 0.45 = 1.10
```

**Adjusted Function Points (full scope)**

```
AFP = UFP × VAF = 188 × 1.10 = 206.8 ≈ 207 FP
```

### 4.2.4 Count B — Must-have scope only (28 functional requirements)

Removing the Should/Could/Won't items from §3.6:

| Component | Removed | Subtotal |
|---|---|---|
| ILF | none — all seven files are required by Must requirements | 61 |
| EI | reassign (3), catch-up (6), request withdrawal (3), approve withdrawal (4), resolve variance (3), user admin (4), configure defaults (3) = **−26** | 32 |
| EO | supervisor dashboard (7), CSV export (5) = **−12** | 36 |
| EQ | audit trail view (4), user list (3) = **−7** | 14 |
| EIF | — | 0 |

```
UFP (Must) = 61 + 32 + 36 + 14 + 0 = 143
AFP (Must) = 143 × 1.10 = 157.3 ≈ 157 FP
```

Cutting ten requirements removes 50 adjusted function points — **24% of the system**.

### 4.2.5 Revision under CR-001

The counts above are the Phase 1 baseline. Change request CR-001 (QR-based client identification) subsequently added two components. The baseline is left intact and the revision recorded separately, so the effect of the change remains visible.

| Added component | Type | Complexity | FP |
|---|---|---|---|
| Printable client QR card | External Output | Average | 5 |
| Client reference resolution | External Inquiry | Simple | 3 |
| | | **Subtotal** | **+8** |

| | Baseline | After CR-001 |
|---|---|---|
| UFP (Must scope) | 143 | **151** |
| AFP (Must scope) | 157 | **166** |
| UFP (full scope) | 188 | 196 |
| AFP (full scope) | 207 | 216 |

All figures in §4.3 below are computed on the **post-CR-001** counts.

---

## 4.3 COCOMO Basic (organic mode)

Constants and equations from Boehm [11]; the effort equation and mode table as presented in Session 6 [6].

### 4.3.1 Size conversion

Function points are converted to KLOC using a language productivity figure.

> **Assumption AS-01:** 30 source lines of code per function point for Python with an ORM and server-side templating. This is a published-average figure for the language class; a sensitivity analysis at 25 and 40 LOC/FP is given in §4.3.4 because the choice materially affects the result. *The specific productivity table used must be cited in* `References` *in the final document.*

```
KLOC (Must scope, post-CR-001) = 166 FP × 30 LOC/FP ÷ 1000 = 4.98 KLOC
```

### 4.3.2 Mode selection

| Mode | a | b | Applies when |
|---|---|---|---|
| **Organic** | **2.4** | **1.05** | Small, familiar team; flexible requirements |
| Semi-detached | 3.0 | 1.12 | Medium size and complexity, mixed experience |
| Embedded | 3.6 | 1.20 | Complex, tightly constrained (safety, hardware) |

**Organic** is selected: the system is small (< 5 KLOC), built by a single developer with full domain familiarity, using well-understood web technology, with no hardware, safety or real-time constraints. Semi-detached would be defensible if the mobile-money and SMS integrations of the full vision were in scope; they are excluded by CO-05 and FR-31.

### 4.3.3 Effort and duration

```
Effort = a × (KLOC)^b
       = 2.4 × (4.98)^1.05
       = 2.4 × 5.396
       = 13.0 person-months
```

Converting at the COCOMO standard of 152 hours per person-month:

```
Person-hours = 13.0 × 152 = 1,969 person-hours
```

Schedule, using Boehm's organic schedule equation `TDEV = 2.5 × (Effort)^0.38` (this extends beyond the Session 6 slide, which stops at the effort equation):

```
TDEV = 2.5 × (13.0)^0.38 = 2.5 × 2.65 = 6.6 months
Average staffing = 13.0 ÷ 6.6 ≈ 2.0 developers
```

Applying instead the simplified division used in the Session 6 worked example, a single developer would need **13.0 months** — the schedule equation's 6.6 months assumes roughly two people working in parallel.

### 4.3.4 Sensitivity to the LOC/FP assumption

| LOC/FP | KLOC | Effort (PM) | Person-hours |
|---|---|---|---|
| 25 (optimistic) | 4.15 | 10.7 | 1,626 |
| **30 (adopted)** | **4.98** | **13.0** | **1,969** |
| 40 (pessimistic) | 6.64 | 17.5 | 2,663 |

The estimate is **not** sensitive enough for the conclusion to change: across the entire range the requirement is over 1,600 person-hours against a 48-hour budget.

### 4.3.5 Full-scope comparison

| Scope | AFP | KLOC | Effort (PM) | Person-hours |
|---|---|---|---|---|
| Full specified (40 FRs) | 216 | 6.48 | 17.1 | 2,595 |
| Must + CR-001 (30 FRs) | 166 | 4.98 | 13.0 | 1,969 |
| | | | **Saving** | **626 person-hours** |

For reference, the Phase 1 baseline before CR-001: full scope 207 AFP / 16.3 PM / 2,482 hours; Must scope 157 AFP / 12.2 PM / 1,857 hours. CR-001 added 0.8 person-months (112 person-hours) to the delivered scope under COCOMO.

---

## 4.4 The gap, stated honestly

```
COCOMO estimate (Must scope):        1,969 person-hours
Available window:                       48 hours (single developer)
Implementation allocation (Phase 3):    20 hours
                                    ──────────────────
Ratio of required to available:          ~98 : 1
```

A ratio of 98:1 is not a usable planning number, and it would be dishonest to present it as one. It requires explanation rather than presentation:

**Why COCOMO overstates this project.**

1. **It prices the whole lifecycle.** The 1,969 hours include requirements engineering, design, integration, formal QA and documentation. Those activities are being performed here, but they are budgeted separately across Phases 1–2 and 4–6, not inside the 20-hour implementation window.
2. **It predates framework leverage.** COCOMO's constants were calibrated on projects where authentication, ORM persistence, routing, templating, session management and CSRF protection were all hand-written. Flask, SQLAlchemy and Jinja supply these as configuration. The LOC/FP figure accounts for language, not for framework scaffolding.
3. **Basic COCOMO ignores team and tool factors entirely.** It has no effort multipliers — that is what COCOMO II's cost drivers exist for. Modern tooling, high developer familiarity and mature libraries are invisible to the Basic model.

4. **It assumes a team, and therefore communication cost.** A solo developer with complete domain knowledge incurs no coordination overhead, no handover, and no specification ambiguity between people.

**What the number is genuinely worth.** It is evidence that the *specified* system is an industrial-scale build, and that anything deliverable in 48 hours is a deliberately reduced subset. It converts "I ran out of time" into a scoping decision made in advance with a stated basis. That is the reason it is retained rather than discarded.

---

## 4.5 Three-point (PERT) bottom-up estimate — the operative plan

`E = (O + 4M + P) / 6`, in hours, over the implementation WBS.

| # | Task | O | M | P | E |
|---|---|---|---|---|---|
| 1 | Project skeleton, config, Docker Postgres, app factory | 0.5 | 1.0 | 2.0 | 1.08 |
| 2 | Domain layer: entities, money type, business rules | 1.5 | 2.5 | 4.0 | 2.58 |
| 3 | Database schema, SQLAlchemy models | 1.0 | 1.5 | 3.0 | 1.67 |
| 4 | Repository layer | 0.5 | 1.0 | 2.0 | 1.08 |
| 5 | Authentication: hashing, sessions, role decorators | 1.0 | 2.0 | 3.5 | 2.08 |
| 6 | Client enrolment, list and search | 1.0 | 1.5 | 2.5 | 1.58 |
| 7 | Cycle open/close logic | 0.5 | 1.0 | 2.0 | 1.08 |
| 8 | Record contribution with all validation rules | 1.5 | 2.5 | 4.0 | 2.58 |
| 9 | Digital susu card view (31-box grid) | 1.0 | 1.5 | 3.0 | 1.67 |
| 10 | Payout computation and release | 1.0 | 1.5 | 2.5 | 1.58 |
| 11 | Remittance declaration and variance | 1.0 | 1.5 | 2.5 | 1.58 |
| 12 | Client self-service views | 0.5 | 1.0 | 2.0 | 1.08 |
| 13 | Append-only audit log and reversal | 0.5 | 1.0 | 2.0 | 1.08 |
| 14 | Collector route sheet | 0.5 | 1.0 | 2.0 | 1.08 |
| 15 | UI, HTMX interactions, mobile layout | 1.0 | 2.0 | 3.5 | 2.08 |
| 16 | Seed data and demo accounts | 0.25 | 0.5 | 1.0 | 0.54 |
| 17 | QR issuance and scan-to-collect *(CR-001)* | 0.5 | 0.85 | 1.6 | 0.92 |
| | **Total expected effort** | | | | **25.4 h** |

**Uncertainty.** Per-task `σ = (P − O)/6`; total `σ = √Σσ² = 1.24 h`.

| Confidence | Range |
|---|---|
| 68% (±1σ) | 24.1 – 26.6 h |
| 95% (±2σ) | 22.9 – 27.8 h |

> **Caveat on that range.** PERT assumes tasks vary independently. They do not: a single developer who is slow on the domain layer will likely be slow on the service layer too, because the cause is shared. The true variance is therefore wider than ±1.22 h, and the range above should be read as a lower bound on uncertainty, not a guarantee.

**Cone of Uncertainty.** Session 6 places a project with requirements defined but design incomplete in the **0.5× – 2× band**. Applied to 25.4 h, the honest interval is **12.7 – 50.7 hours**. The upper bound consumes the entire examination window on implementation alone, leaving nothing for testing, deployment or documentation — 30 of the 50 marks. This is the finding that forces the scope decision below.

---

## 4.6 How the estimation influenced the project scope

**The finding.** Bottom-up expected effort for the Must scope was **24.5 hours** against a **20-hour** implementation allocation — a 23% over-commitment at the *expected* value, before any allowance for the upper half of the cone.

**The decision.** Rather than begin and hope, 4.1 hours were removed in advance by deliberately reducing implementation quality in areas that do not touch BR-01 or BR-02 (protection and verifiability of client funds).

| Cut | Hours saved | What is sacrificed | Debt classification |
|---|---|---|---|
| Schema created via `create_all()` and a seed script instead of Alembic migrations | 0.7 | No versioned schema history; production schema changes need manual intervention | Infrastructure debt |
| Tailwind via CDN with minimal custom CSS instead of a built stylesheet | 1.0 | Render-blocking external stylesheet, no purging, larger payload — works against NFR-01 | Design / infrastructure debt |
| Client list as a plain table, no search or pagination | 0.5 | FR-08 only partially satisfied; degrades past ~50 clients | Usability debt |
| Reversal exposed as a minimal supervisor-only form | 0.6 | Append-only model fully implemented, correction workflow is bare | Code debt |
| Route sheet as a static list without "not yet collected" filtering | 0.5 | FR-23 met literally; collector efficiency (NFR-02) reduced | Usability debt |
| HTMX applied to the three highest-value interactions only, full page loads elsewhere | 0.8 | Inconsistent interaction model across the application | Design debt |
| | **4.1 h** | | |

```
24.5 h − 4.1 h = 20.4 h  ≈  20 h allocation
```

### Revision under CR-001

Change request CR-001 subsequently added QR-based client identification (FR-39, FR-40) at **+0.92 h**, revising the plan:

```
25.4 h − 4.1 h = 21.3 h  vs  20 h allocation  →  +6.4% overrun
```

**The overrun was accepted rather than absorbed.** A further 1.3 hours of quality cuts could have been manufactured to land the arithmetic on exactly 20.0. That was rejected: Session 6 identifies external schedule pressure as a recognised source of estimate distortion, and revising an estimate to fit a predetermined budget *is* that distortion. The overrun is carried openly and tracked against actuals in §4.8.

It is contained by priority rather than by arithmetic. FR-39 and FR-40 are **Should**, so if Phase 3 runs behind, task 17 is the first work abandoned and no Must requirement is affected. The full decision record is in `CHANGELOG-requirements.md`.

**What was explicitly *not* cut, and why.** The domain layer (2.58 h), contribution validation (2.58 h), audit log (1.08 h) and client self-service views (1.08 h) were protected in full. These four carry BR-01, BR-02 and BR-05 — the reason the system exists. Cutting implementation quality elsewhere to protect them is the whole substance of the scoping decision.

**The consequence is recorded, not hidden.** Every cut above is a *prudent and deliberate* entry on Fowler's technical debt quadrant — taken knowingly, for a stated reason, with an intended repayment. Each one carries forward into the debt register in `08-technical-debt.md` with its cause, impact, priority and proposed resolution, and into the repayment plan in `11-maintenance-evolution.md`. The estimation did not merely predict the work; it generated the debt register.

---

## 4.7 Assumptions and constraints on the estimate

### Assumptions

| ID | Assumption |
|---|---|
| AS-01 | 30 LOC per function point for Python with ORM and templating (sensitivity tested, §4.3.4). |
| AS-02 | Organic COCOMO mode applies — small system, familiar technology, no hardware constraints. |
| AS-03 | 152 hours per person-month (COCOMO standard). |
| AS-04 | Requirements remain stable through the window; the change control process of §3.8 governs any deviation. |
| AS-05 | Development environment, Docker and deployment accounts are working and not counted as project effort. |
| AS-06 | The developer sustains productive work across the window; sleep and breaks are outside the 20-hour implementation allocation, not inside it. |

### Constraints on the estimate

| ID | Constraint |
|---|---|
| ES-01 | No historical project data exists, so no analogy-based or ML validation is possible. |
| ES-02 | A single estimator means no Delphi convergence and no protection against optimism bias. Session 6 identifies planning fallacy and optimism bias as the dominant risks here; triangulation across three methods is the only available mitigation. |
| ES-03 | Function point counting was performed once by one person. Independent recounts typically vary by ±10–15%. |
| ES-04 | The 48-hour window is fixed before estimation, which Session 6 identifies as *external pressure* — a known source of estimate distortion. It is mitigated by fixing the deadline and flexing scope, never the reverse. |

---

## 4.8 Estimation accuracy review

Session 6's closing best practice is to record actuals and review accuracy afterwards. Two things can be closed out: the **size** estimate, precisely; and the **effort** estimate, which cannot — for reasons set out in §4.9, because knowing when data does not support a conclusion is part of the practice.

### 4.8.1 Size — closed

| | Estimated | Actual | Variance |
|---|---|---|---|
| Adjusted function points | 166 | — | — |
| LOC per function point (AS-01) | 30 | **20.7** | −31% |
| Source size | 4.98 KLOC | **3.43 KLOC** | **−31%** |
| COCOMO organic effort at that size | 12.95 PM | 8.76 PM | −32% |

```
MRE = |3.43 − 4.98| / 3.43 = 0.45
```

Measured over 3,433 non-blank, non-comment lines of application code (`10-implementation.md` §10.2).

**Assumption AS-01 was wrong, and in a specific direction.** 30 LOC per function point came from published averages for the language class; actual productivity was **20.7** — below even the optimistic 25 used in the sensitivity analysis (§4.3.4). Two causes are consistent with the delivered code: modern frameworks supply as configuration what those averages assume is hand-written (routing, sessions, ORM persistence, CSRF, templating), and some function points produce very little code — the QR card is 5 adjusted function points and roughly 15 lines, because the library does the work.

The sensitivity analysis anticipated this *class* of error but under-bounded it, setting the range at 25–40. Derived from measurement rather than published tables, it would have started lower.

**The conclusion of §4.4 is unaffected.** Even at actual size, COCOMO puts the system at 8.76 person-months — over 1,300 person-hours against a 48-hour window. The gap narrows and remains of the same order.

### 4.8.2 Effort — actuals against the 48-hour plan

The project is planned in hours within a 48-hour window, so the actuals are recorded in the same unit: elapsed time per phase, taken from commit timestamps, against the allocation in the examination's Part B plan.

| Phase | Allocated | Elapsed | Window occupied |
|---|---|---|---|
| 1. Planning & Requirements | 6 h | 0.33 h | h0.00 – h0.33 |
| 2. Analysis & Design | 6 h | 0.12 h | h0.33 – h0.45 |
| 3. Implementation | 20 h | 0.73 h | h0.45 – h1.17 |
| 4. Testing & Refinement | 6 h | **2.34 h** | h1.17 – h3.51 |
| 5. Deployment | 4 h | **2.48 h** | h3.51 – h5.99 |
| 6. Documentation | 6 h | 0.48 h* | h5.99 – h6.47 |
| **Total** | **48 h** | **6.47 h** | |

* Phase 6 was still in progress at the time of measurement.

### 4.8.3 The finding is the distribution, not the total

Comparing totals is the least informative reading. The **shape** of the plan is where the error lies, and that comparison is valid regardless of absolute magnitude:

| Phase | Share of plan | Share of actual | |
|---|---|---|---|
| 1. Planning & Requirements | 12.5% | 5.1% | over-allocated |
| 2. Analysis & Design | 12.5% | 1.9% | over-allocated |
| **3. Implementation** | **41.7%** | **11.3%** | **heavily over-allocated** |
| **4. Testing & Refinement** | **12.5%** | **36.2%** | **heavily under-allocated** |
| **5. Deployment** | **8.3%** | **38.3%** | **heavily under-allocated** |
| 6. Documentation | 12.5% | 7.4%* | in progress |

**Testing and deployment together were allocated 21% of the window and consumed 75% of the time actually spent.** Implementation was allocated 42% and consumed 11%.

This inverts the assumption the whole estimate was built on. §4.5 spent seventeen WBS tasks decomposing implementation and treated testing and deployment as comparatively minor. The reverse held:

- **Deployment (2.48 h)** absorbed VPC egress configuration, Secret Manager wiring, a Cloud Run Job for schema creation because the database is private, and **DEF-08** — Google Front End silently intercepting `/healthz`. None of that is application code, and none of it was decomposed in the WBS at all.
- **Testing (2.34 h)** absorbed the integration and system suites, and four of the eight defects. Two of those were errors in the developer's own assumptions rather than in the code.

**This is the most useful thing the estimate produced.** The magnitude comparison is contaminated (§4.9); the distribution is not, because both figures come from the same project. The corrective action for a future estimate is specific: shift weight out of implementation and into deployment and testing, and decompose

deployment into a WBS rather than treating it as a single 4-hour block.

### 4.8.4 Per-task actuals

The 17-task breakdown below cannot be completed from the record. Commits do not partition along WBS boundaries — a single commit delivers the domain layer, its tests and its documentation together — so per-task figures would be invented rather than measured.

| Task | Estimated (h) | Actual (h) | Variance | Note |
|---|---|---|---|---|

| Task | Estimated (h) | Actual (h) | Variance | Note |
|---|---|---|---|---|
| 1 Project skeleton | 1.08 | | | |
| 2 Domain layer | 2.58 | | | |
| 3 Database schema | 1.67 | | | |
| 4 Repository layer | 1.08 | | | |
| 5 Authentication | 2.08 | | | |
| 6 Client enrolment | 1.58 | | | |
| 7 Cycle logic | 1.08 | | | |
| 8 Record contribution | 2.58 | | | |
| 9 Digital card view | 1.67 | | | |
| 10 Payout | 1.58 | | | |
| 11 Remittance & variance | 1.58 | | | |
| 12 Client views | 1.08 | | | |
| 13 Audit log | 1.08 | | | |
| 14 Route sheet | 1.08 | | | |
| 15 UI & HTMX | 2.08 | | | |
| 16 Seed data | 0.54 | | | |
| 17 QR issuance & scan *(CR-001)* | 0.92 | — | — | Not separable |
| **Total** | **25.4** | — | — | See §4.8.2 for phase-level actuals |

---

## 4.9 How to read these actuals

Three qualifications, so the figures are not read as more than they are.

**Elapsed time is not pure effort.** Intervals between commits include infrastructure setup, container builds, deploy waits and review. The 2.48 hours against Phase 5 covers a period in which much of the work was cloud configuration — which AS-05 excludes from project effort by definition. The distribution finding in §4.8.3 survives this, because the same measure is applied to every phase; the absolute total does not.

**No instrument was in place.** Effort was not tracked as the work happened; these figures are reconstructed from commit timestamps afterwards. Session 6's practice is to *record* actuals as work occurs, and reconstruction is inference. A future project should instrument at the WBS level from the first hour — that is the specific corrective action, and it is what would have made §4.8.4 completable.

**The magnitude comparison is weaker than the distribution comparison.** The 25.4-hour PERT estimate priced a single developer building this system task by task. Dividing the elapsed total into it would yield a tidy MRE, and it would be a number about production method rather than about estimation accuracy. §4.8.3 is reported instead because a ratio between phases, both measured the same way in the same project, does not depend on that.

**What the estimate is nonetheless credited with.** It did the job it was built for. It exposed a 23% over-commitment *before* implementation began, forced the scope decision in §4.6, and generated the technical debt register in the process. That value was realised at the time of estimating and does not depend on retrospective validation.

**What is not claimed.** That the effort estimate was validated against measured effort. It was not, and §4.8.4 is left incomplete rather than filled with plausible numbers.

**Closing the loop to ES-01.** Constraint ES-01 recorded that no historical project data existed to calibrate against. That was true because no earlier project recorded actuals. §4.8.2 and §4.8.3 are the beginning of that data — the first entry in a baseline that Lehman's third law ( `11–maintenance–evolution.md` §11.4) says is what eventually makes effort predictable. Its most useful content is not the total but the finding that implementation was over-weighted by a factor of roughly four while deployment was under-weighted by nearly five.

Magnitude of Relative Error will be computed as `MRE = |Actual − Estimated| / Actual`, and the result — favourable or not — reported as the closure of the estimation process described in §4.1.

# 7. System Analysis and Design

Covers examination document sections 8 (System analysis) and 9 (System design).

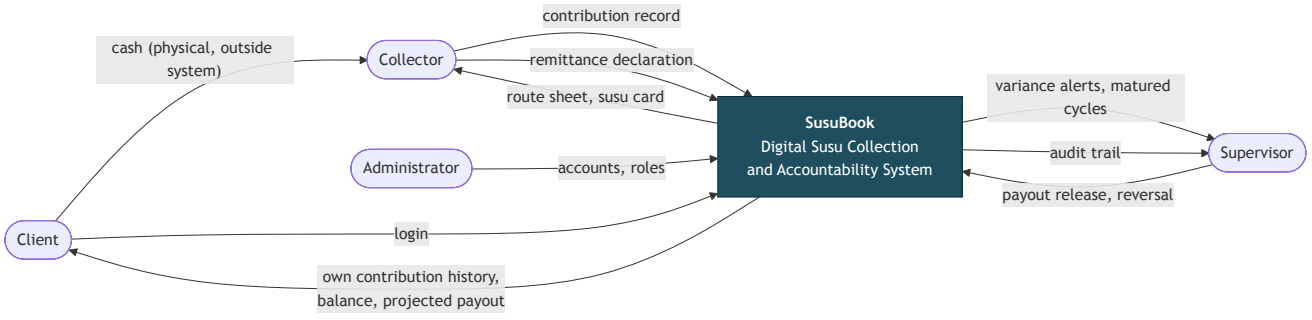
# PART A — SYSTEM ANALYSIS

## 7.1 Analysis of the current (manual) system

| Aspect | Current manual process |
|---|---|
| **Record medium** | A paper card with 31 boxes, held by the collector |
| **Recording act** | Collector marks a box, sometimes initials it |
| **Client's copy** | None. The client holds nothing. |
| **Computation** | Manual, at maturity, by the collector |
| **Reconciliation** | Cash counted at the branch against the collector's own tally |
| **Audit trail** | None beyond the card itself |
| **Failure detection** | On dispute, or when a collector disappears |

**Structural weakness.** The party whose conduct requires verification is the sole custodian of the evidence. Every failure in `01-problem-definition.md` (P1–P4) descends from that one fact, so the analysis below treats *separating the record from the collector* as the system's defining requirement rather than one feature among many.

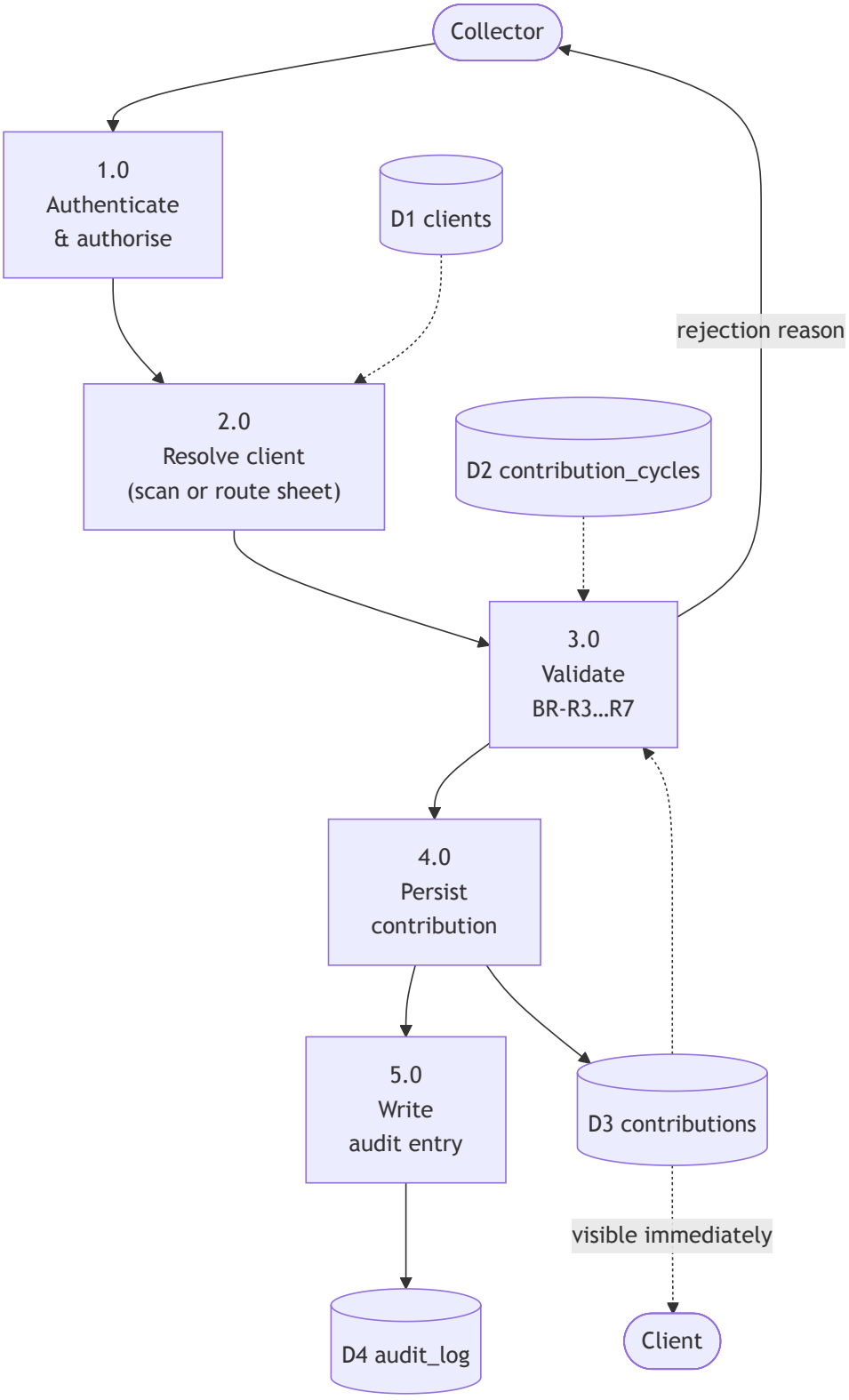
## 7.2 Context diagram (Level 0 DFD)




Context diagram (Level 0 data flow)

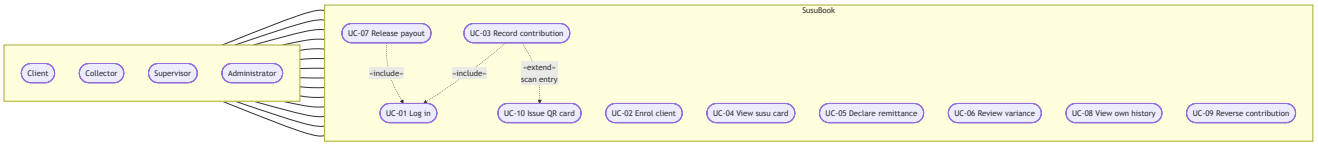
The arrow from the client **into** the system, independent of the collector's, is the structural change. Everything else is bookkeeping around it.

## 7.3 Level 1 data flow — recording a contribution




Level 1 data flow — recording a contribution

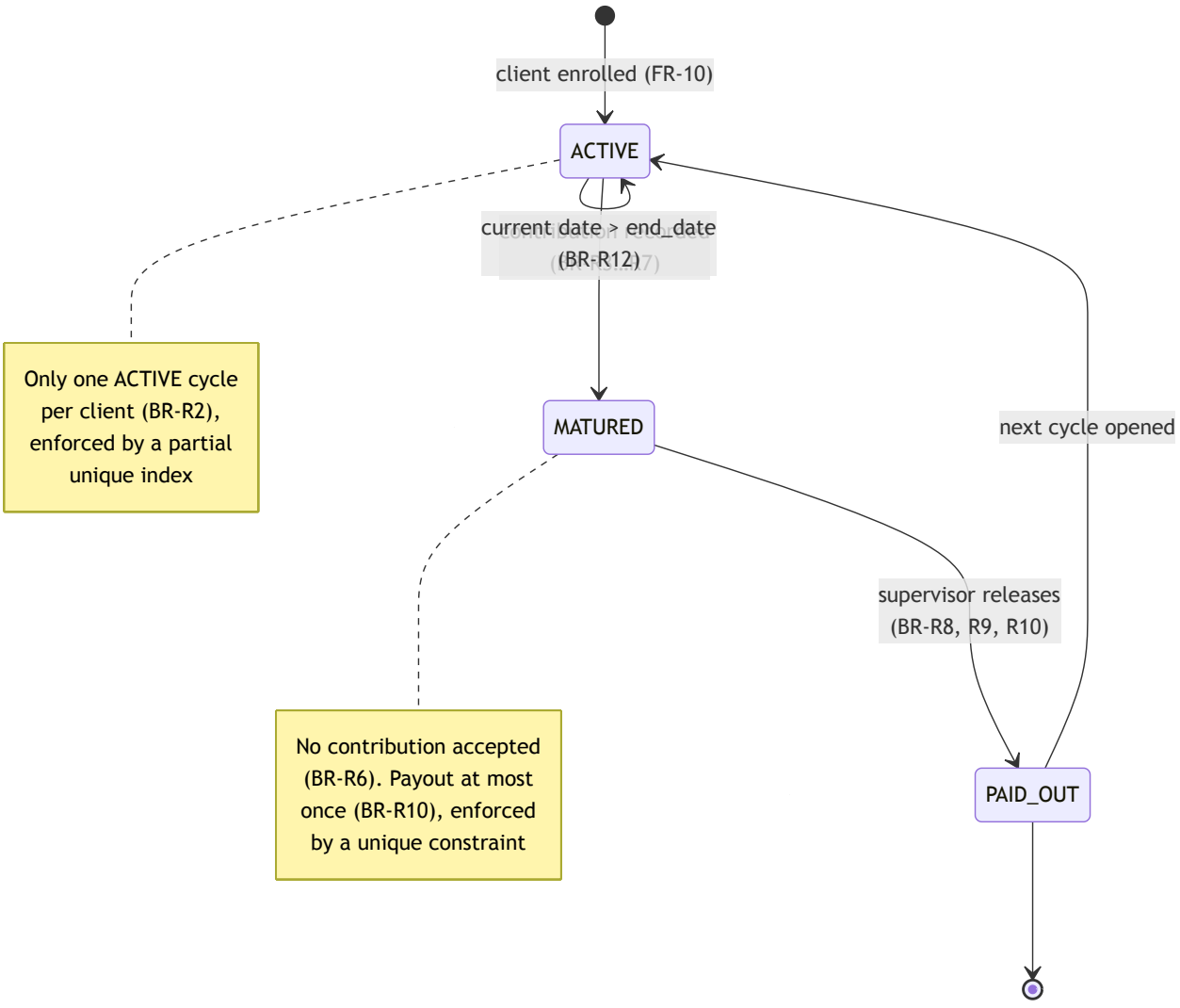
## 7.4 Use case diagram




Use case diagram

## 7.5 Contribution cycle state model

The cycle's lifecycle carries several business rules, so it is modelled explicitly.




Contribution cycle state model

# PART B — SYSTEM DESIGN

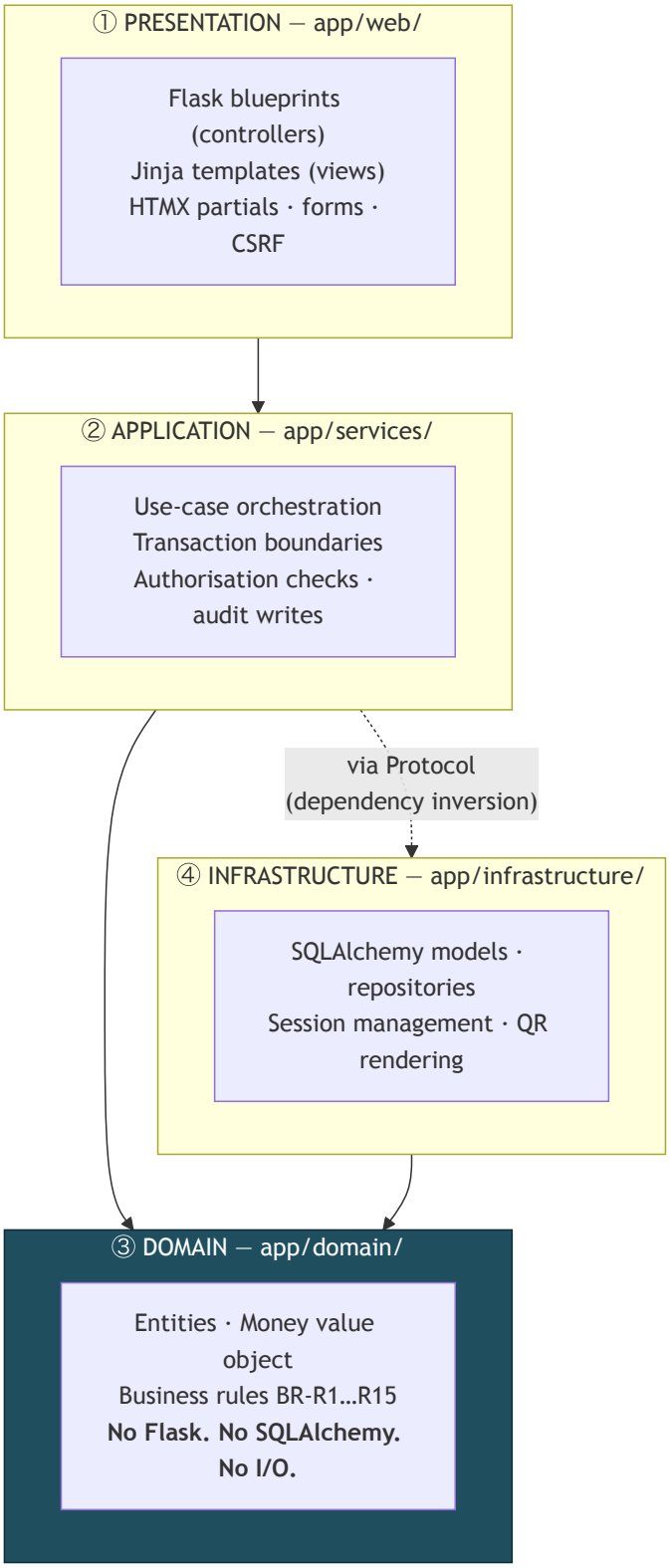
## 7.6 Architectural style: layered (N-tier)

Session 5 presents three architectural options. Each was assessed against this project:

| Style | Assessment | Decision |
|---|---|---|
| **Microservices** | Session 5 lists the preconditions: large teams in parallel, independent scaling, differing stacks, high availability. **None** hold here — one developer, one deployment, free-tier hosting. It would add distributed-tracing and partial-failure complexity for no benefit, and Session 5 explicitly warns it "requires DevOps maturity". | Rejected |
| **MVC alone** | Accurate for the web tier but says nothing about where business rules live. Applied *within* the presentation layer rather than as the whole architecture. | Partially adopted |
| **Layered (N-tier)** | Enforces the downward dependency rule, isolates business rules from Flask and the database, and directly enables NFR-07 (domain testable without a database). | **Selected** |

### Layer stack

Mapped onto Session 5's "Common Layer Stack" slide:




Layered architecture

**The dependency rule.** Dependencies point downward only. The domain layer depends on nothing — not Flask, not SQLAlchemy, not the network. It is plain Python.

**Why this matters more than it looks.** It is what makes the 5 marks for testing achievable. Business rules can be unit-tested with no database, no HTTP client and no fixtures, so the tests run in milliseconds and can be written in the time available. An Active-Record design where rules live on ORM models would

require a live database for every rule test — and under a 48-hour budget those tests simply would not get written.

**The sinkhole risk.** Session 5 warns of the sinkhole anti-pattern — requests passing through layers that add nothing. Accepted here: simple reads (e.g. listing clients) pass through the service layer with little transformation. The cost is a few extra function calls; the benefit is one consistent path for authorisation and audit. Recorded as **TD-08**.

## 7.7 Project structure

The folder tree *is* the architecture diagram:

```
app/
├── __init__.py             application factory, dependency wiring
├── config.py               environment-driven configuration
├── domain/                 ③ pure Python — no framework imports
│   ├── money.py            Money value object (integer pesewas)
│   ├── entities.py         Client, ContributionCycle, Contribution, Payout …
│   ├── rules.py            BR-R1…R15 as pure functions
│   └── errors.py           DomainError hierarchy
├── services/               ② use-case orchestration
│   ├── protocols.py        repository Protocols (dependency inversion)
│   ├── enrolment.py        UC-02
│   ├── collection.py       UC-03, UC-09
│   ├── payout.py           UC-07
│   ├── reconciliation.py   UC-05, UC-06
│   └── audit.py            append-only audit writes
├── infrastructure/         ④ framework-bound
│   ├── models.py           SQLAlchemy ORM models
│   ├── repositories.py     Protocol implementations
│   ├── db.py               engine, session, create_all
│   └── qrcodes.py          segno SVG rendering
└── web/                    ① controllers and views
    ├── auth.py             blueprint
    ├── collector.py        blueprint
    ├── supervisor.py       blueprint
    ├── client.py           blueprint
    ├── templates/
    └── static/
tests/
├── unit/                   domain rules — no database
├── integration/            services + repositories against Postgres
└── system/                 end-to-end through the Flask test client
```

## 7.8 Technology stack justification

The syllabus lab toolkit lists React, TypeScript, Node and Mongo/MySQL. Flask and HTMX are not named, though Python is. Examination Rule 6 requires that frameworks be acknowledged, which this section does; the choice is defended on architectural grounds.

| Layer | Choice | Justification |
|---|---|---|
| **Language** | Python 3.12 | Named in the syllabus toolkit. Dataclasses and `typing.Protocol` express the domain layer and dependency inversion directly. |
| **Web framework** | Flask 3 | A microframework imposes no architecture, so the layered design is genuinely ours rather than the framework's. Blueprints give modular controllers. |
| **Templating** | Jinja2 | Server-rendered HTML — Session 5's MVC pattern, the same family as Django and Rails which that slide names. |
| **Interactivity** | HTMX | The interaction set (forms, tables, inline validation, confirmations) needs partial page updates, not client-side state. HTMX delivers these by returning HTML fragments, with no build step, no bundler and no hydration. |
| **ORM** | SQLAlchemy 2 | Confined to the infrastructure layer. The repository boundary keeps it out of the domain, satisfying NFR-07. |
| **Database** | PostgreSQL 16 | Partial unique indexes let three business invariants be enforced in the database as well as in code (§7.11). Identical engine in dev (Docker) and production, satisfying NFR-10. |
| **QR rendering** | segno | Pure Python, no Pillow dependency, emits inline SVG that scales without an image pipeline. |
| **Testing** | pytest + coverage | The domain layer's independence makes fast, database-free unit tests possible. |
| **Containerisation** | Docker Compose (dev) | Dev/prod parity on the same Postgres engine. |

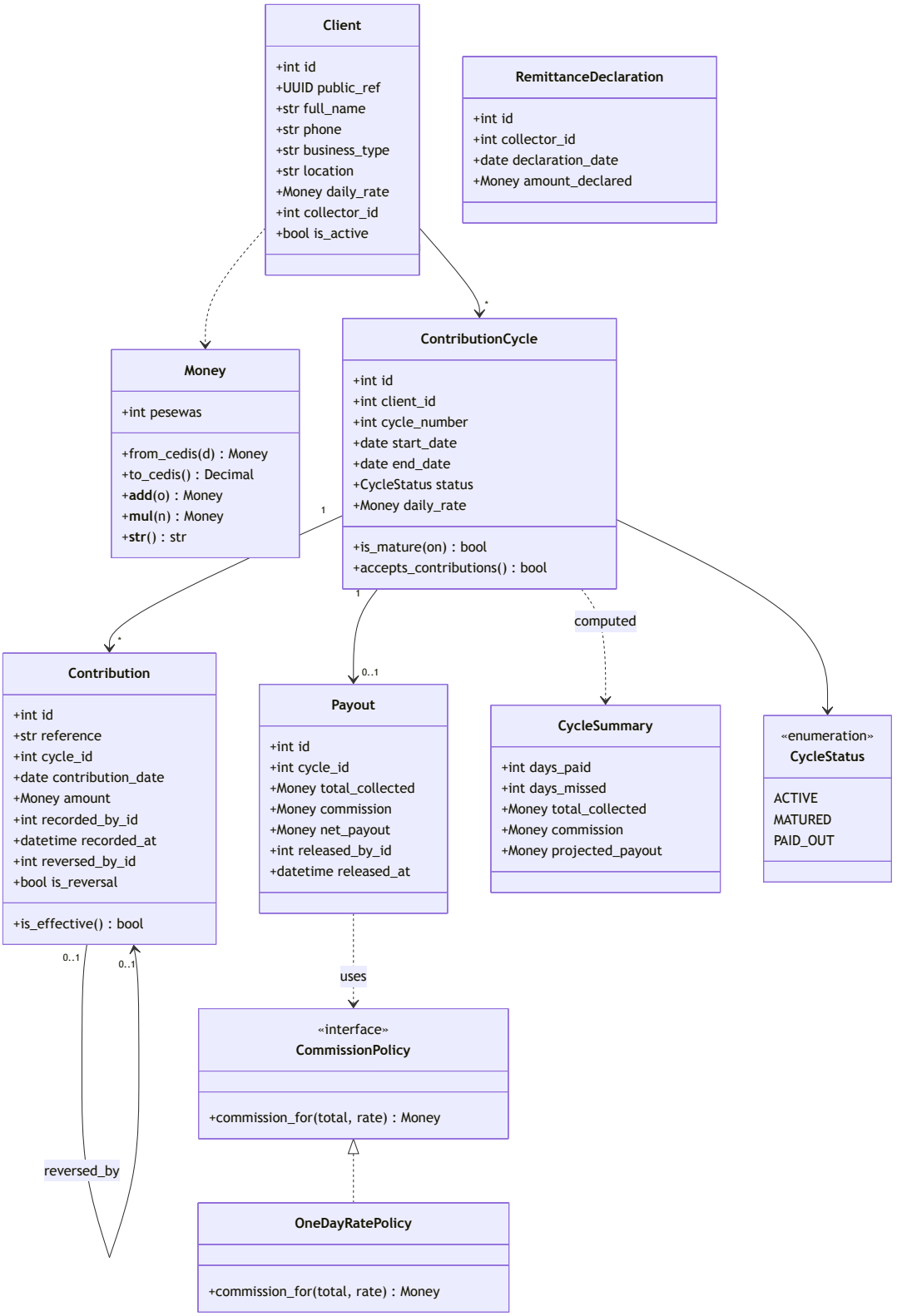
### Why not React

Considered and rejected, on four grounds:

1. **The architecture would be documented, not demonstrated.** A React SPA plus a JSON API means two deployables, CORS configuration and a build toolchain — none of which express a layered architecture any better than server-rendered MVC does.
2. **Deployment risk.** Two services to deploy instead of one, against examination Rule 8 which requires the deployment to remain accessible for grading.
3. **Testing cost.** Server-side business logic is far cheaper to test than component trees, and testing carries 5 marks with no lecture deck to lean on.
4. **The interactivity is not there.** React earns its complexity with substantial client-side state — drag-and-drop, offline-first, realtime collaboration. SusuBook has none of that.

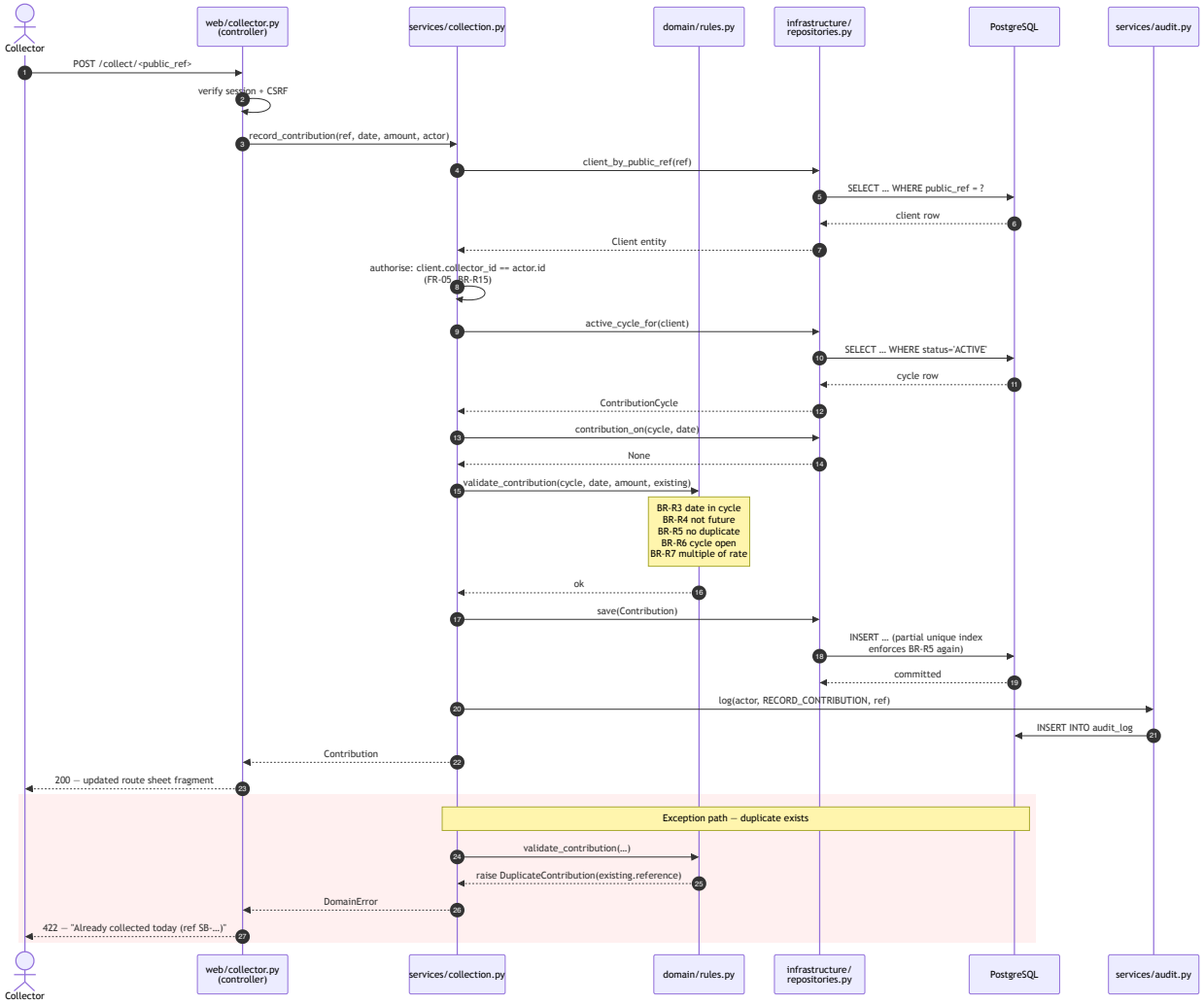
**The honest trade-off.** HTMX means a network round trip for interactions React would handle locally. Under NFR-01 (2 s on a 3G-class connection) that is a real cost. It is accepted because the payloads are small HTML fragments, and it is mitigated by keeping the three highest-frequency interactions — the ones on the collector's critical path — as the partial updates HTMX is applied to.

## 7.9 Domain class diagram




Domain class diagram

## 7.10 Sequence diagram — UC-03 record contribution

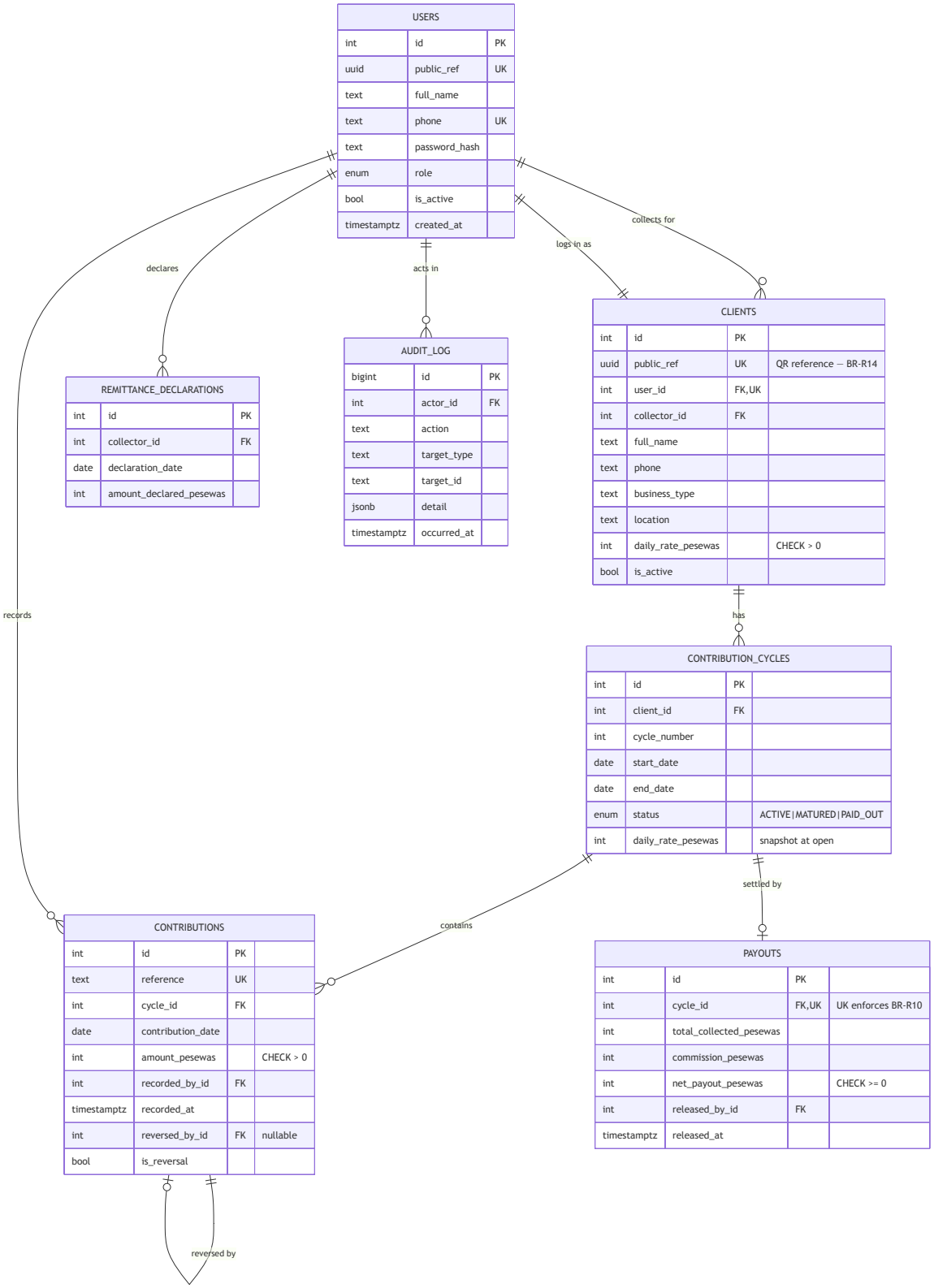



Sequence diagram — UC-03 record contribution

Steps 3–4 are what resolves conflict **C1** from §2.4: the collector supplies one confirmation, while attribution, timestamping and audit are added server-side at no interaction cost.

# 7.11 Database design

## Entity-relationship diagram




Entity-relationship diagram

### Business invariants enforced in the database

Three rules are enforced **twice** — in the domain layer and again by the schema. This is deliberate defence in depth: a bug in the service layer, or a future direct database write, cannot corrupt these invariants.

| Rule | Database mechanism |
|---|---|
| **BR-R2** — one ACTIVE cycle per client | `CREATE UNIQUE INDEX ux_active_cycle ON contribution_cycles (client_id) WHERE status = 'ACTIVE'` |
| **BR-R5** — one effective contribution per client per date | `CREATE UNIQUE INDEX ux_contribution_day ON contributions (cycle_id, contribution_date) WHERE reversed_by_id IS NULL AND is_reversal = FALSE` |
| **BR-R10** — at most one payout per cycle | `UNIQUE (cycle_id)` on `payouts` |
| **BR-R1** — money is integral | every monetary column is `INTEGER` pesewas; no `FLOAT`, `REAL` or `MONEY` type appears in the schema |

PostgreSQL's partial unique indexes are the specific reason it was chosen over MySQL: the first two rules cannot be expressed as plain unique constraints, because reversed contributions must be allowed to coexist with their replacements on the same date.

### Indexes

| Index | Purpose |
|---|---|
| `clients (public_ref)` unique | QR reference resolution (FR-40) |
| `clients (collector_id)` | Route sheet (FR-23) |
| `contribution_cycles (client_id, status)` | Active cycle lookup — the hottest query |
| `contributions (cycle_id)` | Susu card rendering (FR-16) |
| `contributions (recorded_by_id, recorded_at)` | Daily variance computation (FR-25) |
| `audit_log (target_type, target_id)` | Audit trail retrieval (FR-34) |

### Money representation

All monetary values are `INTEGER` pesewas, wrapped in a `Money` value object in the domain layer. GHS 2.50 is stored as `250`.

This is a deliberate decision against the obvious alternative. Binary floating point cannot represent 0.1 exactly, so accumulating 31 daily contributions of GHS 0.10 as floats yields 3.0000000000000004 rather than 3.10. On a savings system, a rounding error of a pesewa compounding across thousands of clients is both a correctness defect and a trust failure — and this system exists to establish trust. `Decimal` would also be correct, but integers are faster, map exactly onto an `INTEGER` column, and make the invariant impossible to violate accidentally.

## 7.12 Application of SOLID principles

Session 5's five principles [5], originally set out by Martin [15], each with the concrete place it applies. Session 5 also notes that SOLID compliance *reduces technical debt* — the connection to §8 is direct.

**S — Single Responsibility.** `domain/rules.py` holds one pure function per computation (`validate_contribution`, `compute_cycle_summary`, `compute_payout`, `compute_variance`). Services orchestrate but compute nothing. `services/audit.py` does nothing but append audit entries. The bad example on the slide — one class handling auth, email and reporting — is avoided by having no class that spans concerns.

**O — Open/Closed.** `CommissionPolicy` is an interface with `OneDayRatePolicy` as the shipped implementation. FR-36 (configurable commission) is deferred, but when it arrives a `PercentagePolicy` is *added*, not an edit to tested payout code. This is the slide's `PaymentMethod` example applied to our actual deferred requirement.

**L — Liskov Substitution.** Repository implementations are substitutable for their Protocols. The test suite injects in-memory fakes wherever production injects SQLAlchemy repositories, and every service test passes against both. That equivalence is the practical proof of substitutability.

**I — Interface Segregation.** Separate narrow Protocols — `ClientRepository`, `CycleRepository`, `ContributionRepository`, `PayoutRepository` — rather than one fat `DataAccess`. `PayoutService` depends only on what it uses, so a change to contribution queries cannot break it.

**D — Dependency Inversion.** Services depend on `typing.Protocol` abstractions; concrete SQLAlchemy repositories are injected by the application factory. `PayoutService` never imports SQLAlchemy. This is what makes the domain and service tests run without a database, and it is the principle the whole architecture rests on.

## 7.13 Design patterns applied

| Pattern | Where | Why |
|---|---|---|
| **Layered architecture** | Whole application | Separation of concerns, testability (Session 5) |
| **MVC** | `web/` — blueprints, Jinja templates, models | Session 5's canonical web pattern |
| **Repository** | `infrastructure/repositories.py` | Decouples the business layer from the persistence technology [16][17] |
| **Service Layer** | `services/` | One place per use case for orchestration, transactions and audit |
| **Value Object** | `domain/money.py` | Money is compared and combined by value; immutability prevents a whole defect class [17] |
| **Strategy** | `CommissionPolicy` | Open/closed extension point for FR-36 [18] |
| **Application Factory** | `app/__init__.py` | Different wiring for production and tests — enables dependency injection |
| **Dependency Injection** | Factory-wired repositories | Session 5 names DI as the practical form of the D in SOLID |

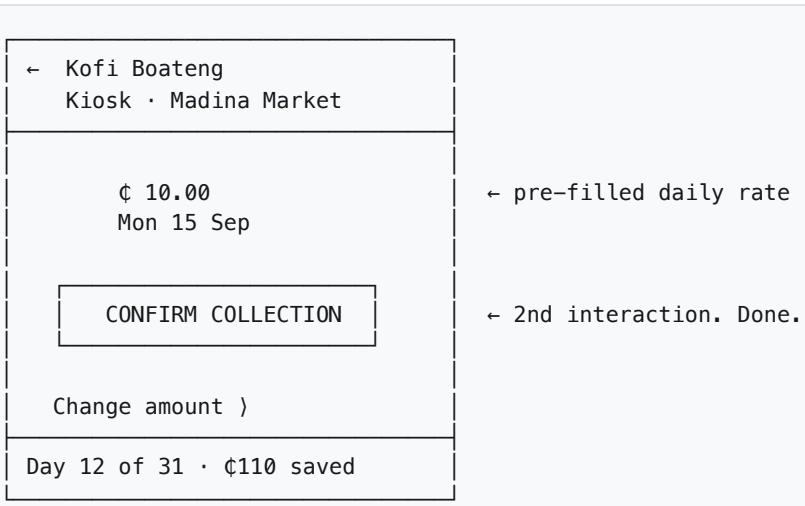
## 7.14 Interface design

Mobile-first, because the collector is standing in a market and the client is on a low-end phone (CO-07, NFR-08).

**Collector — route sheet (primary screen)**

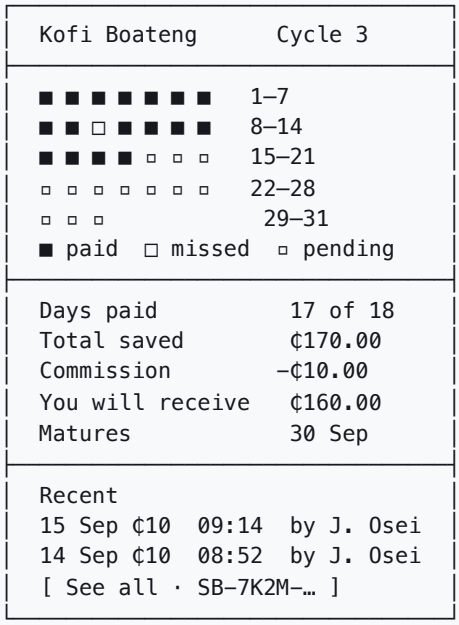
```
┌──────────────────────────────────┐
│ ≡  Today · Mon 15 Sep      ₵0.00 │
├──────────────────────────────────┤
│  [ 📷  Scan client card ]        │  ← QR path (FR-40), 1st interaction
├──────────────────────────────────┤
│  Ama Mensah      ₵5.00   ✓ Paid  │
│  Kofi Boateng    ₵10.00  ⟩       │
│  Akosua Darko    ₵5.00   ⟩       │
│  Yaw Owusu       ₵20.00  ✓ Paid  │
├──────────────────────────────────┤
│ Recorded today      ₵25.00       │
│ [ Declare remittance ]           │
└──────────────────────────────────┘
```

**Collector — confirm contribution** (reached by scan or by tapping a row)



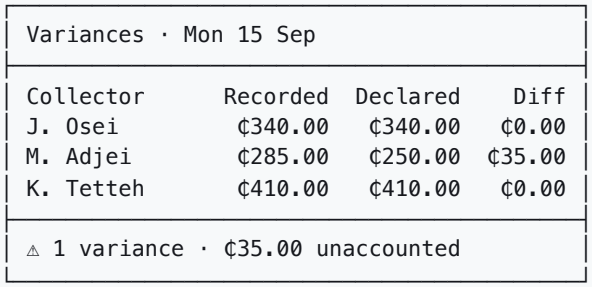

Two interactions from scan to recorded, against NFR-02's limit of three.

**Client — my susu card** (the answer to P1)




Every row names the recording collector and the time. That is the independent record the paper card cannot provide.

**Supervisor — daily variance (the answer to P3)**



### Accessibility (NFR-08)

Interface work follows the user-centred design activities of ISO 9241-210 [28] as presented in Session 5 [5].

WCAG 2.1 AA contrast; touch targets ≥ 44×44 px; the susu card grid uses **shape and text, not colour alone**, to distinguish paid from missed — colour-blind users and a sun-washed screen fail the same way, so the fix serves both. Amounts are rendered large; no interaction depends on hover.

## 7.15 Security design (NFR-03)

| Control | Implementation |
|---|---|
| Password storage | Argon2id via `argon2-cffi`; no plaintext or reversible storage |
| Session management | Flask signed session cookies — `Secure`, `HttpOnly`, `SameSite=Lax`; idle timeout (FR-04) |
| CSRF | Token on every state-changing form (Flask-WTF) |
| Authorisation | Enforced **server-side in the service layer**, not in templates — hiding a link is not a control |
| Object-level authorisation | Every client-scoped operation re-checks `client.collector_id == actor.id` (FR-05) |
| IDOR / enumeration | Opaque UUIDv4 public references in all external URLs (BR-R14) |
| SQL injection | Parameterised queries only, via SQLAlchemy; no string-built SQL |
| Data minimisation | Name, phone, business type, location only — no Ghana Card, address or next of kin (NFR-06, Act 843) |
| Non-repudiation | Append-only audit log; corrections are reversals, never edits (BR-R11) |
| Transport | HTTPS enforced in production |

**Threat: a stolen or photographed QR card.** Mitigated by design rather than by secrecy. The reference identifies but does not authorise (BR-R15): reaching a client's contribution URL still requires an authenticated collector to whom that client is assigned. The QR encodes a URL and an opaque reference only, so a photographed card discloses no personal data.

---

## 7.16 Technical debt identified during design

Session 5's key takeaway — that SOLID compliance reduces technical debt — is treated literally here: debt is identified at design time, before any is incurred, rather than discovered afterwards. TD-01…TD-06 were created by the scope decision in `05-effort-estimation.md` §4.6; TD-07…TD-11 arise from the design choices above.

| ID | Debt | Origin | Fowler quadrant | Type |
|---|---|---|---|---|
| TD-01 | `create_all()` instead of Alembic migrations | Scope cut | Prudent & deliberate | Infrastructure |
| TD-02 | Tailwind via CDN, unpurged | Scope cut | Prudent & deliberate | Design/Infrastructure |
| TD-03 | Client list without search or pagination | Scope cut | Prudent & deliberate | Usability |
| TD-04 | Minimal reversal form | Scope cut | Prudent & deliberate | Code |
| TD-05 | Static route sheet *(partly mitigated by CR-001)* | Scope cut | Prudent & deliberate | Usability |
| TD-06 | HTMX applied inconsistently | Scope cut | Prudent & deliberate | Design |
| **TD-07** | Hand-written mapping between domain dataclasses and ORM models — every schema change must be made in two places | Layered design (§7.6) | Prudent & deliberate | Architecture |
| **TD-08** | Sinkhole: simple reads pass through the service layer adding little | Layered design (§7.6) | Prudent & deliberate | Architecture |
| **TD-09** | Audit log is append-only by convention, not enforced by database permissions or a trigger | Time constraint | Prudent & deliberate | Architecture/Security |
| **TD-10** | Cycle maturity computed on read rather than by a scheduled job; a cycle no one opens is never marked MATURED | No scheduler on free-tier hosting | Prudent & deliberate | Architecture |
| **TD-11** | Cycle summaries recomputed from all contributions on every render; no caching or denormalised totals | Simplicity over optimisation | Prudent & inadvertent | Performance |

Every item is **prudent** — taken knowingly, for a stated reason — and all but TD-11 are **deliberate**. TD-11 is classified *inadvertent* because the performance cost was recognised only while designing the card view, which is precisely Fowler's "we now know how we should have done it" quadrant.

Full cause → impact → priority → resolution analysis follows in `08-technical-debt.md`.

## 7.17 Traceability: design decisions to requirements

| Design decision | Serves | Rationale |
|---|---|---|
| Layered architecture, domain isolated | NFR-07 | Domain testable without a database |
| Integer pesewas + `Money` value object | NFR-04, BR-R1 | Eliminates floating-point error on money |
| Partial unique indexes | BR-R2, BR-R5, BR-R10 | Invariants survive a service-layer bug |
| Opaque UUID public references | BR-R14, NFR-03 | Prevents enumeration and IDOR |
| Append-only audit log with reversals | BR-05, NFR-09, BR-R11 | Non-repudiation; resolves conflict C2 |
| Pre-filled amount, single confirmation | NFR-02 | Two interactions; resolves conflict C1 |
| Server-rendered HTML, small payloads | NFR-01, CO-07 | Works on low-end devices over mobile data |
| Shape and text, not colour alone | NFR-08 | Serves colour-blind users and sunlit screens alike |
| Same Postgres engine dev and prod | NFR-10 | Removes the class of defects that appear only in production |
| `CommissionPolicy` strategy | FR-36 (deferred) | Extension without modifying tested payout code |

# 10. Implementation

> Examination document §10. What was built, the decisions taken while building it, and where the implementation departed from the design.

## 10.1 Delivered system

A functional, deployed web application implementing 30 functional requirements across four roles, with 226 automated tests at 97% coverage.

| | |
|---|---|
| **Live** | https://susubook-fdtbppd7sq-uc.a.run.app |
| **Source** | https://github.com/tee-jay7/susubook |
| **Stack** | Python 3.12 · Flask 3 · SQLAlchemy 2 · PostgreSQL 16 · Jinja2 · HTMX · segno |
| **Tests** | 226 (129 unit, 42 integration, 55 system) |
| **Coverage** | 97% overall; 99–100% on the domain layer |

## 10.2 Structure as built

The folder tree is the architecture diagram. Dependencies point downward only.

| Layer | Path | Lines | Contents |
|---|---|---|---|
| ① Presentation | `app/web/` | 430 | 4 blueprints, session handling, role decorators |
| — templates | `app/web/templates/` | 757 | 18 Jinja templates, shared macros |
| ② Application | `app/services/` | 725 | Use-case orchestration, repository Protocols |
| ③ Domain | `app/domain/` | 484 | Entities, `Money`, business rules BR-R1…R15 |
| ④ Infrastructure | `app/infrastructure/` | 700 | ORM models, repositories, QR rendering |
| — root | `app/` | 337 | Factory, config, seed |
| | **Application total** | **3,433** | |
| | Test code | 2,634 | |

Test code is **77% the size of the application**. That ratio is the practical consequence of the architectural decision described below.

## 10.3 The decision that shaped everything else

**The domain layer imports neither Flask nor SQLAlchemy.**

This looks like architectural purity. It was a scheduling decision.

Business rules expressed as pure functions can be tested with no database, no fixtures and no HTTP client. The 129 unit tests run in **0.34 seconds**. Under an implementation budget of roughly 20 hours, an Active-Record design — rules living on ORM models — would have required a live database for every rule test, and those tests would not have been written at all. Testing carries 5 marks and had no lecture deck behind it; the architecture is what made the suite affordable.

The cost is real and recorded: hand-written mapping between domain entities and ORM records (**TD-07**), which must be updated in two places on any schema change. An integration test asserts that money round-trips the mapping exactly, which is the specific drift that would matter most.

## 10.4 Implementation decisions worth defending

### Money is an integer count of pesewas

```
@dataclass(frozen=True, order=True)
class Money:
    pesewas: int
```

`float` is rejected at construction *and* at multiplication, rather than merely avoided by convention:

```
if isinstance(amount, float):
    raise TypeError(
        "Refusing to build Money from float — binary floating point "
        "cannot represent decimal currency exactly (BR-R1)."
    )
```

Testing this produced **DEF-01**, and the correction is more interesting than the rule. The original test asserted that 31 × GHS 0.10 accumulates to 3.0000000000000004 in floating point. It does not — that sum happens to round back to exactly 3.1. The error is real but **intermittent**: 29 × 0.10 gives 2.9000000000000004, and 3 × 0.10 gives 0.30000000000000004.

Intermittent is worse than consistently wrong, because it survives casual testing and reaches production. The test now asserts exactness across the whole 1–31 day range rather than at a single sample that could pass by luck.

### The zero-payout edge case is made unrepresentable

BR-R9 states that where total collected does not exceed one day's rate, the client receives nothing and the whole balance is retained. The naive implementation — `payout = total - rate` — produces a **negative payout** for a client who contributed on only one day.

```
def commission_for(self, total_collected: Money, daily_rate: Money) -> Money:
    return min(daily_rate, total_collected)
```

Expressed as `min`, a negative payout becomes unrepresentable rather than merely unlikely. It is then guarded three further times: an assertion in the domain layer, a `CHECK (net_payout_pesewas >= 0)`, and a `CHECK` asserting `net + commission = total_collected` so money is conserved. A test exercises all 32 possible completion levels rather than sampling.

### Three invariants are enforced twice

BR-R2, BR-R5 and BR-R10 are checked in the domain layer *and* again by PostgreSQL partial unique indexes:

```
CREATE UNIQUE INDEX ux_effective_contribution_per_day
  ON contributions (cycle_id, contribution_date)
  WHERE reversed_by_id IS NULL AND is_reversal = false;
```

The index is **partial**, and that is the point. A reversed contribution, its reversal, and the replacement must coexist on one date — a plain `UNIQUE(cycle_id, contribution_date)` would make correction by reversal impossible. This is the specific reason PostgreSQL was chosen over MySQL.

The integration suite writes **through the ORM, bypassing the service layer**, to demonstrate the guarantee does not depend on application code being correct.

### Correction is reversal, never deletion

A contribution is never edited or deleted. A correction is a new linked entry, so both remain visible on the client's record. This resolves conflict **C2** from the stakeholder analysis — the collector can fix an honest mistake without weakening the non-repudiation the client relies on.

The client-facing screenshot in `docs/screenshots/05-client-card.png` captures this working: a REVERSED entry and its REVERSAL both visible, days paid correctly reduced, and the affected day returned to "missed" on the card.

### Authorisation never consults the QR reference

```
if actor.is_supervisor or client.is_collected_by(actor.id):
    return client
self._audit.append(action="AUTHORISATION_DENIED", ...)
raise NotAuthorised("This client is not on your route.")
```

BR-R15 in code: possession of a client reference gets you to the lookup and no further. A card photographed in a market confers nothing, because the authorisation decision reads the collector–client assignment rather than the reference.

### Out-of-band updates on the collector's critical path

**DEF-06**, found by exploratory testing rather than by any of the 226 automated tests. Recording a contribution swapped the client's row to "Paid" but left the day's running total stale until a manual refresh — a row marked Paid sitting above a total reading GHS 0.00.

Two figures disagreeing on one screen is worse than a figure that does not update: it makes the collector distrust both. Fixed with an HTMX out-of-band swap, so one request updates both without re-rendering the list.

## 10.5 Where implementation departed from design

Recorded because a design that survived contact with implementation untouched would be a design nobody followed.

| Change | Reason |
|---|---|
| `CycleService.open_for()` takes `client_id` and `daily_rate` rather than a `Client` | The payout path opens the next cycle knowing only what the closing cycle carried. The original signature forced construction of a half-populated entity purely to satisfy it. |
| `BASE_URL` became optional, falling back to the request origin | A Cloud Run URL is not known until the service exists. Also keeps QR cards correct across a domain change, with no reissue. |
| Boolean and status columns gained `server_default` | **DEF-02.** A direct SQL insert failed on NOT NULL before reaching the partial index — and surviving direct writes is the entire purpose of those indexes. |
| `/healthz` became `/health` | **DEF-08.** Google Front End intercepts `/healthz` on Cloud Run before the request reaches the container. |
| `validate_contribution` gained `days_covered` | FR-15 (catch-up payments) is deferred, but parameterising the rule means enabling it later needs no change to the business rules — only an allocation step in the service layer. |
| A health endpoint was added | Not in the original design; required by the deployment target. |

## 10.6 Self-admitted technical debt in the source

Session 3's SATD convention, used with its conventional force: `TODO` for work not done, `FIXME` for something that works but is wrong, `HACK` for a knowingly poor mechanism. Each marker names its register entry, so code and analysis reach each other.

```
app/domain/rules.py:41               TODO(TD-13)   cycle length is a constant
app/domain/rules.py:174              FIXME(TD-11)  summary recomputed per render
app/infrastructure/db.py:28          FIXME(TD-01)  create_all, no migrations
app/infrastructure/models.py:231     HACK(TD-09)   audit append-only by convention
app/infrastructure/repositories.py:6 TODO(TD-07)   hand-written mapping
app/services/collection.py:338       TODO(TD-05)   static route sheet
app/services/collection.py:344       FIXME(TD-16)  N+1 on the route sheet
app/services/security.py:38          FIXME(TD-14)  no login rate limiting
app/web/auth.py:52                   TODO(TD-15)   no password reset
app/web/client.py:60                 FIXME(TD-16)  N+1 in cycle history
app/web/supervisor.py:65             TODO(TD-04)   bare reversal form
requirements.txt:5                   TODO(TD-12)   unpinned dependencies
```

## 10.7 Development process

Seventeen commits, conventional-commit format, each recording what changed and why. Notable process facts:

- **CR-001** (QR client identification) was raised mid-project and put through the change control process defined in `03-requirements.md` §3.8 — options costed, impact traced through the traceability matrix, estimate revised, and a 6.4% schedule overrun **accepted and logged rather than absorbed** by manufacturing further cuts.
- Eight defects were found and closed; five by tests or probing, one by a user, two by inspection.
- Two of the developer's own test assumptions were found wrong and corrected rather than worked around (`09-testing.md` §9.8).

- Container images are tagged with the git SHA, so any deployed revision is traceable to a commit.

## 10.8 Size: estimate against actual

Session 6's closing practice is to record actuals and review estimation accuracy. The size estimate can be closed out precisely.

| | Estimated | Actual | Variance |
|---|---|---|---|
| Adjusted function points | 166 | — | — |
| LOC per function point (AS-01) | 30 | **20.7** | −31% |
| Source size | 4.98 KLOC | **3.43 KLOC** | **−31%** |
| COCOMO effort at that size | 12.95 PM | 8.76 PM | −32% |

**MRE on size = 0.45.**

**The finding is that assumption AS-01 was wrong, and wrong in a specific direction.** 30 LOC per function point was taken from published averages for the language class. The actual figure was **20.7** — below even the optimistic 25 used in the sensitivity analysis (§4.3.4).

Two plausible causes, both consistent with the code as written. Flask, SQLAlchemy and Jinja supply as configuration what the published averages assume is written by hand — routing, session handling, ORM persistence, CSRF, templating. And the function point count included work that produces very little code: the QR card is 5 function points and roughly 15 lines, because `segno` does the work.

The sensitivity analysis in §4.3.4 anticipated this class of error but not its size, having bounded the range at 25–40. Had the range been set from measurement rather than from published tables, it would have started lower.

**This does not change the conclusion of §4.4.** Even at the actual size, COCOMO puts the system at 8.76 person-months — over 1,300 person-hours against a 48-hour window. The gap is smaller than estimated and remains of the same order.

# 9. Testing and Quality Assurance

> Examination Part A §8. Test cases are recorded as **Test case → Expected result → Actual result → Pass/fail**, with a defect log and corrective actions in §9.8.

## 9.1 Test strategy

Testing was shaped by one architectural decision made in Phase 2: **the domain layer imports neither Flask nor SQLAlchemy**. That is what made a real test suite affordable inside the examination window — business rules can be tested with no database, no fixtures and no HTTP client, so the 129 unit tests run in **0.34 seconds**. An Active-Record design would have required a live database for every rule test, and under a 20-hour implementation budget those tests would simply not have been written.

Three levels, each covering what the level below cannot:

| Level | Count | Runtime | Covers | Deliberately excludes |
|---|---|---|---|---|
| **Unit** | 129 | 0.34 s | Business rules BR-R1…R15, `Money`, service orchestration, authorisation logic, audit writes | Anything requiring SQL |
| **Integration** | 42 | 3.3 s | Repositories, entity/record mapping, and above all the **database-enforced invariants** | HTTP, sessions, templates |
| **System** | 55 | 10.4 s | Routing, authentication, role authorisation, CSRF, template rendering, HTMX error path, complete user journeys | Real browsers, real devices |
| **Total** | **226** | **14.3 s** | | |

**Why the integration level exists at all.** The design claims three business invariants are enforced *twice* — in the domain layer and again by PostgreSQL partial unique indexes. A claim about PostgreSQL behaviour cannot be verified by a fake. Those tests therefore write **through the ORM, bypassing the service layer entirely**, to demonstrate that the guarantee does not depend on application code being correct.

## 9.2 Test environment

| | |
|---|---|
| Runner | pytest 8 with pytest-cov |
| Application | Python 3.12.2, Flask 3, SQLAlchemy 2 |
| Database | PostgreSQL 16 (Docker), separate `susubook_test` database, truncated between tests |
| Clock | Injected ( `clock=lambda: date(2026, 9, 15)` ) so date-dependent rules are deterministic |
| Isolation | `TRUNCATE ... RESTART IDENTITY CASCADE` per test rather than transaction rollback, because several tests deliberately provoke integrity errors that would poison an outer transaction |

## 9.3 Coverage

```
app/domain/rules.py                 100%     app/services/reconciliation.py   100%
app/domain/errors.py                100%     app/services/security.py         100%
app/domain/entities.py               99%     app/services/container.py        100%
app/domain/money.py                  94%     app/services/collection.py        99%
app/infrastructure/models.py        100%     app/services/payout.py            98%
app/infrastructure/repositories.py   99%     app/web/collector.py              90%
app/infrastructure/qrcodes.py       100%     app/web/supervisor.py             93%
app/config.py                       100%     app/web/security.py               93%
                                             app/web/client.py                 85%
                            TOTAL    97%
```

NFR-07 requires ≥70% on the domain layer. Achieved: **99–100%** on domain, **97%** overall. The uncovered remainder is chiefly defensive branches — `NotImplemented` returns on `Money` arithmetic, and the `AssertionError` guard in `compute_payout` that is unreachable while the commission policy is correct.

---

## 9.4 Functional test cases

Legend: **P** pass · **F** fail. All results are from the run of the committed suite; no result is stated from memory.

### TC-AUTH — Authentication (UC-01, FR-01…04)

| ID | Test case | Expected | Actual | |
|---|---|---|---|---|
| TC-AUTH-01 | Anonymous request to `/` | Redirect to `/login` | 302 to `/login` | **P** |
| TC-AUTH-02 | Valid collector credentials | 302 to collector landing page | 302 to `/collector/` | **P** |
| TC-AUTH-03 | Valid client credentials | 302 to `/my/` | 302 to `/my/` | **P** |
| TC-AUTH-04 | Valid supervisor credentials | 302 to `/supervisor/` | 302 to `/supervisor/` | **P** |
| TC-AUTH-05 | Correct phone, wrong password | 401, generic message | 401, "Phone number or password is incorrect" | **P** |
| TC-AUTH-06 | Unknown phone number | Identical status and message to TC-AUTH-05 | Both 401, both same message | **P** |
| TC-AUTH-07 | Logout | Session cleared, protected page redirects | 302 on subsequent `/collector/` | **P** |
| TC-AUTH-08 | Failed login is audited | `LOGIN_FAILED` row written | Row present with reason | **P** |

> TC-AUTH-06 is a security test in functional clothing: distinguishing an unknown phone from a wrong password would let an attacker enumerate which numbers are registered.

### TC-AUTHZ — Authorisation (FR-05, NFR-03, BR-R15)

| ID | Test case | Expected | Actual | |
|---|---|---|---|---|
| TC-AUTHZ-01 | Unauthenticated access to `/collector/`, `/supervisor/`, `/my/` | All redirect | All 302 | **P** |
| TC-AUTHZ-02 | Collector opens supervisor pages | 403 | 403 | **P** |
| TC-AUTHZ-03 | Client opens collector pages | 403 | 403 | **P** |
| TC-AUTHZ-04 | Collector opens a client self-service page | 403 | 403 | **P** |
| TC-AUTHZ-05 | **Collector B opens Collector A's client via QR reference** | 403, "not on your route" | 403, message present | **P** |
| TC-AUTHZ-06 | Collector B posts a collection for that client | 403, nothing written | 403, no contribution row | **P** |
| TC-AUTHZ-07 | Denied attempt is audited | `AUTHORISATION_DENIED` row | Row present | **P** (after DEF-05) |
| TC-AUTHZ-08 | Collector B prints that client's QR card | 403 | 403 | **P** |

> **TC-AUTHZ-05 through 08 are the photographed-card threat**, and the reason BR-R15 exists. A QR card carried through a market must be assumed photographable; these cases demonstrate that possession of the reference confers no capability, because the authorisation decision never consults it.

## TC-COL — Recording a contribution (UC-03)

| ID | Test case | Expected | Actual | |
|---|---|---|---|---|
| TC-COL-01 | Route sheet lists only own clients | Other collectors' clients absent | Absent | **P** |
| TC-COL-02 | Scan landing page | Pre-filled daily rate, confirm button | "GHS 10.00", "Confirm collection" | **P** |
| TC-COL-03 | Record a contribution | 302, one row, reference assigned | 302, `SB-` reference present | **P** |
| TC-COL-04 | **Second contribution, same client, same day** (BR-R5) | 422, existing reference quoted | 422, "already recorded (reference SB-…)" | **P** |
| TC-COL-05 | Duplicate over HTMX | 422 **fragment**, not a full page | Fragment, no `<html>` | **P** |
| TC-COL-06 | Amount not a multiple of the rate (BR-R7) | 422 | 422, "whole multiple" | **P** |
| TC-COL-07 | Unparseable amount ("abc") | 422 | 422 | **P** |
| TC-COL-08 | Unknown client reference | 404 | 404 | **P** |
| TC-COL-09 | Future-dated contribution (BR-R4) | Refused | `ContributionDateInFuture` raised | **P** |
| TC-COL-10 | Contribution outside cycle dates (BR-R3) | Refused | `ContributionDateOutsideCycle` raised | **P** |
| TC-COL-11 | Contribution into a closed cycle (BR-R6) | Refused | `CycleClosed` raised | **P** |
| TC-COL-12 | **HTMX collect updates the running total out of band** (DEF-06) | Response carries the day's total marked for out-of-band swap, with the new value | `id="recorded-today"`, `hx-swap-oob="true"`, "Recorded GHS 10.00" | **P** |
| TC-COL-13 | Route sheet total after a collection | GHS 0.00 before, GHS 10.00 after | As expected | **P** |

### TC-ENR — Enrolment (UC-02)

| ID | Test case | Expected | Actual | |
|---|---|---|---|---|
| TC-ENR-01 | Enrol a client | Client, login and cycle 1 created atomically | All three present, cycle ACTIVE | **P** |
| TC-ENR-02 | New client signs in immediately | 302 to `/my/` | 302 to `/my/` | **P** |
| TC-ENR-03 | Password shorter than 6 characters | 422 | 422 | **P** |
| TC-ENR-04 | Zero daily rate | Refused | "must be more than zero" | **P** |
| TC-ENR-05 | Client receives an opaque public reference | UUIDv4, not the row id | 36-char UUID, differs from id | **P** |
| TC-ENR-06 | Cycle snapshots the daily rate | Cycle rate equals agreed rate | Equal | **P** |

### TC-PAY — Payout (UC-07)

| ID | Test case | Expected | Actual | |
|---|---|---|---|---|
| TC-PAY-01 | Matured cycle appears for release | Client listed | Listed | **P** |
| TC-PAY-02 | Release computes commission (BR-R8) | 10 days × GHS 10 → total 100, commission 10, net 90 | 10 000 / 1 000 / 9 000 pesewas | **P** |
| TC-PAY-03 | Money is conserved | net + commission = total | Asserted equal | **P** |
| TC-PAY-04 | Cycle closes and the next opens | `[PAID_OUT, ACTIVE]` | `[PAID_OUT, ACTIVE]` | **P** |
| TC-PAY-05 | New cycle inherits the rate | Same daily rate | Equal | **P** |
| TC-PAY-06 | **One-day client (BR-R9)** | Net exactly zero, never negative | total 1 000, commission 1 000, net 0 | **P** |
| TC-PAY-07 | Client who paid nothing | Zero throughout, no negative | All zero | **P** |
| TC-PAY-08 | **Payout never negative, all 32 completion levels** | Non-negative and conserved for 0…31 days | All 32 pass | **P** |
| TC-PAY-09 | Second release (BR-R10) | 422 | 422, "already been paid out" | **P** |
| TC-PAY-10 | Release before maturity (BR-R12) | Refused | `CycleNotMatured` raised | **P** |
| TC-PAY-11 | Collector attempts release | 403 | 403 | **P** |
| TC-PAY-12 | Release is audited | `RELEASE_PAYOUT` with settlement | Row with all three amounts | **P** |

> TC-PAY-08 is exhaustive rather than sampled. BR-R9's edge case — a client whose entire balance is consumed by commission — is exactly where an off-by-one produces a *negative payout*, so every completion level from 0 to 31 days is checked for non-negativity and conservation.

### TC-REC — Reconciliation (UC-05, UC-06, FR-24…26)

| ID | Test case | Expected | Actual | |
|---|---|---|---|---|
| TC-REC-01 | Declared cash matches recorded | Variance zero, reconciled | GHS 0.00, reconciled | **P** |
| TC-REC-02 | **Declared less than recorded** | Shortfall shown to supervisor | GHS 7.00 shortfall, "unreconciled" | **P** |
| TC-REC-03 | Collector never declared | Flagged as not declared | "NOT DECLARED" | **P** |
| TC-REC-04 | Collector recorded nothing | Still listed, zeroes | Listed with zeroes | **P** |
| TC-REC-05 | Negative declaration | Refused | "cannot be negative" | **P** |
| TC-REC-06 | Future-dated declaration | Refused | "future date" | **P** |
| TC-REC-07 | Collector views all variances | 403 | 403 | **P** |
| TC-REC-08 | Re-declaring the same day overwrites | Latest value stored | GHS 25.00 replaces GHS 10.00 | **P** |
| TC-REC-09 | Declaration is audited with the variance | Variance in audit detail | 700 pesewas recorded | **P** |

> TC-REC-04 matters more than it looks: a collector who simply stopped working must still appear on the variance screen. If idle collectors dropped off the report, the most suspicious case would be the one that became invisible.

### TC-REV — Reversal (UC-09, BR-R11)

| ID | Test case | Expected | Actual | |
|---|---|---|---|---|
| TC-REV-01 | Supervisor reverses a contribution | Original preserved and linked | 2 rows, `reversed_by_id` set | **P** |
| TC-REV-02 | Reversal frees the day for a replacement | New contribution accepted | 302, accepted | **P** |
| TC-REV-03 | Reversing twice | Refused | "already been reversed" | **P** |
| TC-REV-04 | Reason is required | 422 | 422 | **P** |
| TC-REV-05 | Collector attempts reversal | 403 | 403 | **P** |
| TC-REV-06 | Reversal is audited with the reason | Reason in audit detail | Present | **P** |
| TC-REV-07 | **Client still sees both entries** | REVERSED and REVERSAL both visible | Both rendered | **P** |

### TC-CLI — Client self-service (UC-08, FR-28…30)

| ID | Test case | Expected | Actual | |
|---|---|---|---|---|
| TC-CLI-01 | Client sees their own contributions | Card renders with entries | 200, entries present | **P** |
| TC-CLI-02 | **Each entry names the recording collector** | Collector name shown | "Joseph Osei" present | **P** |
| TC-CLI-03 | Balance and projected payout shown | "You will receive" | Present | **P** |
| TC-CLI-04 | Card renders every day of the cycle | 31 boxes | 31 `role="listitem"` | **P** |
| TC-CLI-05 | Past cycles page | 200 | 200 | **P** |
| TC-CLI-06 | Reversed entries remain visible to the client | Both labels shown | REVERSED and REVERSAL present | **P** |

> TC-CLI-02 is the single most important functional test in this document. It is the answer to problem **P1**: the client's record is independent of the collector, and every entry is attributable.

### TC-QR — QR client cards (FR-39, FR-40, CR-001)

| ID | Test case | Expected | Actual | |
|---|---|---|---|---|
| TC-QR-01 | Card page renders inline SVG | `<svg>` present | Present | **P** |
| TC-QR-02 | Card discloses no phone number | Phone absent from page | Absent | **P** |
| TC-QR-03 | Another collector cannot print the card | 403 | 403 | **P** |

### TC-DB — Database-enforced invariants (defence in depth)

Written through the ORM, **bypassing the service layer**, so the guarantee is shown not to depend on application code.

| ID | Test case | Expected | Actual | |
|---|---|---|---|---|
| TC-DB-01 | Second ACTIVE cycle for one client (BR-R2) | Rejected | `ux_active_cycle_per_client` violation | **P** |
| TC-DB-02 | A new cycle after the first is paid out | Permitted | Committed | **P** |
| TC-DB-03 | Two clients each with an active cycle | Permitted | Committed | **P** |
| TC-DB-04 | Duplicate contribution on one day (BR-R5) | Rejected | `ux_effective_contribution_per_day` violation | **P** |
| TC-DB-05 | **Replacement shares a date with a reversed entry** | Permitted; all three rows survive | Committed, 3 rows, 1 effective | **P** |
| TC-DB-06 | Duplicate contribution reference | Rejected | Unique violation | **P** |
| TC-DB-07 | Second payout for one cycle (BR-R10) | Rejected | Unique violation | **P** |
| TC-DB-08 | **Payout where net + commission ≠ total** | Rejected | `ck_payout_balances` violation | **P** |
| TC-DB-09 | Negative net payout | Rejected | Check violation | **P** |
| TC-DB-10 | Zero or negative daily rate | Rejected | `ck_client_rate_positive` | **P** |
| TC-DB-11 | Zero contribution amount | Rejected | `ck_contribution_positive` | **P** |
| TC-DB-12 | Cycle end date before start | Rejected | `ck_cycle_dates` | **P** |
| TC-DB-13 | Unknown user role | Rejected | `ck_user_role` | **P** |
| TC-DB-14 | Two remittance declarations, one collector, one day | Rejected | `uq_declaration_per_day` | **P** |

> **TC-DB-05 is the test that protects the design.** The index on contributions is *partial* precisely so a reversed entry, its reversal, and the replacement may share a date. A future developer "simplifying" it to a plain `UNIQUE(cycle_id, contribution_date)` would make correction by reversal impossible — and this test is what would stop them.

### TC-MON — Money handling (BR-R1, NFR-04)

| ID | Test case | Expected | Actual | |
|---|---|---|---|---|
| TC-MON-01 | Construct from "2.50" | 250 pesewas | 250 | **P** |
| TC-MON-02 | **Construct from float 2.50** | `TypeError` | Raised | **P** |
| TC-MON-03 | Multiply by float | `TypeError` | Raised | **P** |
| TC-MON-04 | `Money(True)` — bool is an int subclass | `TypeError` | Raised | **P** |
| TC-MON-05 | Fractional pesewa ("2.505") | `ValueError` | Raised | **P** |
| TC-MON-06 | **Exact accumulation across all cycle lengths 1–31** | Exact at every length | All exact | **P** |
| TC-MON-07 | Money survives the ORM mapping | 1234 pesewas in and out | Exact, still `int` | **P** |

---

## 9.5 Security testing (NFR-03)

| ID | Test case | Expected | Actual | |
|---|---|---|---|---|
| TC-SEC-01 | POST without a CSRF token, enforcement on | Rejected | 400 | **P** |
| TC-SEC-02 | Password stored hashed | Argon2id, plaintext absent | `$argon2…` , plaintext absent | **P** |
| TC-SEC-03 | Password never rendered in a page | Absent from response | Absent | **P** |
| TC-SEC-04 | URLs expose opaque references, not row ids (BR-R14) | UUID in links, no `/client/1` | UUID present, sequential absent | **P** |
| TC-SEC-05 | Account enumeration via login response | Indistinguishable | Same status and message | **P** |
| TC-SEC-06 | Forced browsing to another role's pages | 403 | 403 | **P** |
| TC-SEC-07 | Unknown route | 404, no stack trace | 404 | **P** |

**Security findings not covered by a passing test**, carried into the debt register rather than presented as satisfied:

- **TD-14** — no rate limiting or lockout. Brute force is possible; failed attempts are audited, so an attack is visible afterwards but nothing stops it. Classified **Critical**.

- **TD-15** — the collector sets the client's initial password and there is no forced change, so the collector can sign in as the client. Classified **Critical**, because it undermines the independence the system exists to provide.
- **TD-09** — the audit log is append-only by application convention, not by database permission. Classified **Critical**.

No penetration testing or automated dependency vulnerability scanning was performed; both are named as gaps in §9.10.

## 9.6 Performance testing (NFR-01)

Server-side render time, median of five requests against local PostgreSQL with seeded data (10 clients, ~170 contributions):

| Page | Server time | HTML size |
|---|---|---|
| Collector route sheet | **9 ms** | 7.5 KB |
| Enrolment form | **2 ms** | 4.3 KB |
| Supervisor variances | **3 ms** | 2.3 KB |
| Client susu card (31 boxes + full history) | **12 ms** | 28.6 KB |

**Interpretation, stated carefully.** These measure *server* time only. NFR-01 specifies 2 seconds end-to-end on a 3G-class connection, which server time does not establish: the dominant cost on that connection is network transfer plus the render-blocking Tailwind CDN request (**TD-02**), neither of which is measured here. The honest reading is that the application's own work leaves the entire budget available for transport — and that **TD-02 is currently the largest threat to NFR-01**, which is why it is scheduled for repayment.

The 28.6 KB client card is the largest page and grows with contribution count; the route sheet's N+1 (**TD-16**) grows with route size. Neither is a problem at demonstration scale and both are recorded.

**Not performed:** load testing, concurrency testing, testing against a real 3G connection or a real low-end Android device. Named as gaps in §9.10.

## 9.7 Requirements verification

| Requirement | Status | Evidence |
|---|---|---|
| FR-01…05 authentication and authorisation | ✅ Satisfied | TC-AUTH, TC-AUTHZ |
| FR-06, FR-07 enrolment | ✅ Satisfied | TC-ENR |
| **FR-08 client list and search** | ⚠️ **Partial** | List renders; **search and pagination not implemented (TD-03)** |
| FR-10…14 cycles and validation | ✅ Satisfied | TC-COL-03…11, TC-DB-01…06 |
| FR-16, FR-17 susu card and summary | ✅ Satisfied | TC-CLI-04, unit summary tests |
| FR-18…21 payout | ✅ Satisfied | TC-PAY |
| **FR-23 route sheet** | ⚠️ **Partial** | Renders with status; **not filtered or route-ordered (TD-05)** |
| FR-24…26 reconciliation | ✅ Satisfied | TC-REC |
| FR-28…30 client transparency | ✅ Satisfied | TC-CLI |
| FR-32, FR-33 audit and reversal | ✅ Satisfied | TC-REV, audit assertions |
| FR-39, FR-40 QR cards | ✅ Satisfied | TC-QR |
| NFR-01 performance | ⚠️ **Partial** | Server time measured; end-to-end on 3G not measured |
| NFR-02 usability (≤3 interactions) | ⚠️ **Unverified** | Design achieves 2 by inspection; **not measured with a participant** |
| NFR-03 security | ⚠️ **Partial** | TC-SEC passes; TD-09, TD-14, TD-15 outstanding |
| NFR-04 money integrity | ✅ Satisfied | TC-MON, TC-DB-08 |
| NFR-06 data minimisation | ✅ Satisfied | Schema review, TC-QR-02 |
| NFR-07 maintainability (≥70% domain) | ✅ Satisfied | 99–100% domain coverage |
| NFR-08 accessibility | ⚠️ **Partial** | Shape+text encoding and 44px targets implemented; **no contrast audit or screen-reader test** |
| NFR-09 auditability | ✅ Satisfied | Audit assertions across all suites |
| NFR-10 portability | ✅ Satisfied | Same engine dev and prod; deploy verification pending Phase 5 |

Three requirements are **partially** satisfied and three NFRs are **unverified or partial**. They are reported as such rather than claimed complete.

## 9.8 Defect log

Defects found during development, with corrective action. All are closed.

| ID | Defect | Found by | Severity | Corrective action | Status |
|---|---|---|---|---|---|
| **DEF-01** | A unit test and a module docstring both asserted that 31 × GHS 0.10 accumulates to 3.0000000000000004 in floating point. It does not — that sum rounds back to exactly 3.1. The example did not demonstrate the error it claimed. | Unit suite (the test failed) | Low (documentation), High (reasoning) | Verified empirically; corrected to n=29 (2.9000000000000004). Test rewritten to assert exactness across the whole 1–31 range rather than at a single sample, and to record that float error here is *intermittent* — which is worse than consistently wrong, because it survives casual testing. | Closed |
| **DEF-02** | Boolean and status columns carried Python-side defaults only. A direct SQL insert omitting them failed on NOT NULL *before* the partial unique index was consulted. | Manual SQL probing of the invariants | Medium | Added `server_default`. Surviving direct writes is the entire purpose of those indexes, so a schema that only works when written through the ORM defeated the design. | Closed |
| **DEF-03** | `Flask(__name__)` looked for templates at the package root; every page returned 500. | Smoke test after first run | High | Pointed `template_folder` at `app/web/templates` to match the layering. | Closed |
| **DEF-04** | `cycle_days` was registered as a context processor, but Jinja macros do not receive the template context. Every susu card view raised `UndefinedError` — the client card, the collector's client detail, both broken. | Manual page walk | High | Moved to `app.jinja_env.globals`. | Closed |
| **DEF-05** | `_assert_on_route` in the web layer raised `NotAuthorised` **without auditing**, while the service-layer path did audit. A collector presenting a client reference they should not hold was recorded only if they attempted a *write*; a denied GET on a scanned card left no trace. | System suite (TC-AUTHZ-07) | Medium–High | Audit on both paths. The denial arrives on the GET, before any write is attempted, and is exactly the signal a supervisor needs. | Closed |
| **DEF-06** | On the route sheet, recording a contribution over HTMX swapped the client's row to "Paid" but | **Manual exploratory testing by** | Medium | `hx-target` swapped only the row; the total sat outside it. Returned the total in the same response as an **out-of-band swap** ( `hx-swap-oob` ), so | Closed |

| ID | Defect | Found by | Severity | Corrective action | Status |
|---|---|---|---|---|---|
| | left the day's running total unchanged until a manual refresh. The row and the total disagreed on the same screen. | **the project owner** | | one request updates both. Two regression tests added. | |
| **DEF-07** | The security test asserting that URLs expose opaque references, not sequential ids, was **flaky**: it passed alone and failed roughly one full-suite run in sixteen. | Full-suite run after fixing DEF-06 | Medium | The negative assertion used substring matching — `/collector/client/1` is a substring of `/collector/client/1a2b3c…`, so it failed whenever a random UUID began with the digit 1. Rewritten to match a complete numeric path segment by regex. Verified stable over five consecutive full runs. | Closed |

**Two of my own test assumptions were also wrong** and were corrected rather than worked around, which is worth recording because the temptation was to adjust the assertion until it passed:

| | Wrong assumption | Reality | Resolution |
|---|---|---|---|
| 1 | Setting `status = 'MATURED'` makes a cycle due for payout | `list_due_for_payout` filters on `end_date < today`, which is what BR-R12 actually describes | Tests now move the cycle genuinely into the past |
| 2 | `list_for_target(type, None)` returns nothing, since `= NULL` never matches | SQLAlchemy renders `column == None` as `IS NULL`, so it does match | Test rewritten to assert what it actually claimed — that an anonymous actor can be stored |

**Defect density:** 7 defects across roughly 1,090 statements. Five (DEF-01, DEF-02, DEF-05, DEF-07, and the two corrected assumptions) were found by tests or by probing rather than by a user, which is the outcome the test strategy was designed to produce.

**DEF-06 is the exception, and the instructive one.** It was found by a person clicking through the interface, not by any of the 209 automated tests — and it could not have been found by them, because every test asserted the *response* to a collect request and none asserted the *state of the page afterwards*. The suite verified that the right row came back; only a human noticed that the total above it now disagreed. It is the clearest argument in this project for the user acceptance testing recorded as outstanding in §9.9: automated tests confirm what you thought to check, and a user finds what you did not.

**DEF-07 is worth recording for the opposite reason.** A test that fails intermittently is worse than no test, because it teaches you to disregard a red result — and this one guarded a security property (BR-R14). It was fixed rather than retried, and stability was then demonstrated over five consecutive full runs rather than assumed.

## 9.9 User acceptance testing — status

**Not yet conducted with an independent participant.** This is stated plainly rather than substituted for.

The 51 system tests exercise complete user journeys end to end, but they were written by the developer and are therefore **system tests, not user acceptance tests**. Presenting them as UAT would misrepresent what they establish.

The prepared UAT protocol, for execution with a participant:

| ID | Scenario | Role | Acceptance criterion |
|---|---|---|---|
| UAT-01 | Sign in and record a day's collection for three clients | Collector | Completed without assistance; each recorded in ≤3 interactions |
| UAT-02 | Attempt to collect twice from one client on the same day | Collector | Refusal understood without explanation |
| UAT-03 | Enrol a new client and print their QR card | Collector | Client, login and card produced; card scans |
| UAT-04 | Declare the day's cash, under-declaring deliberately | Collector | Variance understood as a discrepancy |
| UAT-05 | Sign in and confirm every payment made this week | Client | Locates own record; confirms amounts and collector names |
| UAT-06 | State what will be received at maturity and why the deduction exists | Client | Identifies payout and explains the one-day commission |
| UAT-07 | Identify which collector is unreconciled today | Supervisor | Locates the variance without prompting |
| UAT-08 | Release a matured payout | Supervisor | Understands the settlement before confirming |
| UAT-09 | Reverse a wrongly recorded contribution | Supervisor | Confirms the original remains visible |

**Usability measures to be captured alongside** (Session 5 [5]): task completion rate, time on task, error rate, and a 10-item System Usability Scale score [22]. The nine-scenario protocol uses a small participant group on Nielsen's finding that five users surface the large majority of usability problems [24]. NFR-02's claim that a routine collection takes ≤3 interactions is currently established **by design inspection only** — scan plus confirm is two — and needs a measured value from a real participant to be reported as verified.

## 9.10 Known gaps in testing

Recorded so the coverage claim is not overstated:

| Gap | Consequence | Why |
|---|---|---|
| No UAT with an independent participant | NFR-02 unverified; usability findings unknown | No participant available within the window (§9.9) |
| No real-device or real-network testing | NFR-01 unverified end-to-end; NFR-08 outdoor legibility unverified | No 3G connection or low-end Android available |
| No load or concurrency testing | Behaviour under simultaneous collectors unknown | Out of scope for the window |
| No accessibility audit or screen-reader test | NFR-08 partially verified | Requires tooling and time not budgeted |
| No penetration testing or dependency vulnerability scan | Unknown vulnerabilities may exist | Out of scope; TD-12 also leaves dependencies unpinned |
| No browser-matrix testing | Rendering verified in Chromium only | Single-browser check only |
| QR scanning not tested on a physical printed card | FR-40 verified as a URL route, not as an optical read | No printer or camera test performed |

The last is worth emphasising: **TC-QR verifies that the card renders and that the route resolves and authorises correctly, but nobody has printed a card and scanned it with a phone.** That step belongs in UAT-03.

# 8. Technical Debt Identification and Management

> Examination Part A §7. Every item below records **Debt → Cause → Impact → Priority → Proposed Resolution**, and is classified as *acceptable temporarily*, *scheduled for future resolution*, or *critical and requiring immediate attention*.

---

## 8.1 Definition and framing

The metaphor originates with Cunningham [21]; the four-way classification used throughout this section is Fowler's quadrant [23].

Session 3 [3] defines technical debt as *the long-term cost incurred when developers take shortcuts or implement suboptimal solutions in order to achieve short-term goals such as faster delivery*, and **self-admitted technical debt (SATD)** as debt the developer explicitly documents in code comments.

This project treats that literally. Debt here is not something discovered by inspecting the finished system: it was **identified before it was incurred**, and every item is marked in the source at the site that carries it.

## 8.2 How debt entered this project — three distinct routes

| Route | When | Items | Character |
|---|---|---|---|
| **The scope decision** | Phase 1, before any code | TD-01… TD-06 | Chosen. The effort estimate exceeded the budget by 23%, so 4.1 hours of implementation quality were removed deliberately ( `05-effort-estimation.md` §4.6). |
| **Design decisions** | Phase 2, before any code | TD-07… TD-13 | Consequential. The layered architecture, the free-tier constraint and deferred requirements each carry a known cost. |
| **Implementation and testing** | Phase 3–4 | TD-14… TD-17 | Discovered. Found while building and while writing tests — the only items not anticipated. |

Thirteen of seventeen items were identified **before the code that carries them existed**. That is the substantive claim of this section, and it is why the debt register and the effort estimate are the same document read twice.

## 8.3 Self-admitted technical debt in the source

Session 3's SATD markers, present in the shipped code:

```
app/domain/rules.py:41              TODO(TD-13)   cycle length is a constant
app/domain/rules.py:174             FIXME(TD-11)  summary recomputed per render
app/infrastructure/db.py:28         FIXME(TD-01)  create_all, no migrations
app/infrastructure/models.py:231    HACK(TD-09)   audit append-only by convention
app/infrastructure/repositories.py:6 TODO(TD-07)  hand-written mapping
app/services/collection.py:344      FIXME(TD-16)  N+1 on the route sheet
app/services/collection.py:338      TODO(TD-05)   static route sheet
app/services/security.py:38         FIXME(TD-14)  no login rate limiting
app/web/auth.py:52                  TODO(TD-15)   no password reset
app/web/client.py:60                FIXME(TD-16)  N+1 in cycle history
app/web/supervisor.py:65            TODO(TD-04)   bare reversal form
app/web/templates/base.html:10      TD-02         Tailwind via CDN
requirements.txt:5                  TODO(TD-12)   unpinned dependencies
```

Each marker names the register ID, so a developer reading the code reaches the analysis, and a reader of this document reaches the code. Markers are used with Session 3's conventional force: `TODO` for work not done, `FIXME` for something that works but is wrong, `HACK` for a knowingly poor mechanism.

---

## 8.4 The debt register

### Critical — requiring immediate attention

---

**TD-14 · No login rate limiting or account lockout** `app/services/security.py:38` · Code · Prudent & Deliberate

| | |
|---|---|
| **Debt** | Authentication accepts unlimited attempts. No backoff, no lockout, no CAPTCHA, no per-IP throttle. |
| **Cause** | Rate limiting needs either a shared store (Redis) or a database-backed attempt counter. Neither fitted the 48-hour budget, and the free tier provides no Redis. |
| **Impact** | An online brute-force attack against a savings system is possible. Argon2id makes each guess expensive — which raises the cost of an attack but does not bound it. Failed attempts are audited, so an attack is *visible afterwards*; nothing *prevents* one. Phone numbers are the username, and Ghanaian mobile numbers are highly guessable in format. |
| **Priority** | **Critical.** The single most serious item in this register. |
| **Resolution** | Add an `login_attempts` table keyed on phone and IP; exponential backoff after 3 failures; lock for 15 minutes after 10. Estimated 2–3 hours. Must ship before any real client data enters the system. |

---

**TD-15 · The collector sets and therefore knows the client's password** `app/web/auth.py:52` · Code · Prudent & Inadvertent

| | |
|---|---|
| **Debt** | At enrolment the collector types the client's initial password. There is no forced change at first login and no password reset flow at all. |
| **Cause** | An out-of-band credential channel (SMS one-time code) requires the SMS gateway excluded by CO-04 and FR-31. Enrolment had to produce a usable client login somehow. |
| **Impact** | **This one attacks the system's own premise.** SusuBook exists so the client holds a record independent of the collector (BR-02). If the collector knows the client's password, the collector can sign in as the client — and the independence is nominal rather than real. A client who forgets the password also has no recovery route and is locked out permanently. |
| **Priority** | **Critical.** Not for convenience — because it undermines the property the system is built to provide. |
| **Resolution** | Force a password change on first login, before any record is displayed. Add reset via SMS one-time code when FR-31's gateway lands. Interim mitigation: display a prominent banner until the client changes it, and audit every client-role login so an impersonation is at least visible. Estimated 3–4 hours. |

---

### TD-09 · Audit log is append-only by convention, not by enforcement

`app/infrastructure/models.py:231` · Architecture/Design · Prudent & Deliberate

| | |
|---|---|
| **Debt** | Nothing in the application updates or deletes `audit_log` rows, but the database does not prevent it. The application role holds full UPDATE and DELETE. |
| **Cause** | Enforcement needs a separate least-privilege database role, or a rule/trigger rejecting UPDATE and DELETE. Both need migration infrastructure that TD-01 removed. |
| **Impact** | The audit trail carries non-repudiation (BR-05, NFR-09) — it is the evidence that answers a dispute between client and collector. A compromised application account, or an operator with the production connection string, could rewrite history and leave no trace. The guarantee is currently a promise about the code rather than a property of the system. |
| **Priority** | **Critical.** The whole value of an audit trail is that it cannot be edited. |
| **Resolution** | Revoke UPDATE and DELETE on `audit_log` from the application role; grant INSERT and SELECT only. Add a `BEFORE UPDATE OR DELETE` trigger raising an exception as defence in depth. Estimated 1–2 hours once migrations exist, so it is sequenced after TD-01. |

---

## Scheduled for future resolution

---

### TD-01 · Schema created with `create_all()`; no versioned migrations `app/infrastructure/db.py:28` · Infrastructure · Prudent & Deliberate

| | |
|---|---|
| **Debt** | `Base.metadata.create_all()` plus a seed script. No Alembic, no migration history, no downgrade path. |
| **Cause** | Cut deliberately in the scope decision to recover 0.7 hours ( `05-effort-estimation.md` §4.6). |
| **Impact** | Any schema change in production requires hand-written DDL against live data, with no review artefact and no rollback. Blocks TD-09's resolution. Risk grows the moment real data exists, because `create_all` cannot alter an existing table. |
| **Priority** | **Scheduled — first item.** It is a prerequisite for several others. |
| **Resolution** | Introduce Alembic; autogenerate an initial revision matching the current schema; make it a deployment step. Estimated 2 hours. |

---

### TD-16 · N+1 queries on the route sheet and cycle history `app/services/collection.py:344`, `app/web/client.py:60` · Code · Prudent & Inadvertent

| | |
|---|---|
| **Debt** | `route_sheet` queries the active cycle once per client; `history` runs two queries per cycle. |
| **Cause** | Not anticipated — found while auditing the code during Phase 4. The list comprehension reads cleanly and hides the repeated call. Genuinely inadvertent. |
| **Impact** | The route sheet is the collector's most-loaded screen, opened dozens of times a day on mobile data, and it is the screen NFR-01's 2-second budget most applies to. Cost grows linearly with route size: fine at 20 clients, poor at 200. |
| **Priority** | **Scheduled.** Not yet user-visible at demonstration scale, and it becomes so predictably. |
| **Resolution** | Replace with a single join returning clients and their active cycles together; add a repository method rather than looping in the service. Estimated 1–2 hours. |

---

### TD-02 · Tailwind loaded from CDN, unpurged `app/web/templates/base.html:10` · Infrastructure · Prudent & Deliberate

| | |
|---|---|
| **Debt** | The full Tailwind runtime is fetched from a CDN and compiles classes in the browser. No build step, no purge, no self-hosting. |
| **Cause** | Cut in the scope decision to recover 1.0 hour. |
| **Impact** | A render-blocking third-party request on every page load, working directly against NFR-01 (2 s on a 3G connection) and CO-07 (low-end phones on mobile data) — the exact users this system targets. Also a third-party availability dependency and a supply-chain surface. |
| **Priority** | **Scheduled.** |
| **Resolution** | Add a Tailwind build producing a purged stylesheet served from the application. Estimated 1–2 hours. |

---

### TD-12 · Dependencies are version ranges with no lockfile `requirements.txt:5` · Infrastructure · Prudent & Deliberate

| | |
|---|---|
| **Debt** | `requirements.txt` uses `>=` bounds. No lockfile, no hash pinning. |
| **Cause** | Ranges avoided install failures during a time-boxed build. |
| **Impact** | Builds are not reproducible: the deployed application may not match what was tested, and a transitive release can break production without any change to this repository. Also weakens the reproducibility the examiner would need to rebuild the submission. |
| **Priority** | **Scheduled.** |
| **Resolution** | Pin exact versions and commit a lockfile ( `pip-compile` or `uv lock` ). Estimated 30 minutes. |

---

**TD-10 · Cycle maturity is evaluated on read, not by a scheduler** `app/infrastructure/repositories.py` ( `list_due_for_payout` ) · Architecture/Design · Prudent & Deliberate

| | |
|---|---|
| **Debt** | A cycle becomes visible as matured when a supervisor opens the payouts screen, not at midnight on its end date. No background job exists. |
| **Cause** | Free-tier hosting provides no scheduler or worker process (CO-04). |
| **Impact** | If no supervisor looks, nothing happens: no notification, no automatic transition. A client whose cycle matured is not paid until someone opens a screen. Correctness is preserved — the query is date-driven — but timeliness depends on human attention. |
| **Priority** | **Scheduled.** |
| **Resolution** | Add a scheduled task marking cycles MATURED and notifying supervisors, once hosting supports a worker. Estimated 2 hours plus hosting cost. |

---

**TD-17 · No structured logging, error tracking or monitoring** Application-wide · Process · Prudent & Deliberate

| | |
|---|---|
| **Debt** | Default Flask logging to stdout. No structured logs, no error aggregation, no uptime monitoring, no alerting. |
| **Cause** | Not required to demonstrate the lifecycle, and out of the time budget. |
| **Impact** | A production failure is discovered by a user reporting it. Diagnosis depends on whatever the platform retained. Directly limits the *corrective maintenance* capability described in `11-maintenance-evolution.md` . |
| **Priority** | **Scheduled.** |
| **Resolution** | Structured JSON logging with a request id; error tracking (Sentry free tier); uptime check against a health endpoint. Estimated 2–3 hours. |

---

## Acceptable temporarily

---

**TD-07 · Hand-written mapping between domain entities and ORM models**
`app/infrastructure/repositories.py:6` · Architecture/Design · Prudent & Deliberate

| | |
|---|---|
| **Debt** | Seven `_to_*` functions convert records to entities by hand. A schema change must be made in two places. |
| **Cause** | The direct consequence of keeping the domain layer framework-free (NFR-07). SQLAlchemy imperative mapping onto the dataclasses would remove it but was not affordable. |
| **Impact** | Duplication, and silent drift if a column is added to a model but not its mapper. Partially mitigated: an integration test asserts money round-trips the mapping exactly. |
| **Priority** | **Acceptable.** This is the price of an architecture that pays for itself — it is what makes 129 unit tests run in 0.34 s with no database. Removing the mapping would remove that benefit. |
| **Resolution** | Revisit only if entity count grows substantially. Consider SQLAlchemy imperative mapping. Extend round-trip tests to every field in the meantime. |

---

**TD-08 · Sinkhole: simple reads pass through the service layer** `app/services/` · Architecture/Design · Prudent & Deliberate

| | |
|---|---|
| **Debt** | Session 5's named anti-pattern: reads such as listing clients traverse the service layer adding little transformation. |
| **Cause** | Accepted at design time ( `07-system-analysis-and-design.md` §7.6). |
| **Impact** | A few extra function calls; negligible at this scale. |
| **Priority** | **Acceptable.** The cost buys one consistent path for authorisation and audit. Bypassing the layer for reads would create two paths and invite an unauthorised read. |
| **Resolution** | None planned. This is a deliberate trade, documented so it is not mistaken for an oversight. |

---

**TD-11 · Cycle summaries recomputed on every render** `app/domain/rules.py:174` · Code · Prudent & Inadvertent

| | |
|---|---|
| **Debt** | `compute_cycle_summary` sums the full contribution list each time a card is displayed; no cached or denormalised total. |
| **Cause** | Recognised while designing the card view rather than beforehand — Fowler's "we now know how we should have done it". |
| **Impact** | O(n) where a running total would be O(1). Bounded at 31 rows per cycle, so it is not a present problem. |
| **Priority** | **Acceptable.** |
| **Resolution** | Denormalise a running total onto the cycle row if cycle length ever becomes configurable beyond a month. |

---

**TD-03 · Client list has no search, filter or pagination** · `app/web/collector.py` · Code · Prudent & Deliberate Cut for 0.5 h. FR-08 is therefore only **partially** satisfied and is reported as such in `09-testing.md`. Degrades past roughly 50 clients; QR scanning (FR-40) bypasses it for the common case. **Acceptable.** Resolution: server-side filter plus pagination, ~1 h.

**TD-04 · Reversal is a bare reference-and-reason form** · `app/web/supervisor.py:65` · Code · Prudent & Deliberate Cut for 0.6 h. The supervisor must copy a reference by hand rather than acting from the contribution row, which invites transcription error — though a mistyped reference fails safe, since no contribution matches. **Acceptable.** Resolution: reverse action on each row, reason codes, ~1 h.

**TD-05 · Route sheet is a flat list** · `app/services/collection.py:338` · Code · Prudent & Deliberate Cut for 0.5 h; unordered by geography and unfiltered by outstanding status. FR-23 met literally, collector efficiency reduced. **Partially mitigated by CR-001** — scanning bypasses the list for the common case, which is why the reduced version was acceptable. **Acceptable.** Resolution: filter to uncollected, order by route, ~1 h.

**TD-06 · HTMX applied to three interactions only** · `app/web/` · Architecture/Design · Prudent & Deliberate Cut for 0.8 h. Partial updates on the route sheet's collect action; full page loads elsewhere. Inconsistent interaction model. **Acceptable** — the three chosen sit on the collector's critical path, so the benefit is concentrated where it matters. Resolution: extend to remaining forms, ~1 h.

**TD-13 · Cycle length and commission policy are constants** · `app/domain/rules.py:41` · Code · Prudent & Deliberate FR-36 (configurable institutional defaults) was deferred as *Could*. Cycles snapshot their own length and rate at open, so making these configurable later cannot disturb existing cycles, and `CommissionPolicy` is already a Strategy interface. **Acceptable** — the design has been left open (Open/Closed) even though the feature is absent. Resolution: settings table plus admin UI, ~2 h.

---

## 8.5 Classification summary

### By urgency (as the examination requires)

| Classification | Count | Items |
|---|---|---|
| **Critical — immediate attention** | 3 | TD-09, TD-14, TD-15 |
| **Scheduled for future resolution** | 6 | TD-01, TD-02, TD-10, TD-12, TD-16, TD-17 |
| **Acceptable temporarily** | 8 | TD-03, TD-04, TD-05, TD-06, TD-07, TD-08, TD-11, TD-13 |
| **Total** | **17** | |

> **All three critical items are security items, and none of them was a deliberate scope cut.** TD-09 follows from a scope cut (no migrations), TD-14 from a platform constraint, TD-15 from an excluded requirement. The time-boxed trade-offs that *were* chosen deliberately all landed in the acceptable band. That is the reassuring reading. The uncomfortable one is that the most dangerous debt in a project is the debt nobody decided to take on.

### By Fowler's quadrant (Session 3)

| | Deliberate | Inadvertent |
|---|---|---|
| **Prudent** | **14 items** — TD-01…10, 12, 13, 14, 17. Taken knowingly, for a stated reason, with an intended repayment. | **3 items** — TD-11, TD-15, TD-16. "We now know how we should have done it." |
| **Reckless** | **0** | **0** |

No item is reckless. Every one has a recorded cause and a resolution, which is the distinction Session 3 draws between managed and unmanaged debt.

### By type (Session 3's six categories)

| Type | Count | Items |
|---|---|---|
| Code debt | 8 | TD-03, 04, 05, 11, 13, 14, 15, 16 |
| Architecture / design debt | 5 | TD-06, 07, 08, 09, 10 |
| Infrastructure debt | 3 | TD-01, 02, 12 |
| Process debt | 1 | TD-17 |
| **Test debt** | **0** | — |
| **Documentation debt** | **0** | — |

**Zero test debt and zero documentation debt is a deliberate result, not an oversight.** Session 3 identifies both as arising "when teams skip testing to meet deadlines" and "when teams focus only on coding" — the two most common casualties of a time-boxed build, and the two this project protected. The delivered system carries 209 tests at 97% coverage, and the documentation was written before the code rather than after it. Quality was reduced in styling, in convenience features and in operational tooling instead.

---

## 8.6 Interest analysis

Session 3 describes debt accruing *interest*: increased bug frequency, higher maintenance cost, reduced development speed. Estimating where interest is already accruing:

| Item | Interest being paid now | Rate |
|---|---|---|
| TD-01 no migrations | Blocks TD-09's fix; every future schema change costs manual DDL | **Compounding** — it gates other repayments |
| TD-14 no rate limiting | None yet; the entire cost arrives at once if exploited | **Step function** |
| TD-15 collector knows password | Accruing silently — every day the claim of client independence is weaker than stated | **Compounding** |
| TD-16 N+1 | Proportional to route size; invisible at demo scale | **Linear** |
| TD-02 CDN Tailwind | Paid on every page load by every user, today | **Constant, ongoing** |
| TD-07 mapping | Paid once per schema change | **Per-event** |
| TD-08, TD-11, TD-13 | Effectively zero at current scale | **Negligible** |

**TD-01 is sequenced first not because it is the most dangerous but because it compounds and gates.** TD-09 cannot be repaid without it.

## 8.7 Repayment sequence

| Order | Item | Estimate | Rationale |
|---|---|---|---|
| 1 | TD-01 migrations | 2 h | Prerequisite for TD-09; unblocks all future schema work |
| 2 | TD-14 rate limiting | 2–3 h | Highest severity; independent of everything else |
| 3 | TD-15 forced password change | 3–4 h | Restores the system's central guarantee |
| 4 | TD-09 audit log enforcement | 1–2 h | Needs TD-01 |
| 5 | TD-12 dependency pinning | 0.5 h | Cheap; makes the rest reproducible |
| 6 | TD-16 N+1 | 1–2 h | Before client counts grow |
| 7 | TD-02 Tailwind build | 1–2 h | Directly serves NFR-01 |
| 8 | TD-17 logging and monitoring | 2–3 h | Enables corrective maintenance |
| 9 | TD-03…06 convenience features | ~4 h | User-facing polish, no correctness risk |
| | **Total** | **~19–24 h** | Roughly one working week |

Items 1–4 (**8–11 hours**) must complete before the system handles real client money. This sequence is carried into the repayment plan in `11-maintenance-evolution.md`.

## 8.8 Debt deliberately *not* taken on

Recorded because refusing debt is as much a decision as accepting it, and these were live temptations under time pressure:

| Shortcut available | Would have saved | Why it was refused |
|---|---|---|
| Floats for money | ~0.5 h | BR-R1. A rounding error in a savings ledger is a correctness *and* trust failure, and it is close to unfixable once data exists. |
| Business rules on the ORM models | ~1 h | Would have required a database for every rule test, and those tests would then not have been written at all. |
| Skipping database-level invariants | ~1 h | The domain checks already pass; the indexes exist for the case where the domain layer is wrong. |
| Editing contributions in place instead of reversals | ~0.6 h | Would destroy the non-repudiation that answers a client-collector dispute — the reason the system exists. |
| Hiding actions in templates instead of enforcing authorisation server-side | ~1 h | Hiding a link is presentation, not a control. |
| Skipping the audit log | ~1 h | BR-05. Without it there is no evidence trail and the system solves nothing. |

Roughly five hours of additional shortcuts were available and refused. Each would have created debt in the one area — correctness of the client's money and the record of it — where this system cannot afford any.

# 12. Deployment

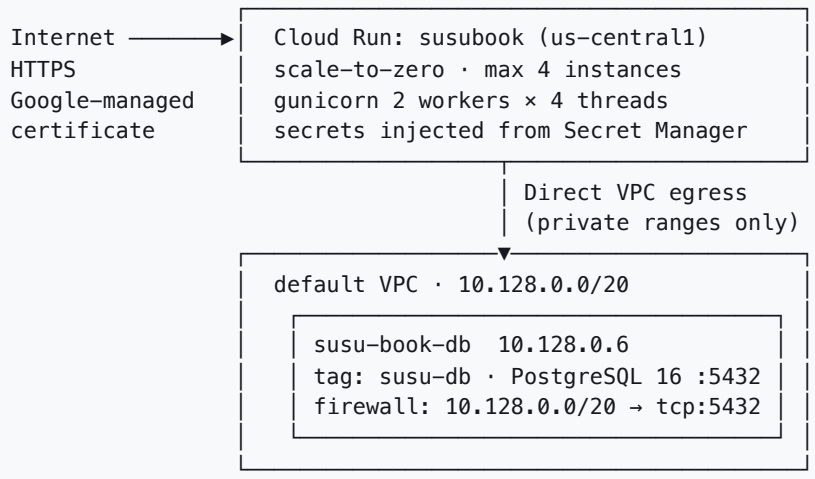
> Examination document §13. The live application, how it is deployed, and how it actually performs — measured, not asserted.

## 12.1 Live application

| | |
|---|---|
| **URL** | https://susubook-fdtbppd7sq-uc.a.run.app |
| **Platform** | Google Cloud Run (managed), region `us-central1` |
| **Database** | PostgreSQL 16 on a Compute Engine VM, private address `10.128.0.6` |
| **Project** | `groovy-bay-311316` |
| **TLS** | Google-managed certificate, HTTPS enforced |
| **Health** | `/health` → `{"database":"ok","status":"ok"}` |

Credentials for grading are in `Deployment_and_Source_Links.txt`.

## 12.2 Architecture




**The database has no public address.** It is reachable only from within the VPC, and the firewall admits only the subnet Cloud Run draws its addresses from. Nothing on the internet — including the developer's own machine — can open a connection to it.

## 12.3 Deployment decisions

**Cloud Run over a free PaaS.** Both scale to zero. The difference is what happens on the first request afterwards: a suspended free-tier PaaS instance can take around 50 seconds to wake, which an examiner would reasonably read as a dead link. Cloud Run's measured cold start here is **3.3 seconds** (§12.6). Under examination Rule 8 — the deployment must remain accessible for grading — that difference is the whole argument.

**Direct VPC egress, not a Serverless VPC Access connector.** A connector provisions instances that bill continuously, which would defeat a free-tier deployment. Direct egress is configuration on the service itself, at no standing cost.

**Region is determined by the subnet, not chosen freely.** `10.128.0.0/20` is the default-VPC range for `us-central1`, and the firewall admits that range only. Cloud Run must therefore deploy to the same region. Changing one means changing both.

**One image, two workloads.** The web service and the administrative jobs run the same container image, so a maintenance task cannot drift from the application it maintains.

**Administrative tasks run as Cloud Run Jobs.** Because the database is private, schema creation and seeding cannot be run from a developer machine. They execute as jobs inside the VPC, using the same image and the same secret — no SSH tunnel, no temporary public IP, and the password never leaves Secret Manager.

## 12.4 Secrets

Configuration is supplied through the environment, following the twelve-factor approach [25]. Only the database **password** is secret. Host, port, database name and role are ordinary configuration passed as environment variables, so Secret Manager holds exactly one value that needs protecting and everything else stays legible in the service definition where an operator can read it.

| Secret | Contents | Created |
|---|---|---|
| `susu-db-password` | Database password only | By hand, before first deploy |
| `susubook-secret-key` | Session signing key, 32 random bytes | Generated by `deploy.sh setup` |

The connection URL is composed at startup by `resolve_database_url()`, which percent-encodes the password so a `@`, `:`, `/`, `#` or `?` cannot corrupt the URL. No credential appears in the repository, in a configuration file, or in the deployment script.

## 12.5 Pipeline

```
./deploy/deploy.sh setup      # once: APIs, registry, signing key, secret IAM
./deploy/deploy.sh deploy     # Cloud Build → Artifact Registry → Cloud Run
./deploy/deploy.sh db-init    # schema, as a job inside the VPC
./deploy/deploy.sh seed       # demo data, as a job inside the VPC
./deploy/deploy.sh url
```

Images are tagged with the short git SHA, so every deployed revision is traceable to a commit. The live revision `susubook-00002-g86` runs image tag `4fb6382`.

Builds run in **Cloud Build** rather than pushing from the developer machine. Besides avoiding local registry credentials, this guarantees the image architecture matches Cloud Run: an arm64 Mac building locally would produce an image Cloud Run cannot start, and the failure appears only after deployment.

## 12.6 Measured performance

Median of seven requests from a residential connection to `us-central1`.

| Phase | `/login` | `/health` |
|---|---|---|
| DNS lookup | 0.004 s | 0.003 s |
| TCP connect | 0.162 s | 0.133 s |
| TLS handshake complete | 0.337 s | 0.313 s |
| **Time to first byte** | **0.618 s** | **0.611 s** |
| Total | 0.619 s | 0.612 s |
| Payload | 2,340 B | 32 B |

**Reading this honestly.** `/health` returns 32 bytes and `/login` returns 2,340 — a seventy-fold difference — yet their timings are within 8 ms of each other. The application is not the constraint. Roughly **0.31 s is consumed before the request is even sent**, on TCP and TLS handshakes; that is pure geography, and the same code measured 2–12 ms of server work when run locally (§9.6).

**Cold start: 3.3 seconds**, measured on the first request after the service had been idle following a deployment. Subsequent requests are ~0.6 s. Setting `--min-instances=1` would remove it entirely at roughly $5–15/month; the debt is not "serverless is slow" but "zero cost was chosen over zero latency, and this is the price of reversing that."

### NFR-01 is probably not met on a 3G connection

NFR-01 specifies a 2-second page render on a 3G-class connection. The measured figures above are from a **good fixed connection**, and they already consume 0.6 s before the browser begins fetching sub-resources. The page then makes a serial request to `cdn.tailwindcss.com`, measured here at **0.52 s** on the same connection (**TD-02**).

A realistic first render is therefore **over 1.1 s on a good connection**. On the 3G-class link NFR-01 actually specifies, with higher latency on every one of those round trips, the 2-second budget is unlikely to hold.

This is recorded as **not met** rather than left as an untested claim. Two contributing causes are already in the debt register and both are addressable: **TD-02** (self-host and purge the stylesheet, removing an entire third-party round trip) and the region choice below.

## 12.7 Region trade-off

`us-central1` sits roughly 150–200 ms from Ghana, and the measurements above show that latency dominating every request. The intended users — market traders in Accra on mobile data — are the ones who pay it.

The region was not chosen for them. It was chosen because Compute Engine's free tier covers `us-central1`, `us-west1` and `us-east1` and nothing closer. A region such as `europe-west1` would roughly halve the round trip, and costs money.

Recorded here as a deliberate trade-off with a named cause, not an oversight. It is the single change that would most improve real-world performance, and it is carried in the evolution plan.

## 12.8 An infrastructure defect worth recording

**DEF-08 — Google Front End intercepts `/healthz` on Cloud Run.**

The health endpoint was originally mounted at `/healthz`, the conventional Kubernetes-style path. In production it returned Google's own 404 page while working correctly in every local test.

The symptom points the wrong way: the route is registered, the container serves it, and the application appears broken. Three observations located the cause:

1. The **identical image**, pulled from Artifact Registry and run locally, returned `200 {"database":"ok","status":"ok"}`.
2. Requests to `/healthz` **never appeared in Cloud Run's request log at all**, while a request to `/nope` did — so the request was being answered upstream.
3. Of **eleven** candidate paths tested against the deployed service, `/healthz` was the **only** one intercepted. `/health`, `/livez`, `/readyz`, `/status`, `/ping`, `/up` and the rest all reached the application.

Resolved by mounting the endpoint at `/health`.

The general lesson is worth more than the fix: **this class of defect is invisible outside a real deployment.** No unit, integration or system test could have found it, because every one of them runs against the application rather than through the infrastructure in front of it. Comparing the same artefact across two environments is what isolated it.

## 12.9 Operational notes

| | |
|---|---|
| **Rollback** | `gcloud run services update-traffic susubook --to-revisions=REVISION=100`. Previous revisions are retained, so rollback is immediate and does not require a rebuild. |
| **Logs** | `./deploy/deploy.sh logs`, or Cloud Logging. Structured logging, error aggregation and alerting remain outstanding (**TD-17**). |
| **Schema changes** | Manual DDL, because there are no migrations (**TD-01**). This is the first debt scheduled for repayment. |
| **Scaling** | 0 to 4 instances, 80 concurrent requests each. Capped at 4 deliberately: several instances each holding a connection pool against one small database would exhaust its connection limit, and the failure mode is being locked out of the database. |
| **Availability for grading** | Cloud Run services do not expire and the VM does not pause. The service should remain reachable without intervention; `/health` confirms both the application and its database in one request. |

# SusuBook User Manual

**Version 1.0**

## Contents



## 1. What SusuBook is

SusuBook replaces the paper susu card.

With a paper card, the collector holds the card and marks it. If the card is lost, damaged, or the marks are disputed, the client has nothing to show for their savings.

With SusuBook, **the client and the collector see the same record, and neither can change what has already been recorded.** Every contribution shows the amount, the date, the time it was recorded, and which collector recorded it.

Money is still cash. The collector still visits, and the client still hands over the same amount as before. SusuBook records what happened.

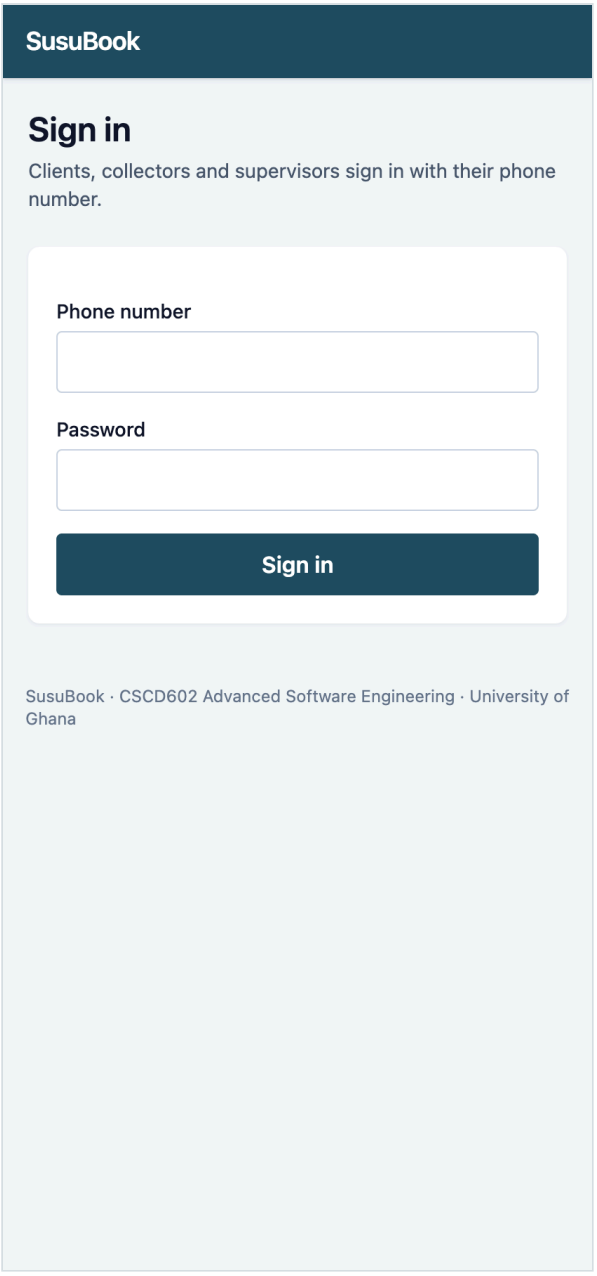
**Four kinds of user:**

| | |
|---|---|
| **Client** | Saves a fixed amount each day and can check their own record at any time |
| **Collector** | Visits clients, receives cash, records each contribution |
| **Supervisor** | Oversees collectors, checks the day's cash, approves payouts |
| **Administrator** | Manages user accounts |

## 2. Signing in

Everyone signs in the same way, on any phone or computer:

**https://ususubook-fdtbppd7sq-uc.a.run.app**




1. Enter your **phone number** — the one you gave when you registered
2. Enter your **password**
3. Tap **Sign in**

You will land on the right page for who you are. Clients see their own card; collectors see today's route; supervisors see the day's cash position.

**If it does not work:** the message "*Phone number or password is incorrect*" appears for both a wrong number and a wrong password. This is deliberate — it stops a stranger from discovering which phone numbers are registered. Check both, and if you still cannot sign in, ask your collector or branch to help.

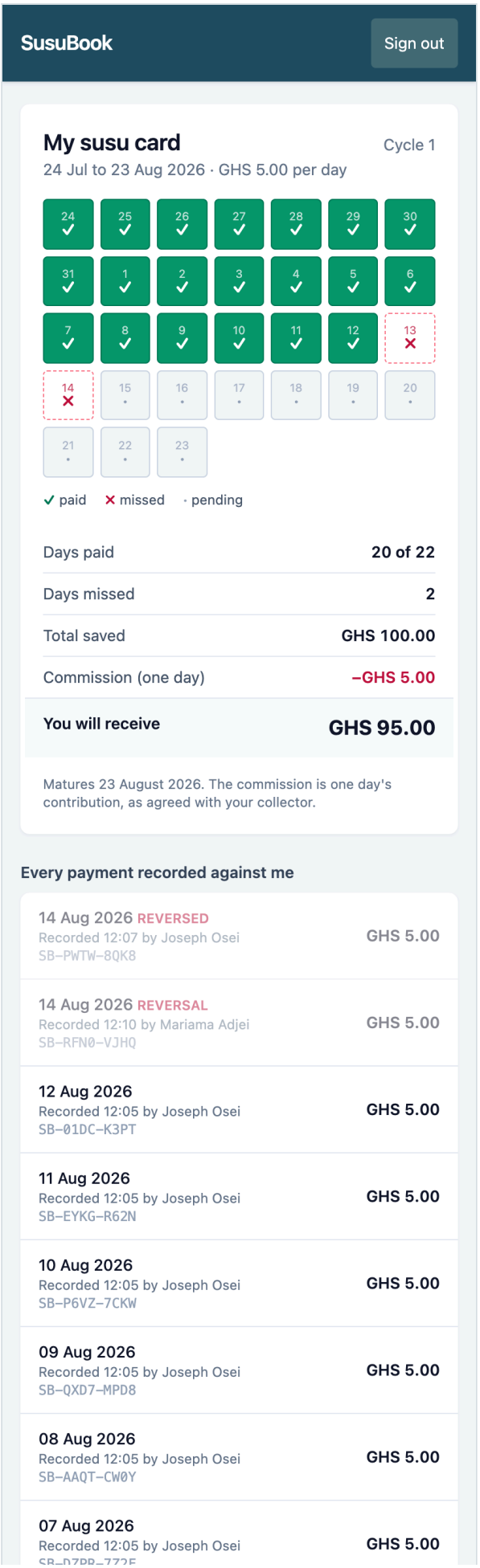
> **Clients: change your password.** Your collector set your first password, so they know it. Ask your branch to change it to something only you know. Until then, your record is not fully private to you.

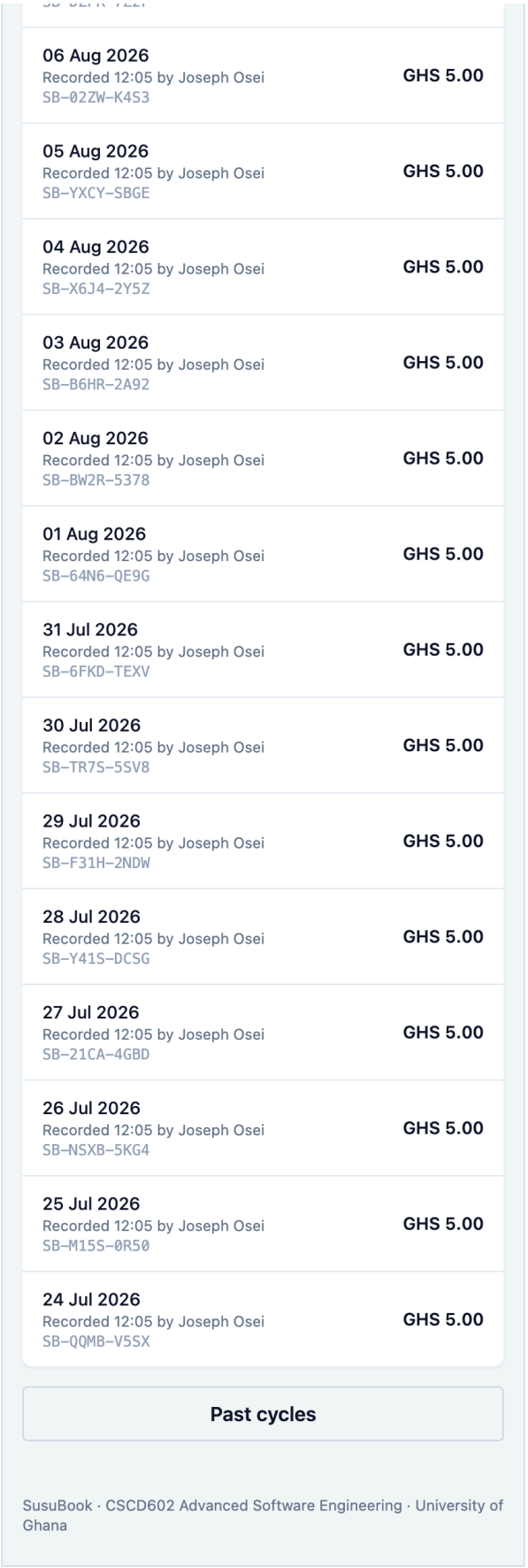
---

## 3. For clients — checking your savings

### Your susu card

After signing in, you see your card for the current cycle.


SusuBook
Sign out
My susu card
Cycle 1
24 Jul to 23 Aug 2026 · GHS 5.00 per day
24 25 26 27 28 29 30
31 1 2 3 4 5 6
7 8 9 10 11 12 13
14 15 16 17 18 19 20
21 22 23
✓ paid ✕ missed · pending
Days paid 20 of 22
Days missed 2
Total saved GHS 100.00
Commission (one day) –GHS 5.00
You will receive GHS 95.00
Matures 23 August 2026. The commission is one day's contribution, as agreed with your collector.
Every payment recorded against me
14 Aug 2026 REVERSED
Recorded 12:07 by Joseph Osei
SB–PWTW–8QK8
GHS 5.00
14 Aug 2026 REVERSAL
Recorded 12:10 by Mariama Adjei
SB–RFN0–VJHQ
GHS 5.00
12 Aug 2026
Recorded 12:05 by Joseph Osei
SB–01DC–K3PT
GHS 5.00
11 Aug 2026
Recorded 12:05 by Joseph Osei
SB–EYKG–R62N
GHS 5.00
10 Aug 2026
Recorded 12:05 by Joseph Osei
SB–P6VZ–7CKW
GHS 5.00
09 Aug 2026
Recorded 12:05 by Joseph Osei
SB–QXD7–MPD8
GHS 5.00
08 Aug 2026
Recorded 12:05 by Joseph Osei
SB–AAQT–CW0Y
GHS 5.00
07 Aug 2026
Recorded 12:05 by Joseph Osei
GHS 5.00




**The grid** shows every day of your 31-day cycle:

| Mark | Meaning |
|---|---|
| ✓ green | You paid on that day |
| ✗ red, dashed | That day has passed and no payment was recorded |
| · grey | That day has not arrived yet |

The marks use both a **symbol and a colour**, so the card can be read in bright sunlight and by anyone who has difficulty telling colours apart.

**Below the grid** you see:

- **Days paid** — how many days you have paid, out of the days so far
- **Total saved** — everything recorded for you this cycle
- **Commission** — one day's contribution, kept by the collector. This is the normal susu arrangement, agreed when you registered
- **You will receive** — what you get at the end of the cycle
- **Matures** — the date your cycle ends

### Every payment recorded against you

Below the card is the list of your payments. Each one shows:

- the **date**
- the **amount**
- the **time** it was recorded
- **which collector** recorded it
- a **reference** such as `SB-4K2M-7X9P`

**This is the important part.** If you ever disagree with your collector about a payment, this list is your evidence. It is written when the collector records the payment, and it cannot be quietly altered afterwards.

### If a payment is corrected

Sometimes a collector makes an honest mistake — the wrong client, or the wrong day. Corrections do not delete anything. You will see **two** entries, marked `REVERSED` and `REVERSAL`, both greyed out. The original stays visible so you can see exactly what happened and when.

### Past cycles

Tap **Past cycles** to see your earlier cycles and what you received from each.

### What to check

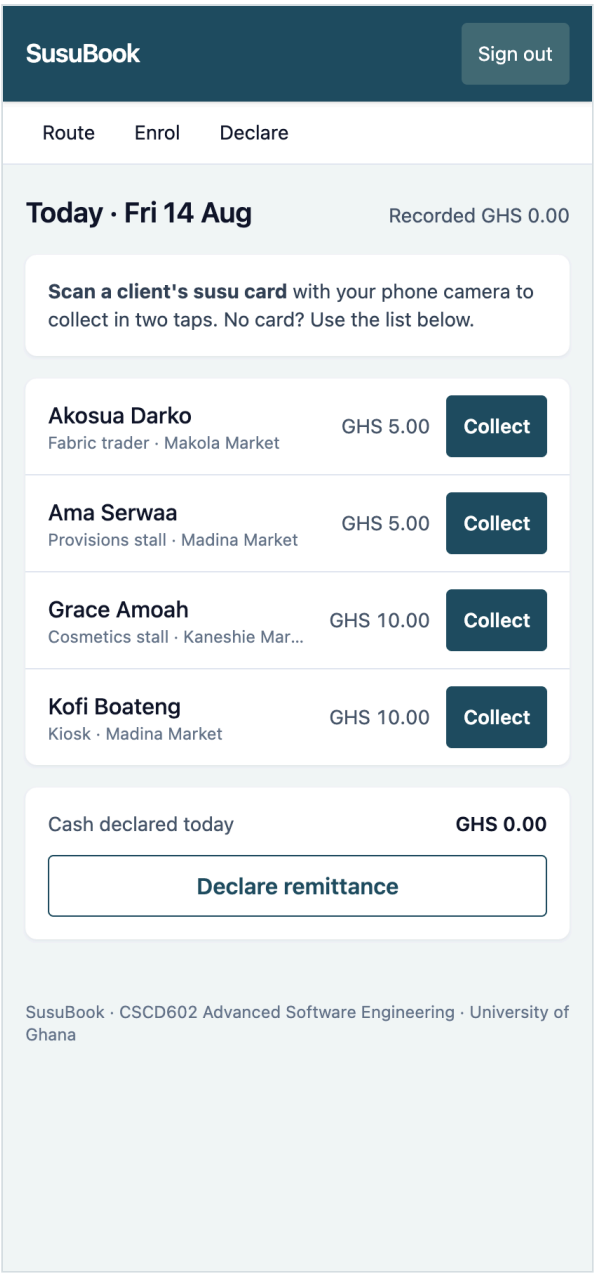
- After your collector visits, open your card and confirm the payment appears
- If it does not appear within a day, tell your collector or the branch
- Before your cycle matures, check **days paid** and **you will receive**

---

## 4. For collectors — daily collection

### Today's route

Signing in takes you to today's route.



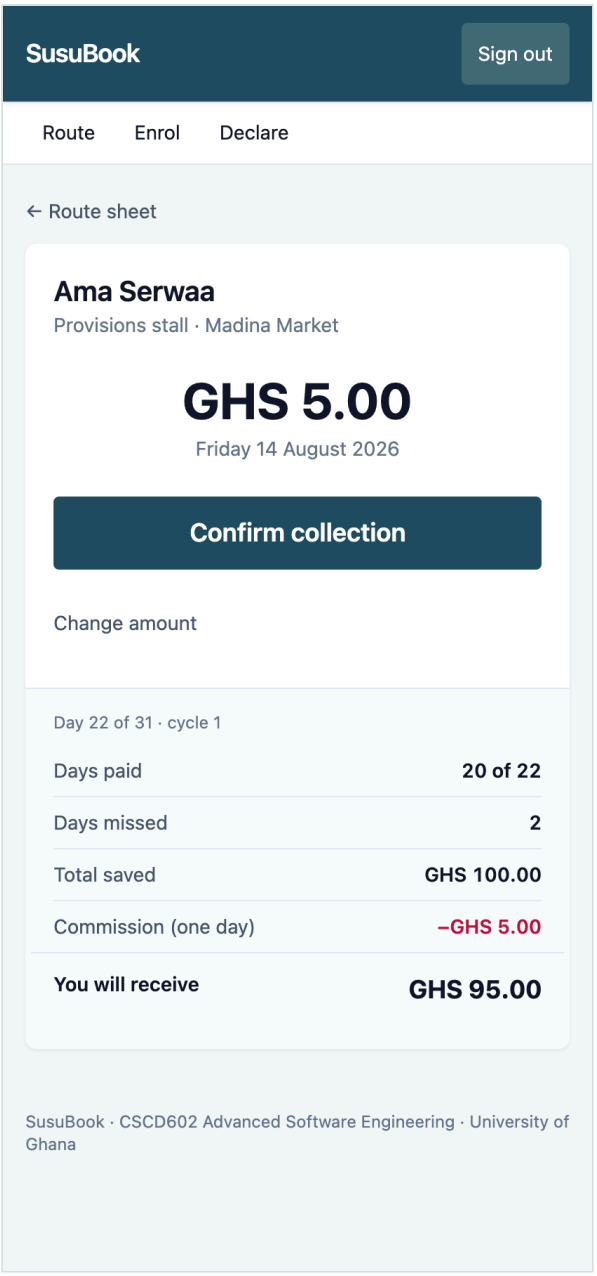

Each client shows their name, business, location and agreed daily amount. Clients you have already collected from today show ✓ **Paid**. The figure at the top right is **your running total for today** — it updates as you collect.

## Recording a collection — two ways

### By scanning the client's card (fastest)

1. Open your phone's **normal camera**
2. Point it at the QR code on the client's susu card
3. Tap the link that appears
4. Tap **Confirm collection**

Two taps. There is no special scanner to open — your phone's own camera reads the code.




The amount is already filled in with the client's agreed daily rate, along with their name and today's date. Check the name matches the person in front of you, then confirm.

**From the route sheet**

If the client has lost their card or left it at home, find their name on the route sheet and tap **Collect**. The result is identical.

## Collecting a different amount

Tap **Change amount** on the confirmation screen. The amount must be a **whole multiple of the agreed daily rate** — so for a client saving GHS 10.00 per day you may enter 10.00, 20.00 or 30.00, but not 7.50. This prevents typing errors from becoming wrong balances.

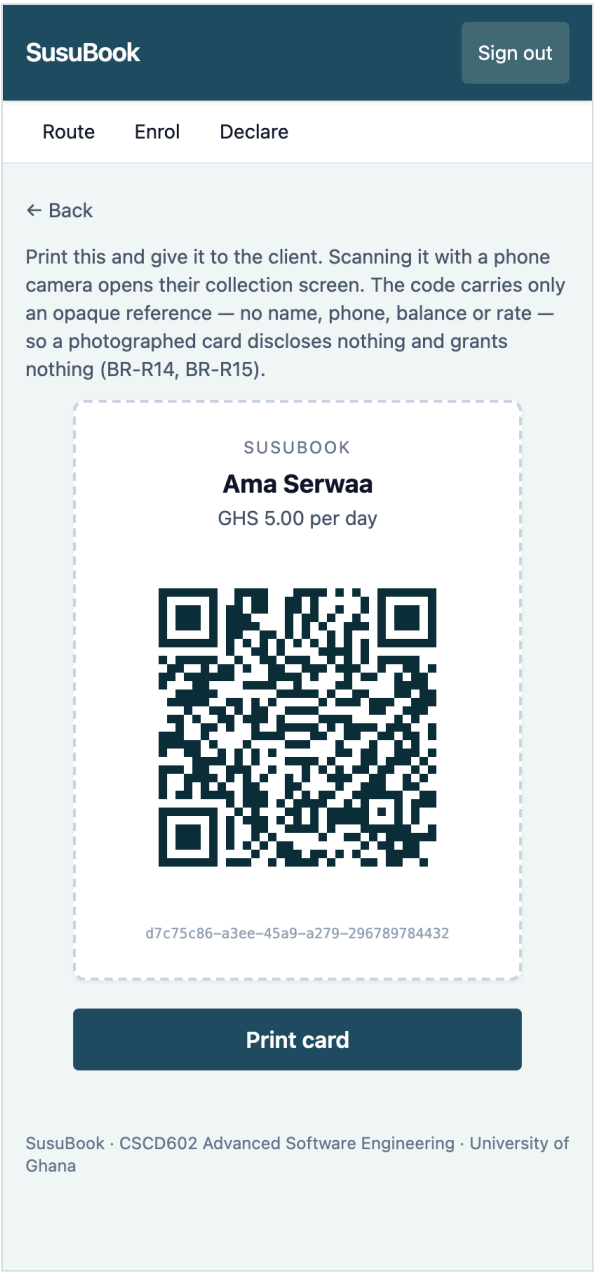
## Enrolling a new client

Tap **Enrol** and fill in the client's name, phone number, agreed daily amount, business and location, then set an initial password for them.

This creates the client's own login and opens their first 31-day cycle in one step. Give them the password and **print their susu card** on the next screen.

Only these details are stored — no Ghana Card number, no address. The system keeps the minimum it needs.

### Printing a susu card




After enrolling, or from any client's page, tap **QR card** then **Print card**. Give the printed card to the client to keep, like the paper card it replaces.

**The card is safe to carry.** The code contains only a reference — no name, no phone number, no balance. If it is lost or photographed by a stranger, they cannot use it to collect anything: only *you*, signed in, and only for clients on *your* route.

### Declaring your cash at the end of the day

When you bank the day's cash, record what you handed over:

1. Tap **Declare**
2. Enter the **total cash you banked**
3. Tap **Declare**

The system compares this with what you recorded in the field. If they match, you see "*Remittance declared and reconciled.*" If they differ, the difference is recorded for your supervisor to look at.

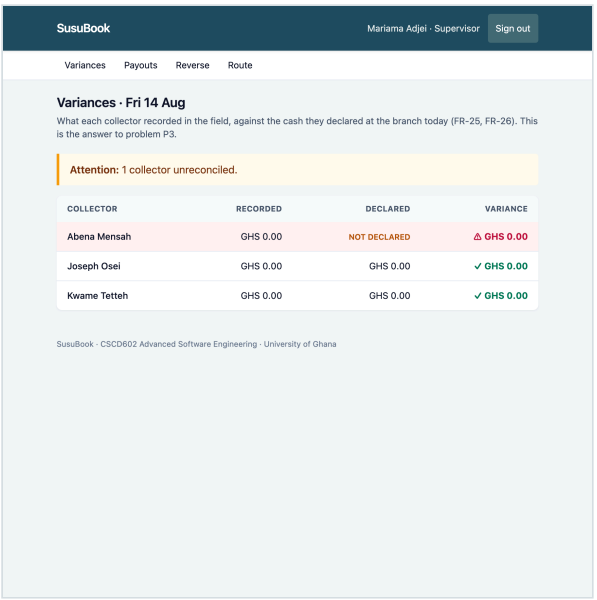
**Declare every day, even if you collected nothing.** A collector who never declares appears on the supervisor's screen as **NOT DECLARED**, which looks worse than a declared zero.

---

## 5. For supervisors — oversight and payouts

### The day's cash position

Signing in takes you to today's variances.




For each collector:

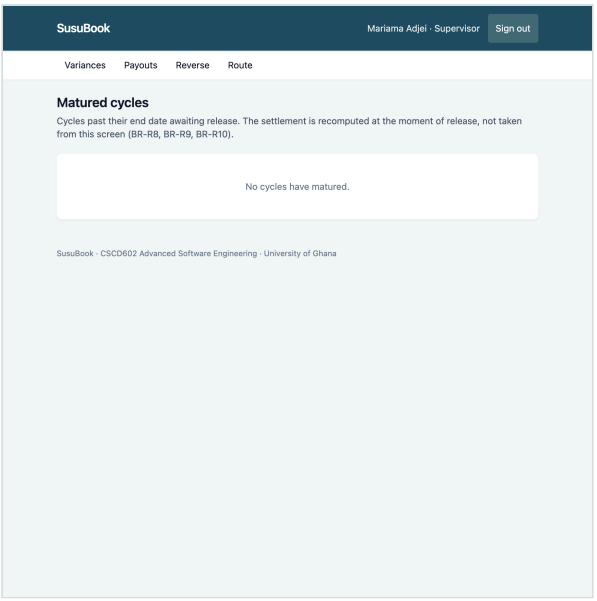
| Column | Meaning |
|---|---|
| **Recorded** | Total the collector recorded in the field today |
| **Declared** | Cash the collector says they banked |
| **Variance** | The difference |

A **✓ GHS 0.00** in green means the two agree. A **⚠ red figure** means they do not, and the row is highlighted. **NOT DECLARED** means the collector has not yet reported their cash.

A positive variance means the collector recorded more than they banked — the case that matters most. Act on it the same day, while the money is still recoverable.

Collectors who recorded nothing still appear, with zeroes. This is deliberate: a collector who has stopped working should be visible, not absent.

### Releasing a matured payout




Tap **Payouts** to see cycles past their end date. Each shows the client, days paid, total collected, commission and net payout.

1. Check the figures against the client's card if you wish
2. Tap **Release payout & open next cycle**
3. Confirm

The client's cycle closes, and a new one opens immediately at the same daily rate, so they can keep saving without re-registering.

**A payout can only be released once.** A second attempt is refused.

> **If the net payout is GHS 0.00**, the client contributed only one day. That single contribution *is* the commission, so there is nothing left to pay out. The screen explains this before you confirm. It is correct, not an error.

### Correcting a mistaken contribution

1. Tap **Reverse**
2. Enter the **contribution reference** (for example `SB-4K2M-7X9P`)
3. Enter the **reason** — this is required and is stored permanently
4. Tap **Reverse contribution**

Nothing is deleted. The original entry stays on the client's record alongside the reversal, and the client can see both. The day then becomes free, so a corrected contribution can be recorded for it.

Only supervisors can reverse. Collectors cannot.

---

## 6. For administrators

Administrators can see everything supervisors can.

User account management is **not available in this version** — accounts are created by a collector at enrolment (clients) or set up directly by the branch (collectors and supervisors). A management screen is planned for a future release.

---

## 7. Messages you may see

| Message | What it means | What to do |
|---|---|---|
| *Phone number or password is incorrect* | Either the number or the password is wrong. The message is the same for both, deliberately. | Check both carefully |
| *Already collected today* | A contribution for this client and today already exists | Check the client's card; if it is genuinely wrong, ask a supervisor to reverse it |
| *A contribution for … was already recorded (reference SB-…)* | The same, with the existing reference | Quote that reference to your supervisor |
| *Expected GHS X … Contributions must be a whole multiple of the agreed daily rate* | The amount is not a multiple of the client's daily rate | Enter the daily amount, or a whole multiple of it |
| *'…' is not a valid amount in cedis* | The amount could not be read | Enter it as digits, e.g. `10.00` |
| *This client is not on your route* | The client belongs to a different collector | Ask your supervisor to reassign them |
| *This cycle is matured / paid out and no longer accepts contributions* | The cycle has ended | A supervisor must release the payout; a new cycle then opens |
| *Cannot record a contribution dated … that date has not yet arrived* | A future date was entered | Use today's date |
| *That contribution has already been reversed* | Someone reversed it already | Check the client's record |
| *Cash declared cannot be negative* | A negative amount was entered | Enter zero or more |
| *Cannot declare a remittance for a future date* | The date is in the future | Use today's date |
| *The agreed daily contribution must be more than zero* | The daily rate was left blank or zero | Enter the amount agreed with the client |

---

## 8. Common questions

**Can my collector change what I have already paid?** No. Records cannot be edited or deleted. A supervisor can add a *correction*, and you will see both the original and the correction on your card, with the reason.

**What if I lose my susu card?** Ask your collector — they can collect from you using the route sheet instead, and print you a new card. Nobody who finds your card can use it.

**Why is one day's contribution deducted?** That is the collector's commission and the normal susu arrangement, agreed when you registered. It is shown on your card as **Commission** so you can see exactly what is being deducted.

**I missed some days. Is that a problem?** No. Missed days show as ✗ and you simply save less that cycle. You are paid what you actually contributed, less the one-day commission.

**Can I pay for several missed days at once?** Not in this version. Each day is recorded separately, on the day. This is planned for a future release.

**Will I be told when a payment is recorded?** Not by SMS in this version — you check your card. SMS notification is the first feature planned for the next release.

**What happens when my cycle ends?** Your supervisor releases the payout and a new cycle opens automatically at the same daily amount. You do not need to register again.

**The page is slow to load the first time.** The system sleeps when nobody is using it and takes a few seconds to wake. After that it is fast. This keeps running costs at zero.

---

## 9. Words used in this manual

| Word | Meaning |
|---|---|
| **Cycle** | A savings period of 31 days, ending in a payout |
| **Daily rate** | The fixed amount agreed when a client registers |
| **Contribution** | One payment, recorded on one day |
| **Commission** | One day's contribution, kept by the collector |
| **Payout** | What the client receives at the end: total saved less commission |
| **Reference** | A code such as `SB-4K2M-7X9P` identifying one payment |
| **Susu card** | The printed card carrying the client's QR code |
| **Route sheet** | A collector's list of clients for the day |
| **Remittance** | Cash a collector banks at the branch |
| **Variance** | The difference between what was recorded and what was banked |
| **Reversal** | A correction that cancels an earlier contribution without deleting it |
| **Maturity** | The date a cycle ends and payout becomes due |

---

## Getting help

Contact your branch or supervisor. When reporting a problem with a specific payment, quote its **reference** — that identifies it exactly.

# 11. Maintenance Strategy and Future Evolution

> Examination document §15 and §16, including the technical debt repayment plan the paper requires.

# PART A — MAINTENANCE STRATEGY

## 11.1 The four maintenance categories, applied

Generic definitions are of little use; what follows is what each category actually means for *this* system, with instances already identified.

### Corrective — fixing defects

| | |
|---|---|
| **Trigger** | Defect reported by a user, or surfaced by the audit log |
| **Current capability** | **Weak.** No error aggregation and no alerting (**TD-17**), so a production failure is discovered when a user reports it |
| **Evidence** | Seven of the eight defects found so far ( `09-testing.md` §9.8) came from tests or probing; **DEF-06 came from a person clicking through**, and no automated test could have caught it |
| **Improvement** | TD-17's repayment — structured logging, error tracking, uptime checks — moves detection ahead of the user report |

### Adaptive — responding to environmental change

The environment this system sits in is unusually active:

| Change | Consequence |
|---|---|
| Data Protection Act amendments | Stored fields, retention, consent at enrolment (NFR-06) |
| Bank of Ghana microfinance directives | Audit and reporting obligations |
| Mobile money displacing cash collection | The system records cash; large-scale MoMo adoption changes the domain, not just the code |
| Python, Flask, PostgreSQL releases | Security patches, deprecations |
| Android browser changes | QR scanning depends on the native camera app's behaviour (FR-40) |

The last is a real dependency worth naming: **CR-001 deliberately delegated QR decoding to the operating system.** That bought simplicity and reliability, and it accepted that a change in Android's camera behaviour becomes our problem.

### Perfective — improving what already works

Driven by the debt register and by measurement, not by taste:

- **TD-02** — self-host and purge the stylesheet. Measured at 0.52 s of serial round trip today (§12.6), and the largest single contributor to NFR-01 failing.
- **TD-16** — the N+1 on the route sheet, before client counts make it visible.
- **TD-03, TD-04, TD-05, TD-06** — the convenience features cut under time pressure. FR-08 and FR-23 remain *partially* satisfied until these land.

### Preventive — reducing future cost

- **TD-01** — migrations. Every further schema change costs manual DDL until this is done, and it blocks TD-09.
- **TD-12** — pin dependencies and commit a lockfile, so a transitive release cannot break production without a change in this repository.
- Keep test coverage at its current level (97%) as features are added. Session 3 names test debt as the classic casualty of delivery pressure; this project has none, and that is a position to defend rather than assume.

## 11.2 Defect triage

| Severity | Definition | Response | Example from this project |
|---|---|---|---|
| **S1 Critical** | Client funds misrecorded, or the audit trail compromised | Immediate; roll back if needed | A negative payout (guarded by BR-R9 and `ck_payout_balances`) |
| **S2 High** | A role cannot complete their core task | Same day | DEF-03, DEF-04 — every page 500ing |
| **S3 Medium** | Feature impaired, workaround exists | Next release | DEF-06 — the stale running total |
| **S4 Low** | Cosmetic or documentation | Scheduled | DEF-01 — an incorrect claim in a docstring |

Rollback is a first-class response: Cloud Run retains previous revisions, so reverting is one command and needs no rebuild (§12.9).

## 11.3 Security and dependency maintenance

| Activity | Cadence | Notes |
|---|---|---|
| Dependency vulnerability scan | Monthly, and on every release | Not yet automated; **TD-12** leaves versions unpinned |
| Framework security releases | As published | Flask, SQLAlchemy, psycopg, gunicorn |
| Base image rebuild | Monthly | `python:3.12-slim` accumulates OS-level CVEs even when the application does not change |
| Secret rotation | Annually, and on any suspected exposure | Both live in Secret Manager; rotation is a new version plus a redeploy |
| Audit log review | Weekly | `AUTHORISATION_DENIED` and `LOGIN_FAILED` are the signals that matter — and with **TD-14** unfixed, the log is the *only* defence against brute force |

**The three Critical debt items are security items, and they are maintenance work, not features.** TD-14 (no rate limiting), TD-15 (the collector knows the client's password) and TD-09 (audit log mutable) must be closed before the system holds real client money.

---

# PART B — FUTURE EVOLUTION

## 11.4 Lehman's Laws applied to SusuBook

Session 4's laws [4], formulated by Lehman [19] and developed with Belady [14], describe how systems behave over time. Each is taken in turn, with what it predicts *for this system* and what has been done — or must be done — in response.

### 1. Continuing change

> *A program used in a real-world environment must change, or become progressively less useful in that environment.*

Susu is not a static practice. Mobile money is displacing cash collection, regulation is tightening, and a collector who finds SusuBook slower than a paper card will return to the card — the fallback is always available and costs nothing.

**Response.** The deferred requirements are not a wish list, they are the change pipeline: FR-31 (SMS), FR-15 (catch-up payments), FR-22 (early withdrawal). The architecture was left open where change is most likely — `CommissionPolicy` is a Strategy interface precisely because commission policy is the term most likely to be renegotiated (FR-36).

### 2. Increasing complexity

> *As a program evolves, its structure becomes more complex unless work is done explicitly to reduce it.*

Fifteen business rules, four layers, seventeen debt items — at version one. Every deferred requirement adds branching to the same domain: catch-up payments complicate contribution allocation, early withdrawal complicates the cycle state machine, multi-institution complicates every query.

**Response — and this law is the reason for several earlier decisions.** The domain layer's isolation from Flask and SQLAlchemy, the 226 tests, and the business rules expressed as pure functions are precisely the "work done explicitly to reduce complexity" the law demands. They are not architectural taste; they are the mechanism by which the second law is resisted.

The countermeasure must continue: a standing refactoring budget, and the debt register kept current rather than allowed to become archaeology.

### 3. Self-regulation

> *Evolution is self-regulating; system attributes such as size and time between releases are approximately invariant across releases.*

**This law cannot yet be demonstrated here, and saying otherwise would be dishonest.** It describes behaviour across many releases; SusuBook has one.

What exists is the beginning of the baseline the law depends on: the estimation record in `05-effort-estimation.md` §4.8, with estimates against actuals and an MRE. Session 6's closing advice — record actuals, review accuracy — is exactly how a team acquires the historical data that makes this law predictive rather than merely descriptive.

### 4. Conservation of organisational stability

> *The average rate of development is approximately constant and independent of the resources devoted to it.*

The debt repayment plan totals 19–24 hours (§11.6). The temptation, facing a deadline, is to assume two developers would halve it.

**Response.** The law says otherwise — as does Brooks [13] — and the plan is not built on that assumption. Items 1–4 are sequenced by dependency — TD-01 gates TD-09 — not by what could be parallelised. Adding people to a system with one person's worth of context in it would, in the short term, slow it down.

### 5. Conservation of familiarity

> *The incremental change in each release is approximately constant.*

Ten deferred requirements and seventeen debt items constitute a large backlog. The law warns that discharging it in one release exceeds what users and maintainers can absorb.

**Response.** Release in increments of comparable size: security debt first, then one feature area at a time. A release containing SMS notification, catch-up payments, early withdrawal *and* multi-institution support would be unreviewable and unlearnable — and collectors, who are the least able to absorb disruption, would bear the cost.

### 6. Continuing growth

> *Functional content must continually increase to maintain user satisfaction.*

Every stakeholder group already wants more than v1 delivers: clients want SMS confirmation (FR-31), collectors want catch-up payments (FR-15), supervisors want a dashboard (FR-37) and variance resolution (FR-27), administrators want user management (FR-35).

**Response.** MoSCoW prioritisation gave a defensible v1; the same discipline must govern growth. Note the tension with the second law — growth adds functionality, which adds complexity — which is why perfective work must be funded alongside features rather than after them.

### 7. Declining quality

> *Quality will appear to decline unless the system is adapted to changes in its operational environment.*

**This law is already operating on SusuBook, with no code change required.** NFR-01 specifies 2 seconds on a 3G connection. Measurement (§12.6) shows first render already exceeding 1.1 s on a *good* connection, so the requirement is probably not met today. As client counts grow, TD-16's N+1 degrades the route sheet further. Nothing will have changed in the code; the environment will have moved.

**Response.** Quality attributes need measurement over time, not a one-off verification at release. The performance figures in §12.6 are a baseline to be re-measured, and NFR-01 is recorded as *not met* rather than quietly dropped.

### 8. Feedback system

> *Evolution processes are multi-loop, multi-agent feedback systems and must be treated as such.*

SusuBook contains three feedback loops by design:

| Loop | Mechanism |
|---|---|
| Collector accountability | Recorded collections → daily variance → supervisor action |
| Client verification | Contribution → immediately visible to the client → dispute raised early |
| Institutional oversight | Audit log → review → policy change |

And the development process has its own, with evidence from this project: **DEF-06 was found by a user clicking through the interface, not by any of the 226 automated tests — and could not have been, because every test asserted the *response* to a request and none asserted the state of the page afterwards.**

That is Lehman's eighth law demonstrated inside the project rather than quoted at it. Automated tests confirm what the developer thought to check; only the user loop finds what they did not. It is also the clearest argument for closing the outstanding UAT (§9.9).

## 11.5 Evolution roadmap

| Release | Contents | Rationale |
|---|---|---|
| **v1.1 — Security** | TD-01 migrations, TD-14 rate limiting, TD-15 forced password change, TD-09 audit enforcement | Everything blocking real client money. Nothing else ships first. |
| **v1.2 — Reach** | FR-31 SMS notification | Mitigates assumption **A5** — that clients can reach a web page — on which the entire value proposition rests. Highest user value of any single item. |
| **v1.3 — Field efficiency** | FR-15 catch-up payments, TD-05 route ordering, TD-03 search, TD-16 N+1 | The collector's daily experience. Domain rules already accept `days_covered`, so FR-15 needs no change to the business rules. |
| **v1.4 — Oversight** | FR-34 audit viewer, FR-37 dashboard, FR-27 variance resolution, FR-35 user admin | Supervisor and administrator capability |
| **v1.5 — Performance** | TD-02 stylesheet build, region relocation, TD-11 denormalised totals | Directly targets NFR-01, which is currently not met |
| **v2.0 — Scale** | Multi-institution tenancy, mobile money integration, offline capability | Each changes the data model or the product's scope, not merely its features |

## 11.6 Technical debt repayment plan

> Required explicitly by the examination. Full analysis in `08-technical-debt.md`.

| Order | Item | Est. | Why here |
|---|---|---|---|
| 1 | **TD-01** migrations | 2 h | Compounds and gates: TD-09 cannot be repaid without it |
| 2 | **TD-14** rate limiting | 2–3 h | Highest severity; independent of everything else |
| 3 | **TD-15** forced password change | 3–4 h | Restores the client independence the system exists to provide |
| 4 | **TD-09** audit log enforcement | 1–2 h | Requires TD-01 |
| 5 | **TD-12** pin dependencies | 0.5 h | Cheap; makes everything after it reproducible |
| 6 | **TD-16** N+1 queries | 1–2 h | Before client counts make it visible |
| 7 | **TD-02** stylesheet build | 1–2 h | Largest measured contributor to NFR-01 failing |
| 8 | **TD-17** logging and monitoring | 2–3 h | Enables corrective maintenance at all |
| 9 | **TD-03…06** convenience features | ~4 h | User-facing polish, no correctness risk |
| | **Total** | **19–24 h** | |

**Items 1–4 — 8 to 11 hours — must complete before the system handles real client money.** They are not improvements; they are the difference between a system that demonstrates the idea and one that can be trusted with savings.

Items TD-07, TD-08, TD-10, TD-11 and TD-13 are classified *acceptable temporarily* and carry no scheduled repayment. Each is documented with the condition that would change that assessment — TD-11, for instance, becomes worth fixing only if cycle length ever exceeds a month.

## 11.7 Evolution risks

| Risk | Law | Mitigation |
|---|---|---|
| Features ship, debt does not | 2, 7 | Repayment plan sequenced ahead of features in v1.1 |
| Backlog discharged in one release | 5 | Roadmap deliberately incremental |
| Quality erodes without anyone noticing | 7 | Re-measure §12.6 baseline each release; NFR-01 already recorded as not met |
| Maintainer changes, context lost | 2 | SATD markers name their register entry; every rule traces to the SRS |
| Feedback loops not closed | 8 | UAT completed and repeated; audit log reviewed weekly |
| MoMo displaces cash collection | 1 | Monitored as a domain shift, not a feature request |

# 14. Limitations, Conclusion and References

> Examination document §17, §18 and §19.

# 17. Limitations

Stated as limitations of the delivered system and of the process that produced it. Anything already reported as satisfied elsewhere is not repeated here; anything below is a genuine shortfall.

## 17.1 Requirements and validation

**No primary stakeholder elicitation was conducted.** No collector, client, supervisor or branch officer was interviewed. Requirements were derived from analysis of a well-documented manual process, and every point where a real stakeholder would have been consulted is recorded as a numbered assumption (§2.5). This is the most significant limitation of the requirements work, and it was declared at the time rather than discovered afterwards.

**Assumption A5 is unvalidated and the value proposition rests on it.** A5 holds that clients can reach a mobile web page. If false, the client-transparency feature — the reason the system exists — does not reach the people it is for. The mitigation (SMS notification, FR-31) is excluded from this release by the same constraints that make A5 risky: no paid gateway.

**User acceptance testing was not carried out.** Nine UAT scenarios are prepared (§9.9) and none has been executed with an independent participant. The 55 system tests exercise complete journeys but were written by the developer, and presenting them as UAT would misrepresent what they establish.

## 17.2 Verification gaps

| Requirement | Status | Why |
|---|---|---|
| **NFR-01** performance | **Not met** | Measured first render exceeds 1.1 s on a *good* connection against a 2 s budget specified for 3G (§12.6) |
| **NFR-02** ≤3 interactions | **Unverified** | Design achieves two by inspection; never measured with a participant |
| **NFR-08** accessibility | **Partial** | Shape-and-text encoding and 44 px targets implemented; no contrast audit, no screen-reader test, no outdoor-light trial |
| **FR-08** client list | **Partial** | List renders; search and pagination not implemented (TD-03) |
| **FR-23** route sheet | **Partial** | Renders with status; not filtered or route-ordered (TD-05) |

**No testing on real devices or real networks.** All measurement was from a desktop browser on a fixed connection. The target user is on a low-end Android handset over mobile data, and that configuration was never tested.

**No one has printed a QR card and scanned it with a phone.** TC-QR verifies the card renders and the route resolves and authorises correctly. The optical read off paper — the actual interaction — is untested.

**No load, concurrency, penetration or dependency-vulnerability testing.**

## 17.3 Security

Three debt items are classified **Critical** and remain open (§8.4):

- **TD-14** — no login rate limiting or lockout. Failed attempts are audited, so an attack is visible afterwards; nothing prevents one.
- **TD-15** — the collector sets the client's initial password and there is no forced change, so a collector can sign in as their client. This weakens the independence the system is built to provide.
- **TD-09** — the audit log is append-only by application convention, not by database permission.

**The system is not fit to hold real client money until these are closed.** That is a statement about this release, not a hypothetical.

## 17.4 Scope

Deliberately excluded, with reasons in §6.5: mobile money and bank integration, offline operation, multi-institution tenancy, native mobile applications. Ten specified requirements were deferred (§6.4).

Scope was reduced against the estimate before implementation began, and six areas were knowingly delivered below standard (TD-01…TD-06). The system demonstrates the lifecycle; it is not a production deployment.

## 17.5 Deployment

**Region.** `us-central1` sits roughly 150–200 ms from Ghana, and measurement shows that latency dominating every request (§12.6). It was chosen for Compute Engine free-tier eligibility, not for the users.

**Cold start.** 3.3 s on the first request after idle.

**No migrations** (TD-01), so schema changes require manual DDL. **No structured logging or alerting** (TD-17), so a production failure is discovered when a user reports it.

## 17.6 Process

**Effort was not instrumented.** Actuals are reconstructed from commit timestamps rather than recorded as work occurred, so per-task figures could not be produced and the effort estimate could not be validated against measured effort (§4.9). The phase-level distribution finding survives; the magnitude comparison does not.

**Single developer, single estimator.** No Delphi convergence, no independent function point recount, no code review by a second person. Constraints ES-01 to ES-04 record the consequences.

### 17.7 Documentation

**Referencing was retrofitted.** Sources are cited in §19 below, but they were assembled at the end rather than recorded as the work drew on them. One assumption — AS-01, the 30 LOC per function point used to convert function points to size — is flagged in §19.6 as requiring verification before submission, because the specific published table it came from was never recorded.

---

# 18. Conclusion

## 18.1 What was built

A functional, deployed web application that replaces the paper susu card with a record both the client and the collector can see and neither can silently alter. Thirty functional requirements across four roles, 3,433 lines of application code, 226 automated tests at 97% coverage, live at https://susubook-fdtbppd7sq-uc.a.run.app.

## 18.2 Against the objectives

| | Objective | Outcome |
|---|---|---|
| **O1** | Elicit, analyse, prioritise requirements; produce an SRS | **Met** — 40 requirements, MoSCoW-prioritised, traceability matrix, IEEE 830 SRS. Limited by the absence of primary elicitation (§17.1) |
| **O2** | Estimate effort and use it to define achievable scope | **Met** — FPA → COCOMO → PERT triangulated; the estimate exposed a 23% over-commitment *before* implementation and drove the scope decision |
| **O3** | Design a maintainable layered architecture with UML | **Met** — four layers, eight UML artefacts, SOLID applied at named sites |
| **O4** | Implement the prioritised requirements as a deployed application | **Met** — deployed, with authentication, role-based authorisation, validation and an append-only audit trail |
| **O5** | Verify through unit, integration, system and acceptance testing | **Partially met** — 226 tests across three levels; **UAT not conducted** |
| **O6** | Identify, classify and document technical debt with a repayment plan | **Met** — 17 items, each with cause, impact, priority and resolution; 13 identified before the code carrying them existed |
| **O7** | Define maintenance and evolution grounded in theory | **Met** — four maintenance categories, Lehman's eight laws applied individually, six-release roadmap |

Six of seven met; O5 partially, and the shortfall is named rather than glossed.

## 18.3 What the project demonstrates

**Estimation is worth doing even when the number is wrong.** COCOMO put this at 1,969 person-hours against a 48-hour window — a ratio of no operational use. Its value was not the figure but what the figure forced: an explicit scope decision, taken in advance, with a stated basis. The estimate also generated the

technical debt register, because every quality reduction made to fit the budget became a debt entry with a known cause. The two sections that carry the most marks turned out to be the same decision documented twice.

**Debt taken knowingly behaves differently from debt taken accidentally.** Of 17 items, 13 were identified before the code carrying them existed. All 13 landed in the *acceptable* or *scheduled* bands. All three **Critical** items were discovered later — during implementation and testing — and none was a deliberate trade-off. The dangerous debt was the debt nobody decided to take on.

**Architecture is a scheduling decision.** Keeping the domain layer free of Flask and SQLAlchemy reads as purity. It was the reason 129 unit tests run in 0.34 seconds with no database, and therefore the reason a real test suite was affordable at all inside the window.

**Some defects are only reachable by a person.** DEF-06 — a route row marked "Paid" above a total still reading GHS 0.00 — was found by clicking through the interface, not by any of the 226 tests, and could not have been: every test asserted the *response* to a request and none asserted the state of the page afterwards. DEF-08 was similar in kind, invisible outside a real deployment. Lehman's eighth law describes evolution as a feedback system; this project produced its own evidence for it rather than quoting the law.

**Honest reporting is more useful than favourable reporting.** NFR-01 is recorded as not met, with arithmetic. UAT is recorded as not done. Two of the developer's own test assumptions were found wrong and corrected rather than adjusted until they passed. A flaky security test was fixed and then demonstrated stable over five runs rather than assumed. Each of these could have been quietly omitted, and the document would have been weaker for it — a claim nobody can check is worth less than a limitation anyone can verify.

### 18.4 If the work continued

The order is already fixed by §11.6: migrations, rate limiting, forced password change, audit enforcement — 8 to 11 hours — before the system touches real client money. Then SMS notification, which mitigates the one assumption the value proposition depends on.

The system as delivered demonstrates that the problem is solvable. It does not yet solve it for anyone.

---

## 19. References

### 19.1 Course materials


1. Mensah, S. (2025) *CSCD602 Session 1: Introduction to Software Engineering*. Department of Computer Science, University of Ghana.
2. Mensah, S. (2025) *CSCD602 Session 2: Requirements Engineering*. Department of Computer Science, University of Ghana.
3. Mensah, S. (2025) *CSCD602 Session 3: Technical Debt*. Department of Computer Science, University of Ghana.
4. Mensah, S. (2025) *CSCD602 Session 4: Program Evolution Dynamics*. Department of Computer Science, University of Ghana.

5. Mensah, S. (2025) *CSCD602 Session 5: Software Design and Architecture*. Department of Computer Science, University of Ghana.
6. Mensah, S. (2025) *CSCD602 Session 6: Software Effort Estimation*. Department of Computer Science, University of Ghana.
7. Mensah, S. (2025) *CSCD602 Advanced Software Engineering: Course Syllabus, First Semester 2025/2026*. Department of Computer Science, University of Ghana.

## 19.2 Books

8. Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson Education.
9. Farley, D. (2021) *Modern Software Engineering: Doing What Works to Build Better Software Faster*. Boston: Addison-Wesley.
10. Tsui, F., Karam, O. and Bernal, B. (2022) *Essentials of Software Engineering*. Burlington: Jones & Bartlett Learning.
11. Boehm, B.W. (1981) *Software Engineering Economics*. Englewood Cliffs: Prentice Hall.
12. Boehm, B.W., Abts, C., Brown, A.W., Chulani, S., Clark, B.K., Horowitz, E., Madachy, R., Reifer, D.J. and Steece, B. (2000) *Software Cost Estimation with COCOMO II*. Upper Saddle River: Prentice Hall.
13. Brooks, F.P. (1975) *The Mythical Man-Month: Essays on Software Engineering*. Reading: Addison-Wesley.
14. Lehman, M.M. and Belady, L.A. (1985) *Program Evolution: Processes of Software Change*. London: Academic Press.
15. Martin, R.C. (2003) *Agile Software Development: Principles, Patterns, and Practices*. Upper Saddle River: Prentice Hall.
16. Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.
17. Evans, E. (2003) *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston: Addison-Wesley.
18. Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading: Addison-Wesley.

## 19.3 Papers and articles

19. Lehman, M.M. (1980) 'Programs, life cycles, and laws of software evolution', *Proceedings of the IEEE*, 68(9), pp. 1060–1076.
20. Albrecht, A.J. (1979) 'Measuring application development productivity', *Proceedings of the IBM Applications Development Symposium*, pp. 83–92.
21. Cunningham, W. (1992) 'The WyCash portfolio management system', *OOPSLA '92 Experience Report*.
22. Brooke, J. (1996) 'SUS: A quick and dirty usability scale', in Jordan, P.W. et al. (eds) *Usability Evaluation in Industry*. London: Taylor & Francis, pp. 189–194.

## 19.4 Online sources

23. Fowler, M. (2009) *TechnicalDebtQuadrant*. Available at: https://martinfowler.com/bliki/TechnicalDebtQuadrant.html

24. Nielsen, J. (2000) *Why You Only Need to Test with 5 Users*. Nielsen Norman Group. Available at: https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/

25. Wiggins, A. (2017) *The Twelve-Factor App*. Available at: https://12factor.net/

26. Google Cloud (2024) *Cloud Run documentation: Connect to a VPC network*. Available at: https://cloud.google.com/run/docs/configuring/vpc-direct-vpc

## 19.5 Standards and legislation

27. IEEE (1998) *IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications*. New York: Institute of Electrical and Electronics Engineers.

28. ISO (2019) *ISO 9241-210:2019 Ergonomics of human-system interaction — Part 210: Human-centred design for interactive systems*. Geneva: International Organization for Standardization.

29. W3C (2018) *Web Content Accessibility Guidelines (WCAG) 2.1*. Available at: https://www.w3.org/TR/WCAG21/

30. Republic of Ghana (2012) *Data Protection Act, 2012 (Act 843)*. Accra: Ghana Publishing Company.

## 19.6 Note on assumption AS-01

**This reference is incomplete and must be resolved before submission.**

§4.3.1 converts function points to source size at **30 lines of code per function point** for Python with an ORM and server-side templating. That figure was taken as a published average for the language class, but **the specific table it came from was not recorded at the time**, and it is not cited here because citing a source that was not consulted would be worse than admitting the gap.

The figure is not incidental — it drives the KLOC input to COCOMO and therefore every effort figure in §4.3. Its unreliability is partly demonstrated by the project's own result: measured productivity was **20.7 LOC/FP** (§4.8.1), below even the optimistic bound of the sensitivity analysis.

**Action required:** consult a published language-productivity table — Jones's language levels or the QSM function point languages table are the usual sources — and either cite it here or restate AS-01 as an unsourced estimate. The sensitivity analysis in §4.3.4 already shows the conclusion holds across 25–40 LOC/FP, so the argument does not depend on the resolution.

## 19.7 Software and libraries acknowledged

Per examination Rule 6.

| Component | Version | Licence | Use |
|---|---|---|---|
| Python | 3.12 | PSF | Language |
| Flask | ≥3.0 | BSD-3-Clause | Web framework |
| Jinja2 | ≥3.1 | BSD-3-Clause | Templating |
| Flask-WTF | ≥1.2 | BSD-3-Clause | CSRF protection |
| SQLAlchemy | ≥2.0 | MIT | ORM |
| psycopg | ≥3.1 | LGPL-3.0 | PostgreSQL driver |
| PostgreSQL | 16 | PostgreSQL Licence | Database |
| argon2-cffi | ≥23.1 | MIT | Password hashing |
| segno | ≥1.6 | BSD-3-Clause | QR code generation |
| python-dotenv | ≥1.0 | BSD-3-Clause | Configuration |
| gunicorn | ≥21.2 | MIT | WSGI server |
| pytest, pytest-cov | ≥8.0, ≥5.0 | MIT | Testing |
| HTMX | 1.9.12 | BSD-2-Clause | Partial page updates |
| Tailwind CSS | 3.x (CDN) | MIT | Styling |
| Docker, Docker Compose | — | Apache-2.0 | Local database, containerisation |
| Google Cloud Run, Compute Engine, Secret Manager, Artifact Registry, Cloud Build | — | Commercial (free tier) | Hosting and deployment |
| Mermaid CLI | — | MIT | UML diagram rendering |

No third-party code was copied into this project. All application source in `app/` was written for this examination.

## 19.8 Declaration

Submitted as individual work for CSCD602, in accordance with examination Rules 1 and 13. No previously submitted academic work has been reused, so no disclosure is required under Rule 12.

**Student:** [STUDENT NAME] **Student ID:** [STUDENT ID]

# 6. Project Scope Definition

> Required by examination Part A §3.7 — "Define the scope of the system to be completed within the 48-hour period." This scope is a **consequence** of the estimation in `05-effort-estimation.md`, not a precursor to it.

## 6.1 How this scope was arrived at

1. All 38 functional requirements were specified without regard to the time budget (`03-requirements.md`).
2. They were MoSCoW-prioritised against a single test: *does this protect or make verifiable the client's funds?* (§3.6).
3. The Must subset was sized at 157 adjusted function points and estimated at **1,857 person-hours** by COCOMO, and at **24.5 hours** by PERT bottom-up (`05-effort-estimation.md`).
4. PERT's 24.5 hours exceeded the 20-hour implementation allocation by 23%, and the Cone of Uncertainty's upper bound (49 hours) would consume the entire examination window.
5. **4.1 hours of implementation quality** were therefore removed in advance, in areas that do not touch BR-01 or BR-02, giving a 20.4-hour plan.
6. Change request **CR-001** subsequently added QR-based client identification at +0.92 h, taking the plan to **21.3 hours against the 20-hour allocation — a 6.4% overrun, deliberately accepted and logged** rather than absorbed by manufacturing further cuts.

The scope below is the output of that process.

## 6.2 In scope — will be built and deployed

| Area | Delivered capability | FRs |
|---|---|---|
| **Authentication & authorisation** | Login with hashed passwords, four roles, server-side route authorisation, collector restricted to own clients, session timeout | FR-01…05 |
| **Client management** | Enrol client with daily rate, assign to collector, list clients | FR-06…08 |
| **Contribution cycles** | Auto-open 31-day cycle on enrolment and after payout; maturity transition | FR-10 |
| **Contribution recording** | Record contribution with full validation — duplicate, date range, future date, closed cycle, rate multiple | FR-11…14 |
| **Digital susu card** | 31-day grid showing paid / missed / pending; computed days paid, total, projected payout | FR-16, FR-17 |
| **Payout** | Commission computation, zero-payout edge case, supervisor release, double-payout prevention | FR-18…21 |
| **Reconciliation** | Daily route sheet, remittance declaration, variance computation, supervisor variance list | FR-23…26 |
| **Client transparency** | Client login showing every contribution with amount, date, time, recording collector and reference; balance and projected payout | FR-28…30 |
| **Audit** | Append-only audit log of every state change; reversal instead of edit or delete | FR-32, FR-33 |
| **Client identification** *(CR-001)* | Opaque UUID public reference per client; printable QR card encoding the contribution URL; scan-to-collect via the phone's native camera | FR-39, FR-40 |

**30 functional requirements · 10 non-functional requirements.**

> FR-39 and FR-40 entered scope after the baseline, under CR-001. They are **Should** priority: if Phase 3 runs behind they are the first work abandoned, costing no Must requirement. The opaque-reference rule BR-R14 is retained either way, being a security improvement independent of the QR feature.

## 6.3 Reduced in quality — built, but knowingly below standard

These are delivered but deliberately degraded to recover the 4.1 hours. Each is a technical debt entry, not an omission.

| Area | Delivered as | Full implementation would be | Debt ID |
|---|---|---|---|
| Schema management | `create_all()` plus seed script | Alembic versioned migrations | TD-01 |
| Styling | Tailwind via CDN, minimal custom CSS | Built and purged stylesheet | TD-02 |
| Client list | Plain table | Search, filter and pagination | TD-03 |
| Correction workflow | Minimal supervisor-only reversal form | Guided correction with reason codes | TD-04 |
| Route sheet | Static list | Filtered to "not yet collected", ordered by route | TD-05 † |
| Interaction model | HTMX on three key interactions, full page loads elsewhere | Consistent partial updates throughout | TD-06 |

FR-08 and FR-23 are therefore **partially** satisfied, and are reported as such in `09-testing.md` rather than claimed as complete.

† **TD-05 is partially mitigated by CR-001.** QR scanning bypasses the route sheet for the common case, so the degraded static list is exercised far less than originally assumed. The debt is retained at reduced impact rather than closed, because the fallback path still matters whenever a card is lost, damaged or left at home.

## 6.4 Out of scope — specified but not built

| Deferred | Priority | Why | Where it goes |
|---|---|---|---|
| FR-09 Client reassignment | Should | Not needed to demonstrate the core loop | Evolution plan |
| FR-15 Catch-up contribution | Should | Real and common, but the allocation logic is a meaningful build | Evolution plan |
| FR-22 Early withdrawal | Should | Requires an approval workflow beyond the core cycle | Evolution plan |
| FR-27 Variance resolution notes | Could | Detection is the value; resolution workflow is secondary | Evolution plan |
| FR-34 Audit trail viewer | Should | Log is written and complete; only the UI is deferred | Evolution plan |
| FR-35 User administration UI | Should | Accounts seeded directly; no UI within budget | Evolution plan |
| FR-36 Configurable defaults | Could | Cycle length and commission are constants for now | Evolution plan |
| FR-37 Supervisor dashboard | Could | Variance list carries the essential signal | Evolution plan |
| FR-38 CSV export | Could | Not required to demonstrate any principle | Evolution plan |
| **FR-31 SMS notification** | **Won't** | **Requires a paid gateway (CO-04) with account lead time exceeding CO-01** | **First item in evolution plan** |

## 6.5 Explicitly excluded from the product vision

Not deferred — outside what this system is:

- **Mobile money / bank integration** — no merchant credentials obtainable (CO-05). SusuBook records cash movement; it does not effect it.
- **Offline operation** — a genuine field requirement given mobile-data coverage, but a service worker and conflict-resolved sync is a project in itself.
- **Multi-institution tenancy** — the data model assumes one institution.
- **Native mobile applications** — a responsive web application meets CO-07 at a fraction of the cost.

## 6.6 What protects this scope

The four items below were ring-fenced during the cut and are delivered at full quality, because they are the reason the system exists:

| Protected | Hours | Carries |
|---|---|---|
| Domain layer with all business rules BR-R1…R13 | 2.58 | Correctness of every computation |
| Contribution recording with complete validation | 2.58 | BR-01 — prevention of under-recording |
| Append-only audit log | 1.08 | BR-05 — non-repudiation |
| Client self-service views | 1.08 | BR-02 — the independent client record |

If the schedule slips further, quality is reduced elsewhere again. These four are not available to be cut, because without them the system no longer addresses the problem in `01-problem-definition.md`.

## 6.7 Definition of Done

A requirement counts as delivered only when all of the following hold:

1. The business rule is implemented in the domain layer and unit-tested.
2. The route enforces role-based authorisation server-side.
3. Input is validated and errors are returned to the user intelligibly.
4. The state change is written to the audit log.
5. A test case exists in `09-testing.md` with a recorded actual result.
6. It works against the deployed instance, not only locally.

Anything falling short of all six is reported as partial in the testing document.

# Requirements Change Log

Changes to the requirements baseline are recorded here under the formal change control process defined in `03-requirements.md` §3.8. Nothing is removed from or added to the baseline silently; each entry records the request, its impact, its cost, the decision taken and who took it.

---

## CR-001 — QR-based client identification

| Field | Value |
|---|---|
| **Raised** | Phase 1 close, before Analysis & Design |
| **Raised by** | Developer |
| **Status** | **Approved and incorporated** |
| **Affects** | `03-requirements.md`, `04-srs.md`, `05-effort-estimation.md`, `06-scope.md` |

### 1. Change request

Issue each client a printed QR code. The collector scans it to reach that client's contribution screen directly, rather than locating the client in a list.

**Stated reason.** Collector throughput. A collector working a market may serve dozens of clients in a morning; finding each one in a list is the dominant cost of the interaction and works directly against NFR-02. The paper susu card is already a physical artefact the client keeps, so a printed QR card preserves a familiar ritual rather than imposing a new one — a user-centred design argument (Session 5) as much as an efficiency one.

### 2. Options considered

| Option | Description | PERT effort | Assessment |
|---|---|---|---|
| **A** | In-app camera scanner: `getUserMedia`, JS QR-decoding library, live viewfinder | **2.12 h** | Rejected — see §3 |
| **B** | **QR encodes a URL; collector uses the phone's native camera app, which opens the client's contribution screen** | **0.92 h** | **Selected** |
| C | Opaque public references only, QR deferred to evolution | ~0.1 h | Rejected — forgoes the throughput benefit that motivated the request |
| D | Reject entirely | 0 h | Rejected — the benefit is real and the cost is modest |

**Why Option B over Option A.** Option A costs 2.3× more, introduces a client-side JavaScript dependency into a stack deliberately kept JS-light, and requires camera permission handling. Its field reliability is also the weaker of the two: live camera decoding on a low-end Android in strong outdoor light, aimed at a printed card subjected to weeks of market conditions, is a meaningful failure risk (CO-07). Option B delegates decoding to the operating system's camera application, which every current Android and iOS device performs natively, and reduces the routine collection to **two interactions** — better than NFR-02 requires. Both options degrade to the same fallback: the route sheet.

## 3. Impact analysis

Traced through the matrix at `03-requirements.md` §3.7.

### Requirements added

| ID | Requirement | Priority |
|---|---|---|
| FR-39 | The system shall assign each client an opaque, non-sequential public reference and render it as a printable QR code encoding the client's contribution URL. | Should |
| FR-40 | The system shall resolve a scanned client reference to that client's contribution screen, subject to the same authorisation as any other route. | Should |

### Business rules added

| ID | Rule |
|---|---|
| BR-R14 | Public references exposed in URLs or QR codes shall be opaque and non-sequential (UUIDv4). Sequential database identifiers shall never appear in a URL. |
| BR-R15 | A client reference identifies; it does not authorise. Possession of a reference confers no permission; authorisation remains the collector–client assignment enforced server-side (FR-05). |

**Security analysis.** A QR code on a card carried through a public market must be assumed to be photographable by anyone. Two consequences follow:

- *Enumeration.* A sequential identifier would let an observer derive every other client's reference by incrementing. BR-R14 requires UUIDv4, making references unguessable.
- *Privilege.* The reference must confer no capability. Scanning only reaches the contribution screen; recording a contribution still requires an authenticated collector to whom that client is assigned. BR-R15 states this explicitly so it is not eroded by a later change.
- *Data protection.* The QR encodes a URL containing an opaque reference and nothing else — no name, phone number, balance or rate. A photographed card leaks no personal data, preserving NFR-06 and Act 843 compliance.

### Requirements affected but not changed

| Requirement | Effect |
|---|---|
| NFR-02 (three-interaction limit) | Strengthened — scan-and-confirm is two interactions |
| FR-05 (collector restricted to own clients) | Unchanged, and now load-bearing for BR-R15 |
| FR-23 (route sheet) | Unchanged, but becomes the fallback path rather than the primary one |
| NFR-06 (data minimisation) | Preserved by the opaque-reference requirement |

**Design consequence beyond the change itself.** BR-R14 applies to the whole system, not only to QR codes. Opaque public references replace sequential identifiers in **all** externally visible URLs, closing an insecure-direct-object-reference exposure that existed in the original design. This is a security improvement to parts of the system the change request did not touch.

**Technical debt effect.** TD-05 (route sheet reduced to a static list) is **partially mitigated**: scanning bypasses the route sheet for the common case, so the degraded list is exercised less. TD-05 is retained at reduced impact rather than closed, because the fallback path still matters when a card is lost or damaged.

## 4. Cost, schedule and risk assessment

### Function point impact

| | Component | Complexity | FP |
|---|---|---|---|
| Added | Printable client QR card (External Output) | Average | 5 |
| Added | Client reference resolution (External Inquiry) | Simple | 3 |
| | | **Subtotal** | **+8 UFP** |

| Measure | Before | After | Δ |
|---|---|---|---|
| UFP (Must scope) | 143 | 151 | +8 |
| AFP (Must scope, VAF 1.10) | 157 | 166 | +9 |
| KLOC | 4.71 | 4.98 | +0.27 |
| COCOMO effort | 12.2 PM | 13.0 PM | +0.8 PM |
| COCOMO person-hours | 1,857 | 1,969 | +112 |

### PERT impact

New WBS task 17 — QR issuance and scan-to-collect: O = 0.5, M = 0.85, P = 1.6 → **E = 0.92 h**, σ = 0.18 h.

| Measure | Before | After |
|---|---|---|
| Total expected effort | 24.46 h | **25.38 h** |
| Total σ | 1.22 h | 1.24 h |
| 68% confidence range | 23.2 – 25.7 h | 24.1 – 26.6 h |
| Cone of Uncertainty (0.5×–2×) | 12.2 – 48.9 h | 12.7 – 50.7 h |
| After the 4.1 h of agreed cuts | 20.4 h | **21.3 h** |
| Against the 20 h allocation | +2% | **+6.4%** |

### Risk assessment

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Native camera app does not detect the QR on some device | Low | Low | Route sheet fallback is unchanged and always available |
| Printed card damaged or lost in market conditions | Medium | Low | Fallback to route sheet; reference re-issuance deferred to evolution |
| Task overruns its 0.92 h estimate | Medium | Medium | Feature is Should-priority — it is abandoned if Phase 3 runs late, without affecting any Must requirement |
| Scope creep toward a full in-app scanner | Medium | Medium | Option A is explicitly recorded as rejected here, with its cost |

## 5. Decision

**Approved**, with the schedule overrun accepted and recorded rather than absorbed.

The implementation plan now stands at **21.3 hours against a 20-hour allocation — a 6.4% overrun**, deliberately accepted. The alternative was to manufacture a further 1.3 hours of quality cuts so the arithmetic landed on exactly 20.0; that would have produced a tidier number and a less honest plan. Session 6 identifies external schedule pressure as a recognised source of estimate distortion, and adjusting an estimate to fit a predetermined budget is precisely that distortion. The overrun is therefore carried openly and tracked against actuals in `05-effort-estimation.md` §4.8.

**Containment.** FR-39 and FR-40 are **Should**, not Must. If Phase 3 runs behind, this is the first work abandoned, and abandoning it costs no Must requirement. BR-R14 (opaque references) is retained regardless of whether the QR feature ships, because it is a security improvement independent of it.

With no Change Control Board available on an individual assessment, the decision was taken by the developer and recorded here with its full justification so that it is reviewable — as specified in `03-requirements.md` §3.8 step 4.

## 6. Implementation record

- [x] `03-requirements.md` — FR-39, FR-40 added; MoSCoW counts updated; traceability matrix extended
- [x] `04-srs.md` — BR-R14, BR-R15 added; domain model updated with `public_ref`; UC-03 alternate flow 3b added; definitions extended

- [x] `05-effort-estimation.md` — WBS task 17 added; FPA, COCOMO and PERT figures revised; overrun recorded
- [x] `06-scope.md` — QR capability added to in-scope table; overrun recorded
- [ ] Implementation in Phase 3
- [ ] Test cases TC-QR-01…03 in `09-testing.md`